

ĐẠI HỌC BÁCH KHOA HÀ NỘI

LUẬN VĂN THẠC SỸ

**Nghiên cứu giải pháp phát hiện truy cập dữ liệu trái
phép ở cấp độ nhân hệ điều hành**

NGUYỄN MINH ĐỨC

duc.nm241052m@sis.hust.edu.vn

Chương trình: Thạc sỹ Kỹ thuật Máy tính

Giảng viên hướng dẫn: TS. Tống Văn Vạn

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

Hà Nội, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

LUẬN VĂN THẠC SỸ

**Nghiên cứu giải pháp phát hiện truy cập dữ liệu trái
phép ở cấp độ nhân hệ điều hành**

NGUYỄN MINH ĐỨC

duc.nm241052m@sis.hust.edu.vn

Chương trình: Thạc sỹ Kỹ thuật Máy tính

Giảng viên hướng dẫn: TS. Tống Văn Vạn _____

Chữ ký GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

Hà Nội, 06/2022

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG

Các máy chủ Linux thường trở thành mục tiêu của các hành vi sau xâm nhập như leo thang đặc quyền, thiết lập kết nối ngược hay đánh cắp dữ liệu. Vì các hành vi này đều phải tương tác với nhân hệ điều hành qua lời gọi hệ thống (syscall), chuỗi syscall là nguồn quan sát quan trọng cho các hệ thống phát hiện xâm nhập trên máy chủ (HIDS). Tuy nhiên, các hướng tiếp cận hiện tại đối mặt với một khoảng cách thực thi chính sách (enforcement gap): các mô hình học sâu như Transformer có khả năng học mẫu tuần tự tốt nhưng quá nặng để chạy trực tiếp trong kernel, trong khi các công cụ luật tĩnh như Falco hay Tetragon có chi phí thấp nhưng phụ thuộc vào tri thức chuyên gia và khó bao phủ đầy đủ các biến thể tấn công.

Luận văn đề xuất Guepard Shield, một hệ thống HIDS lai lắp khoảng trống đó bằng cách tách giai đoạn học biểu diễn ngoại tuyến ra khỏi giai đoạn thực thi thời gian thực. Hướng tiếp cận là chưng cất tri thức từ Transformer Teacher sang một automata hữu hạn đơn định (DFA) Student đủ nhỏ để nạp vào BPF map và truy vấn với độ phức tạp $O(1)$ mỗi syscall. DFA vừa phù hợp với ràng buộc của eBPF Verifier, vừa bảo toàn tri thức ngữ cảnh mà Teacher đã học.

Pipeline gồm bốn giai đoạn: token hóa tên syscall và tạo cửa sổ trượt (Giai đoạn 1), huấn luyện Decoder-only Transformer trên dữ liệu bình thường (Giai đoạn 2), rời rạc hóa vector trạng thái ẩn bằng K-Means để xây dựng bảng chuyển DFA (Giai đoạn 3), và nạp bảng chuyển vào BPF map để thực thi qua eBPF (Giai đoạn 4).

Thực nghiệm trên LID-DS-2021 cho thấy Teacher đạt AUROC = 0.8503 và PR-AUC = 0.7093 trên gần 100 triệu window kiểm thử không dùng nhãn tấn công khi huấn luyện. DFA Student tại $K = 64$ tạo ra 64 trạng thái và 1.973 bước chuyển (204 KB), đạt tỷ lệ phát hiện cấp độ bản ghi 90.85%, tỷ lệ báo động giả 1.41% và fidelity 98.12% so với Teacher, xác nhận tính khả thi của hướng chưng cất Transformer sang DFA cho bài toán giám sát syscall thời gian thực.

Học viên

(Signature and full name)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Bối cảnh và các vấn đề nghiên cứu	2
1.3 Mục tiêu nghiên cứu và Khung khái niệm	3
1.4 Ứng dụng của nghiên cứu	5
1.5 Các đóng góp dự kiến.....	5
1.6 Bố cục của luận văn	6
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	8
2.1 Phạm vi nghiên cứu (Scope of Research)	8
2.2 Các nghiên cứu liên quan (Related Work)	10
2.2.1 Phát hiện bất thường dựa trên chuỗi lời gọi hệ thống.....	10
2.2.2 Trích xuất luật khả diễn và Automata từ mạng nơ-ron.....	11
2.2.3 Thực thi chính sách bảo mật thời gian thực qua eBPF	12
2.3 Mạng Transformer Decoder-only.....	14
2.4 Automata hữu hạn đơn định (DFA).....	15
2.5 Công nghệ eBPF (Extended Berkeley Packet Filter).....	16
2.5.1 Sự tiến hóa từ Classic BPF đến eBPF	16
2.5.2 Cơ chế hoạt động và Điểm móc (Hooks)	16
2.5.3 Trình xác thực eBPF (Verifier) và Tính an toàn.....	17
2.5.4 Cấu trúc BPF Maps và Lưu trữ trạng thái	17
2.5.5 Tính di động và eBPF Ecosystem (BTF & CO-RE)	17
2.5.6 Cập nhật nguyên tử và Nạp nóng (Atomic Update & Hot-reload)....	18
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	19
3.1 Tổng quan kiến trúc	19

3.2 Giai đoạn 1 - Tiền xử lý dữ liệu.....	21
3.2.1 Token hóa tên lời gọi hệ thống.....	21
3.2.2 Phân đoạn cửa sổ trượt và gán nhãn	22
3.3 Giai đoạn 2 - Huấn luyện Transformer Teacher.....	23
3.3.1 Kiến trúc mô hình	23
3.3.2 Điểm bất thường dựa trên NLL của token cuối.....	27
3.4 Giai đoạn 3 - Chứng cất DFA Student.....	28
3.4.1 Định nghĩa và trích xuất trạng thái ẩn.....	28
3.4.2 Rời rạc hóa trạng thái bằng K-Means	31
3.4.3 Xây dựng bảng chuyển trạng thái.....	32
3.4.4 Giải quyết tính không xác định.....	32
3.4.5 Giao thức đánh giá DFA	34
3.4.6 Xuất cấu hình DFA.....	35
3.5 Giai đoạn 4 - Thực thi tại Runtime bằng eBPF.....	37
3.5.1 Bước DFA trên mỗi sự kiện syscall	37
3.5.2 Chính sách xử lý theo mức độ nghi ngờ.....	39
3.5.3 Khả năng kháng mimicry attacks.....	39
3.5.4 Vòng lặp cập nhật liên tục	40
CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT.....	41
4.1 Độ phức tạp tính toán (Computational Complexity)	41
4.1.1 Transformer Inference (Ngoại tuyến)	41
4.1.2 Duyệt DFA (Runtime)	42
4.1.3 Trích xuất K-Means (Ngoại tuyến).....	43
4.2 Kích thước DFA và Bộ nhớ eBPF (DFA Size and eBPF Memory).....	45
4.3 Tỷ lệ không xác định và Lựa chọn chiến lược (Conflict Rate and Policy Selection).....	46

4.4 Khả năng kháng Mimicry Attack (Resistance to Mimicry Attack)	48
4.5 Giới hạn độ trung thực của DFA (DFA Fidelity Bound).....	49
CHƯƠNG 5. THỰC NGHIỆM	51
5.1 Bộ dữ liệu	51
5.2 Thiết lập đánh giá	52
5.3 Các chỉ số đánh giá	53
5.4 Kết quả Giai đoạn 2: Transformer Teacher	54
5.4.1 Cấu hình huấn luyện.....	54
5.4.2 Hội tụ huấn luyện.....	54
5.4.3 Đánh giá cấp độ window.....	55
5.4.4 Nhiễu nhân trong LID-DS và ý nghĩa đánh giá	57
5.4.5 Đánh giá cấp độ bản ghi	57
5.5 Kết quả Giai đoạn 3: Chứng cất DFA Student.....	58
5.5.1 Thiết lập trích xuất DFA	58
5.5.2 So sánh chiến lược giải quyết tính không xác định.....	58
5.5.3 Kích thước DFA và tính khả thi eBPF	61
5.5.4 Giới hạn của thí nghiệm Giai đoạn 3	62
5.6 Kết quả Giai đoạn 4 — Độ trễ Runtime eBPF DFA.....	63
5.6.1 Thiết lập thực nghiệm.....	63
5.6.2 E1 — Histogram Latency eBPF DFA.....	64
5.6.3 E2 — Latency Transformer CPU	64
5.6.4 E3 — Overhead Throughput nginx	65
5.6.5 So sánh tổng hợp.....	66
5.7 Thảo luận tổng hợp	67

CHƯƠNG 6. KẾT LUẬN	69
6.1 Tổng kết.....	69
6.1.1 Các kết quả đã đạt được	69
6.1.2 Các vấn đề còn tồn đọng	70
6.2 Hướng phát triển trong tương lai	71
TÀI LIỆU THAM KHẢO.....	74

DANH MỤC HÌNH VẼ

Hình 3.1	Sơ đồ pipeline bốn giai đoạn của hệ thống Guepard Shield . . .	20
Hình 3.2	Cơ chế tương tác động và vòng lặp cập nhật giữa Kernel-space và User-space	20
Hình 3.3	Minh họa phân đoạn cửa sổ trượt với $W = 4, S = 2$. Các ô nền đỏ là các syscall xảy ra sau thời điểm <code>exploit_start</code> . Window w_3 nhận nhãn Attack vì timestamp của syscall cuối cùng của nó nằm sau mốc khai thác; w_1 và w_2 nhận nhãn Normal dù có gói đầu với vùng attack.	23
Hình 3.4	Kiến trúc Transformer Decoder-only của mô hình Teacher với $L=4$ lớp ($d=128, h=4, d_{ff}=512$). Cấu trúc Pre-Norm đặt RMSNorm trước mỗi sub-layer, cải thiện ổn định gradient trong huấn luyện sâu. Mặt nạ nhân quả (causal mask) trong MHA đảm bảo h_t chỉ phụ thuộc ngữ cảnh $(s_1, \dots, s_t)$, phản ánh đúng tính nhân quả của luồng syscall tại runtime. Trọng số embedding và projection được ràng buộc chung (weight tying) để giảm số tham số.	26
Hình 3.5	Quá trình rời rạc hóa trạng thái ẩn Transformer thành các trạng thái DFA thông qua gom cụm K-Means. Mỗi tâm cụm c_k trong không gian $\mathbb{R}^d$ trở thành một trạng thái $q_k \in Q$ của DFA. . . .	32
Hình 3.6	Ví dụ xây dựng quan hệ chuyển đổi từ một đoạn chuỗi token. K-Means gán mỗi vector trạng thái ẩn h_t vào một cụm (hàng giữa). Mỗi cặp liên tiếp $(q_i, s_{t+1}) \rightarrow q_j$ đóng góp một ứng viên vào bảng chuyển trạng thái thô (hàng dưới). Hai bước chuyển liên tiếp $(q_2, \text{write}) \rightarrow q_5$ và $(q_5, \text{open}) \rightarrow q_5$ minh họa trường hợp một trạng thái tự lặp sau khi nhận token đầu vào mới.	33
Hình 3.7	Minh họa chiến lược S4 Statistical Pruning. Các bước chuyển (ví dụ minh họa; tên rút gọn: r=read, w=write, m=mmap, o=open, f=futex, c=clone) được sắp xếp giảm dần theo tần suất. Đường ngưỡng θ loại bỏ các nhánh thiểu số (màu đỏ), vốn là tạo vật của lượng tử hóa K-Means thay vì hành vi thực sự. Phân phối lệch nặng về nhóm bước chuyển chính là đặc trưng điển hình của workload máy chủ lặp lại cao.	34

Hình 5.1	Pipeline đánh giá thực nghiệm. Transformer Teacher tạo điểm bất thường liên tục bằng NLL, trong khi DFA Student thay điểm số liên tục bằng kiểm tra bước chuyển trạng thái đã từng xuất hiện trong dữ liệu huấn luyện bình thường.	53
Hình 5.2	Đường cong hội tụ của Transformer Teacher. Validation loss đạt giá trị tốt nhất ở epoch 1 và sau đó đi vào vùng plateau hẹp. . . .	55
Hình 5.3	Đường cong ROC của điểm last-token NLL ở cấp độ window. Diện tích dưới đường cong đạt 0.8503, cho thấy Teacher xếp hạng window attack cao hơn window normal trong phần lớn các cặp so sánh, dù nhãn attack của LID-DS có nhiều cấu trúc.	56
Hình 5.4	So sánh precision và recall attack tại ba cách chọn ngưỡng. Ngưỡng oracle thể hiện năng lực tiềm năng của điểm NLL, nhưng không phải ngưỡng vận hành vì cần nhãn test.	56
Hình 5.5	Cấu trúc nhiều nhãn trong các window được LID-DS gán nhãn attack. Phần tín hiệu exploit có NLL cao chỉ chiếm một tỷ lệ rất nhỏ; phần lớn tail sau khai thác có hành vi giống normal.	57
Hình 5.6	Minh họa bước K-Means trong Giai đoạn 3. Các vector hidden state của Teacher được gom quanh các centroid, sau đó mỗi centroid được xem như một trạng thái rời rạc của DFA. Bảng chuyển $\delta(q, s)$ được xây dựng bằng cách quan sát các cặp trạng thái liên tiếp trong chuỗi syscall bình thường.	58
Hình 5.7	Đánh đổi giữa FPR và DR_{rec} của các chiến lược DFA. S4 đạt DR_{rec} tuyệt đối bằng cách gán attack cho gần như toàn bộ window normal, nên không thể dùng trong triển khai.	59
Hình 5.8	Một graph con được lấy trực tiếp từ bảng chuyển DFA S3 tại $K = 64$. Mỗi cạnh gom các dòng có cùng cặp state nguồn-đích; nhãn trên cạnh có dạng mã cạnh / số syscall token. Start state của bảng chuyển là q_{59} và toàn bộ DFA S3 có 1,973 transitions.	60
Hình 5.9	Biểu diễn bảng chuyển DFA dưới dạng mảng trực tiếp. Kích thước của cấu hình S3 đủ nhỏ để nạp vào BPF map và cho phép tra cứu trạng thái kế tiếp bằng một phép chỉ mục hóa hằng số.	62
Hình 5.10	CDF latency eBPF DFA trong điều kiện E3 embedded (34.550.232 samples, nginx worker only). Mỗi bước trên trục x tương ứng một bucket 100 ns. CDF nhảy từ 0% lên 93,87% ngay tại mốc 100 ns, cho thấy hơn 9 trong 10 syscall hoàn thành DFA lookup trong dưới 100 ns.	64

Hình 5.11	Throughput nginx (req/s) trong hai điều kiện E3. Gắn eBPF DFA làm giảm throughput 10,97%, từ 92.210 xuống 82.091 req/s. .	65
Hình 5.12	So sánh latency DFA (eBPF) và Transformer (CPU) tại p50, p99 và p999 trên thang $\log_{10}$. Giá trị DFA là trung điểm của bucket tương ứng trong histogram E1. DFA thấp hơn Transformer hơn 4 bậc độ lớn ở mọi phân vị.	67

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các phương pháp phát hiện xâm nhập dựa trên syscall	13
Bảng 3.1	Ví dụ bộ từ vựng syscall: ánh xạ từ tên lời gọi hệ thống sang chỉ số token nguyên. Phần lớn token ID tương ứng với các syscall xuất hiện thường xuyên trong khối lượng công việc máy chủ; token 0 dành riêng cho <UNK>.	22
Bảng 3.2	Các chiến lược giải quyết tính không xác định trong quá trình trích xuất DFA	33
Bảng 3.3	Cấu trúc ánh xạ từ thành phần DFA sang các BPF Maps trong kernel	35
Bảng 3.4	Ba mức phản ứng của chương trình eBPF tại runtime.	39
Bảng 5.1	Các nhóm kịch bản tiêu biểu trong LID-DS-2021.	52
Bảng 5.2	Phân chia bản ghi trong LID-DS-2021 được sử dụng trong thực nghiệm.	53
Bảng 5.3	Diễn biến train loss và validation loss của Transformer Teacher.	55
Bảng 5.4	Kết quả đánh giá last-token NLL ở cấp độ window trên tập test.	55
Bảng 5.5	So sánh các chiến lược chứng cất DFA tại $K = 64$.	59
Bảng 5.6	Chú giải các cạnh chính trong graph con DFA ở Hình 5.8.	60
Bảng 5.7	Kết quả E1 — latency eBPF DFA theo hai điều kiện đo.	64
Bảng 5.8	Kết quả E2 — latency single-sample Transformer inference trên CPU (10.000 mẫu, 100 warmup).	65
Bảng 5.9	Kết quả E3 — overhead throughput nginx live workload, đo hai điều kiện tuần tự trên cùng nginx instance.	65
Bảng 5.10	So sánh latency và chi phí CPU giữa DFA (eBPF) và Transformer (CPU inline) tại 647k syscalls/s.	66

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AUROC	Diện tích dưới đường cong ROC (Area Under the Receiver Operating Characteristic Curve)
BPE	Mã hóa cặp byte, một thuật toán nén từ vựng (Byte-Pair Encoding)
BPF	Berkeley Packet Filter, nền tảng gốc của eBPF
BRL	Danh sách luật Bayes (Bayesian Rule Lists)
CPU	Bộ xử lý trung tâm (Central Processing Unit)
CVE	Danh mục lỗ hổng bảo mật phổ biến (Common Vulnerabilities and Exposures)
CWE	Danh mục điểm yếu phần mềm phổ biến (Common Weakness Enumeration)
DFA	Automata hữu hạn đơn định (Deterministic Finite Automaton)
eBPF	extended Berkeley Packet Filter, cơ chế chạy chương trình kiểm tra an toàn trong nhân Linux
F1	Thước đo trung hòa giữa precision và recall
FPR	Tỷ lệ dương tính giả (False Positive Rate)
GPU	Bộ xử lý đồ họa, thường dùng để tăng tốc huấn luyện mô hình học sâu (Graphics Processing Unit)
GRU	Gated Recurrent Unit, một biến thể gọn hơn của mạng nơ-ron hồi quy
HIDS	Hệ thống phát hiện xâm nhập trên máy chủ (Host-based Intrusion Detection System)

Thuật ngữ	Ý nghĩa
HMM	Mô hình Markov ẩn (Hidden Markov Model)
IDS	Hệ thống phát hiện xâm nhập (Intrusion Detection System)
KD	Chưng cất tri thức (Knowledge Distillation)
LID-DS	Leipzig Intrusion Detection Data Set, bộ dữ liệu phát hiện xâm nhập Leipzig phục vụ cho đánh giá HIDS
LIME	Phương pháp giải thích cục bộ không phụ thuộc mô hình (Local Interpretable Model-agnostic Explanations)
LSM	Mô-đun bảo mật Linux (Linux Security Modules)
LSTM	Long Short-Term Memory, một biến thể của mạng nơ-ron hồi quy
LTL	Logic thời gian tuyến tính (Linear Temporal Logic)
MITRE ATT&CK	Khung tri thức mô tả chiến thuật và kỹ thuật tấn công mạng trong thực tế
NFA	Automata hữu hạn không đơn định (Nondeterministic Finite Automaton)
NLL	Độ mất mát log âm (Negative Log-Likelihood)
OGNL	Ngôn ngữ điều hướng đồ thị đối tượng (Object-Graph Navigation Language)
OOV	Từ ngoài từ vựng (Out-of-Vocabulary)
PR-AUC	Diện tích dưới đường cong Precision-Recall (Precision-Recall Area Under the Curve)
RNN	Mạng nơ-ron hồi quy (Recurrent Neural Network)

Thuật ngữ	Ý nghĩa
SHAP	Phương pháp giải thích giá trị Shapley cộng gộp (SHapley Additive exPlanations)
STIDE	Sequence Time-Delay Embedding, phương pháp phát hiện bất thường dựa trên chuỗi con syscall
STL	Logic thời gian tín hiệu (Signal Temporal Logic)
syscall	Lời gọi hệ thống, giao diện giữa tiến trình người dùng và nhân hệ điều hành
TID	Định danh luồng thực thi (Thread ID)
TPR	Tỷ lệ dương tính thật (True Positive Rate)
XAI	Trí tuệ nhân tạo giải thích được (Explainable Artificial Intelligence)

CHAPTER 1. GIỚI THIỆU ĐỀ TÀI

Chương 1 tập trung vào bài toán phát hiện xâm nhập trên máy chủ Linux thông qua chuỗi lời gọi hệ thống, các giới hạn của những hướng tiếp cận hiện tại, và định hướng xây dựng Guepard Shield như một cơ chế lai giữa mô hình học sâu và thực thi chính sách nhẹ trong nhân hệ điều hành. Cách trình bày trong chương này nhằm làm rõ vì sao bài toán vừa cần khả năng học từ dữ liệu, vừa cần một dạng biểu diễn đủ đơn giản để có thể được kiểm tra tại thời điểm chạy.

1.1 Đặt vấn đề

Các hệ thống máy chủ Linux hiện là nền tảng vận hành của nhiều dịch vụ quan trọng, từ máy chủ web, cơ sở dữ liệu đến các vi dịch vụ (microservices) trong môi trường điện toán đám mây. Vì các hệ thống này xử lý dữ liệu và tài nguyên có giá trị, chúng thường trở thành mục tiêu của các hành vi sau xâm nhập (post-exploitation), chẳng hạn như thiết lập kết nối ngược (reverse shells), leo thang đặc quyền (privilege escalation), đánh cắp dữ liệu (data exfiltration) hoặc duy trì sự hiện diện lâu dài trong hệ thống (persistence). Dù khác nhau về mục tiêu cụ thể, các hành vi này đều cần tương tác với nhân hệ điều hành thông qua giao diện lời gọi hệ thống (system calls - syscalls). Vì vậy, chuỗi syscall cung cấp một nguồn quan sát quan trọng để mô tả hành vi của tiến trình ở ranh giới giữa không gian người dùng (user space) và không gian hạt nhân (kernel space).

Sự dịch chuyển mạnh mẽ sang kiến trúc vi dịch vụ và điện toán đám mây (cloud-native) mang đến những thách thức mới cho công tác giám sát an ninh. Trong các môi trường này, các tiến trình thường có vòng đời ngắn, được triển khai dưới dạng các container và có hành vi tương tác mạng phức tạp. Các giải pháp bảo mật truyền thống dựa trên vùng biên (perimeter-based security) như tường lửa hay IDS mạng dần bộc lộ hạn chế khi không thể quan sát được lưu lượng trao đổi nội bộ giữa các dịch vụ trong cùng một cụm máy chủ hoặc các hành vi khai thác lỗ hổng trực tiếp trên ứng dụng. Điều này thúc đẩy nhu cầu về các hệ thống giám sát tại chỗ (host-based), có khả năng bám sát thực thi của từng tiến trình để phát hiện sớm các dấu hiệu bất thường ngay khi cuộc tấn công vừa vượt qua lớp bảo vệ bên ngoài.

Để bảo vệ máy chủ, các hệ thống phát hiện xâm nhập trên máy chủ (Host-based Intrusion Detection Systems - HIDS) dựa trên syscall đã được nghiên cứu trong nhiều năm. Tuy nhiên, khi chuyển từ phát hiện ngoại tuyến sang giám sát tại thời điểm chạy, các hệ thống này thường gặp một đánh đổi quan trọng giữa khả năng mô hình hóa hành vi phức tạp và chi phí thực thi. Thách thức lớn nhất nằm ở việc các cuộc tấn công hiện đại ngày càng tinh vi hơn, sử dụng các kỹ thuật tấn công không

tệp tin (fileless attacks) hoặc khai thác bộ nhớ (memory exploitation) để thực thi mã độc mà không để lại dấu vết trên đĩa cứng. Những hành vi này chỉ có thể được nhận diện thông qua việc phân tích sâu ngữ cảnh của các lời gọi hệ thống theo thời gian. Luận văn này gọi khoảng cách đó là khoảng cách thực thi chính sách bảo mật (enforcement gap):

1. Các phương pháp phát hiện dựa trên học sâu (Deep Learning) như Transformer, LSTM hoặc GRU có khả năng học quan hệ ngữ cảnh trong chuỗi syscall và thường đạt kết quả tốt trong các thí nghiệm ngoại tuyến. Tuy nhiên, các mô hình này có nhiều tham số, cần phép toán dấu phẩy động và có độ trễ suy luận (inference latency) khó phù hợp với môi trường kernel. Trong bối cảnh một syscall cần được xử lý với chi phí rất nhỏ, việc chạy trực tiếp một mô hình nơ-ron trong đường đi thực thi của hệ điều hành là không khả thi về mặt tài nguyên và an toàn.
2. Các hệ thống phát hiện dựa trên tập luật truyền thống (chẳng hạn như Falco hay Tetragon) có thể vận hành với chi phí thấp nhờ eBPF hoặc kernel modules. Điểm mạnh của hướng này là luật có thể được kiểm tra nhanh và phù hợp với cơ chế giám sát ở mức hệ thống. Ngược lại, phần lớn các luật được xây dựng thủ công, phụ thuộc vào kinh nghiệm của chuyên gia và khó bao phủ đầy đủ các biến thể trình tự trong dữ liệu thực tế.

Từ các hạn chế trên, vấn đề nghiên cứu chính của luận văn là làm thế nào để khai thác khả năng học mẫu tuần tự của mô hình học sâu, nhưng vẫn đưa được kết quả phát hiện về một cơ chế đủ nhẹ để thực thi trong kernel. Luận văn đề xuất **Guepard Shield**, một khung HIDS hướng tới việc chứng cất tri thức từ mô hình Transformer thành một biểu diễn automata hữu hạn đơn định (Deterministic Finite Automata - DFA), sau đó triển khai bảng chuyển trạng thái này bằng eBPF. So với cách chạy trực tiếp mô hình học sâu, hướng tiếp cận này đặt trọng tâm vào việc tách giai đoạn học phức tạp ra khỏi giai đoạn thực thi thời gian thực.

1.2 Bối cảnh và các vấn đề nghiên cứu

Các nghiên cứu về phát hiện xâm nhập dựa trên syscall có thể được nhìn theo nhiều hướng tiếp cận. Nhóm phương pháp ban đầu thường dựa trên thống kê chuỗi con, tiêu biểu là STIDE (Sequence Time-Delay Embedding) do Forrest và các cộng sự đề xuất [1]. STIDE lưu lại các chuỗi syscall bình thường có độ dài cố định và đánh giá bất thường dựa trên sự xuất hiện của những chuỗi chưa từng gặp. Cách tiếp cận này đơn giản và có chi phí tính toán thấp, nhưng phụ thuộc mạnh vào kích thước cửa sổ n . Khi hành vi của tiến trình có phụ thuộc dài hoặc nhiều nhánh thực thi hợp lệ, mô hình n-gram dễ bỏ sót ngữ cảnh hoặc sinh cảnh báo giả.

Sự phát triển của học máy và học sâu cho phép các hệ thống HIDS học được biểu diễn linh hoạt hơn từ dữ liệu. DeepLog [2], chẳng hạn, sử dụng LSTM để dự đoán sự kiện kế tiếp trong chuỗi log hoặc syscall. Các nghiên cứu gần đây dựa trên Transformer và mô hình ngôn ngữ, như công trình của Fournier và các cộng sự [3] hoặc TransCall của Al Siam và các cộng sự [4], khai thác cơ chế tự chú ý (Self-Attention) để mô hình hóa ngữ cảnh dài hơn. Những kết quả này cho thấy học sâu là hướng phù hợp cho bài toán phát hiện bất thường trình tự. Tuy nhiên, phần lớn đánh giá vẫn được thực hiện ở chế độ ngoại tuyến, nơi mô hình có thể chạy trên CPU/GPU ở không gian người dùng và không phải chịu các ràng buộc của kernel.

Bên cạnh các mô hình học từ dữ liệu, các công cụ giám sát hệ thống dựa trên eBPF như Falco, Tetragon hoặc KubeArmor [5] đã được sử dụng rộng rãi trong thực tế. Các công cụ này phù hợp với môi trường sản xuất vì có thể quan sát sự kiện hệ thống với chi phí thấp. Tuy nhiên, tập luật của chúng thường được thiết kế thủ công và mang tính đặc tả. Một hướng nghiên cứu liên quan là giải thích mô hình (XAI) và trích xuất luật (Rule Extraction) từ mạng nơ-ron, như nghiên cứu của Abou El Houda và các cộng sự [6] hoặc các kỹ thuật trích xuất automata từ RNN của Weiss và các cộng sự [7]. Các công trình này cung cấp gợi ý quan trọng cho việc chuyển tri thức từ mô hình học sâu sang biểu diễn rời rạc, nhưng vẫn cần được điều chỉnh cho bài toán syscall, kiến trúc Transformer và môi trường eBPF.

Từ các nhóm tiếp cận trên, khoảng trống chính nằm ở việc kết nối hai nhóm khả năng: học được mẫu tuần tự từ dữ liệu và kiểm tra được mẫu đó với chi phí thấp tại runtime. Luận văn này tập trung vào một quy trình đầu-cuối để xấp xỉ ranh giới quyết định (decision boundary) của mô hình Transformer bằng một biểu diễn trạng thái hữu hạn, từ đó đưa cơ chế phát hiện về dạng có thể triển khai bằng eBPF.

1.3 Mục tiêu nghiên cứu và Khung khái niệm

Mục tiêu của luận văn là nghiên cứu và xây dựng **Guepard Shield**, một pipeline đầu-cuối (end-to-end) cho phát hiện bất thường dựa trên syscall. Pipeline này sử dụng mô hình Decoder-only Transformer như mô hình Teacher để học hành vi bình thường từ dữ liệu, sau đó chuyển biểu diễn đã học sang một DFA Student gọn hơn để phục vụ giám sát thời gian thực. DFA không thay thế hoàn toàn năng lực biểu diễn của Transformer, mà đóng vai trò như một xấp xỉ rời rạc của các mẫu hành vi quan trọng, phù hợp hơn với yêu cầu thực thi bằng eBPF trong Linux kernel.

Để đạt được mục tiêu này, luận văn áp dụng phương pháp nghiên cứu thực nghiệm phối hợp giữa phân tích lý thuyết và triển khai hệ thống. Quá trình huấn luyện và trích xuất DFA được thực hiện trên các bộ dữ liệu chuẩn về syscall như LID-DS [8] và bộ dữ liệu do chúng tôi tự thu thập trên các hệ thống thực tế (DongT-

ing). Việc sử dụng đa dạng các bộ dữ liệu giúp đánh giá được khả năng tổng quát hóa của mô hình trên nhiều loại tải công việc và các kịch bản tấn công khác nhau, từ các cuộc tấn công Brute-force đơn giản đến các kỹ thuật khai thác lỗ hổng CVE phức tạp.

Hệ thống được đặt trong mô hình thời gian thực theo từng luồng tiến trình (thread-level syscall stream). Thay vì thu thập toàn bộ nhật ký rồi phân tích ngoại tuyến, Guepard Shield duy trì trạng thái DFA cho từng định danh luồng (Thread ID - TID) ngay trong kernel. Mỗi khi một syscall mới xuất hiện, chương trình eBPF cập nhật trạng thái hiện tại dựa trên bảng chuyển trạng thái đã nạp trước. Nếu chuỗi quan sát được đưa DFA tới trạng thái không xác định hoặc trạng thái tấn công (attack state), hệ thống có thể đánh dấu hành vi đó là bất thường và chuyển sự kiện nghi vấn lên không gian người dùng để xử lý tiếp. Điều này giúp giảm thiểu đáng kể lượng dữ liệu cần chuyển vùng (context switching) giữa kernel và user space, vốn là nút thắt cổ chai về hiệu năng trong các hệ thống giám sát truyền thống.

Quy trình nghiên cứu được chia thành 4 giai đoạn, từ chuẩn bị dữ liệu đến thực thi trong kernel.

Ngoài pipeline huấn luyện và trích xuất DFA, luận văn còn xem xét cơ chế vận hành giữa kernel space và user space. Thành phần eBPF đảm nhận kiểm tra nhanh trên luồng syscall, trong khi Rust Agent và Transformer Teacher ở user space xử lý các trường hợp nghi vấn hoặc cần đánh giá lại. Cách tổ chức này giữ đường đi runtime trong kernel ở mức đơn giản, đồng thời vẫn cho phép hệ thống sử dụng mô hình học sâu cho các bước phân tích nặng hơn. Sự tương tác này được mô tả trong phần phương pháp nghiên cứu ở Chương 3.

Việc sử dụng DFA cho đường đi runtime có hai lý do chính:

- **Độ phức tạp tính toán:** Khi mô hình Transformer thực hiện suy luận trên một cửa sổ trượt độ dài W với số lượng lớp attention L và chiều ẩn d , độ phức tạp tính toán cho mỗi bước dịch chuyển là $O(L \cdot W^2 \cdot d + L \cdot W \cdot d^2)$. Chi phí này phù hợp với huấn luyện hoặc suy luận ngoại tuyến hơn là với đường đi syscall trong kernel. Ngược lại, việc duyệt trạng thái trên DFA chỉ yêu cầu tra cứu bảng chuyển đổi và cập nhật trạng thái mới. Trong eBPF, thao tác này có thể được biểu diễn thông qua BPF map và hướng tới chi phí gần hằng số $O(1)$ cho mỗi sự kiện.
- **Tính khả thi của Verifier:** Bộ xác minh mã nguồn nhân eBPF (Verifier) áp đặt những giới hạn rất chặt chẽ về số lượng chỉ thị lệnh tối đa được thực thi trong một luồng (complexity limit), kích thước vùng nhớ stack chỉ 512 bytes và hạn chế các vòng lặp vô hạn. Một mô hình Transformer với nhiều phép toán

ma trận và tham số học được không phù hợp với các ràng buộc này. Trái lại, một chương trình eBPF kiểm tra DFA có thể được viết dưới dạng các thao tác tra cứu và cập nhật trạng thái đơn giản, để đáp ứng hơn yêu cầu của Verifier.

1.4 Ứng dụng của nghiên cứu

Việc nghiên cứu và phát triển hệ thống Guepard Shield không chỉ mang ý nghĩa về mặt học thuật trong việc kết nối giữa học sâu và hệ thống máy tính, mà còn có giá trị thực tiễn cao trong bối cảnh an ninh mạng hiện nay.

Thứ nhất, nghiên cứu góp phần giải quyết bài toán hiệu năng của các hệ thống HIDS hiện đại. Bằng cách chưng cất tri thức từ các mô hình nơ-ron phức tạp sang biểu diễn automata, chúng ta có thể tận dụng được sức mạnh của AI trong việc học các mẫu hành vi tinh vi mà không phải hy sinh tốc độ xử lý của hệ điều hành. Điều này đặc biệt quan trọng đối với các nhà cung cấp dịch vụ đám mây (Cloud Service Providers), nơi mà mỗi mili giây độ trễ thêm vào đường đi syscall có thể dẫn tới thiệt hại đáng kể về trải nghiệm người dùng và chi phí tài nguyên.

Thứ hai, việc sử dụng eBPF làm công cụ thực thi chính sách giúp tăng cường tính an toàn và ổn định cho máy chủ. So với việc sử dụng kernel modules truyền thống (thường có nguy cơ gây lỗi "kernel panic" nếu mã nguồn không ổn định), eBPF cung cấp một môi trường thực thi được kiểm chứng (sandboxed), đảm bảo rằng cơ chế giám sát không gây ảnh hưởng tiêu cực đến sự vận hành của nhân hệ điều hành. Guepard Shield tận dụng lợi thế này để triển khai một whitelist động được học tự động từ dữ liệu, thay thế cho các tập luật tĩnh vốn dễ bị lỗi thời.

Thứ ba, nghiên cứu này mở ra một hướng đi mới cho việc triển khai AI tại biên (AI at the edge) hoặc trong các môi trường có ràng buộc khắt khe về tài nguyên. Quy trình trích xuất và tối ưu DFA từ Transformer có thể được mở rộng cho các bài toán khác ngoài phát hiện xâm nhập, chẳng hạn như kiểm soát luồng thực thi phần mềm (Control-Flow Integrity) hoặc giám sát giao thức mạng tại tầng liên kết dữ liệu.

Cuối cùng, kết quả của luận văn cung cấp một công cụ hỗ trợ cho các chuyên gia phân tích an ninh (SOC Analysts) thông qua việc ánh xạ các cảnh báo bất thường với các kỹ thuật tấn công trong ma trận MITRE ATT&CK. Điều này giúp thu hẹp khoảng cách giữa việc "phát hiện một cái gì đó lạ" và việc "hiểu kẻ tấn công đang làm gì", từ đó đẩy nhanh quá trình phản ứng và xử lý sự cố.

1.5 Các đóng góp dự kiến

Luận văn hướng tới 5 đóng góp chính sau:

1. **Thiết kế pipeline HIDS đầu-cuối:** Nghiên cứu đề xuất một quy trình từ tiền xử lý dữ liệu syscall, huấn luyện mô hình Transformer causal, xây dựng DFA
- 5 Student, đến triển khai bảng chuyển trạng thái bằng eBPF; quy trình này giúp

liên kết rõ hơn giữa pha học từ dữ liệu và pha kiểm tra tại runtime.

2. Hình thức hóa phương pháp rời rạc hóa trạng thái ẩn của Transformer:

Luận văn xây dựng cách ánh xạ không gian embedding liên tục của lớp cuối Transformer (vector h_t) sang các trạng thái rời rạc của automata thông qua K-Means. Trên cơ sở đó, luận văn khảo sát các chiến lược xử lý tính không xác định của bước chuyển trạng thái, trong đó có cắt tỉa thống kê (Statistical Pruning, S4) đối với các bước chuyển tần suất thấp.

3. Phân tích khả năng chống mimicry attacks:

Luận văn xem xét cách các chuỗi tấn công có thể bị biến đổi bằng cách chèn thêm syscall bình thường (syscall padding), sau đó phân tích tác động của các biến đổi này lên trạng thái DFA; nội dung này giúp đánh giá khi nào cấu trúc whitelist của DFA có thể hỗ trợ phát hiện hành vi bất thường, và khi nào cần chuyển sự kiện nghi vấn sang bước kiểm tra ở user space.

4. Đánh giá thực nghiệm trên các bộ dữ liệu syscall:

Luận văn đo đặc hiệu năng phát hiện của mô hình Teacher, độ trung thành (fidelity) của DFA Student và chi phí runtime của cơ chế eBPF trên các khối lượng công việc máy chủ như nginx, redis hoặc postgres. Các thí nghiệm này được dùng để phân tích đánh đổi giữa số lượng trạng thái, độ chính xác và chi phí thực thi.

5. Phân tích và đối sánh độ bao phủ với ma trận MITRE ATT&CK:

Luận văn khảo sát khả năng ánh xạ các mẫu trạng thái bị tấn công trong DFA với một số kỹ thuật tấn công trong MITRE ATT&CK, chẳng hạn như leo thang đặc quyền hoặc chiếm quyền điều khiển tiến trình. Mục tiêu của phần này là bổ sung ngữ cảnh cho cảnh báo, thay vì chỉ báo cáo một nhân bất thường đơn lẻ.

1.6 Bố cục của luận văn

Phần còn lại của luận văn được tổ chức thành 5 chương tiếp theo.

Chương 2 trình bày tổng quan tài liệu và cơ sở lý thuyết liên quan đến bài toán giám sát an ninh mức syscall. Chương này phân tích các phương pháp phát hiện bất thường truyền thống như STIDE, các mô hình học sâu như DeepLog và Transformer, nền tảng automata hữu hạn đơn định, cơ chế eBPF và các công cụ thực tế như Falco hoặc Tetragon. Từ đó, chương làm rõ khoảng trống nghiên cứu mà luận văn hướng tới.

Chương 3 trình bày phương pháp nghiên cứu và kiến trúc của Guepard Shield. Nội dung chính bao gồm tiền xử lý dữ liệu và token hóa tên syscall, kiến trúc Decoder-only Transformer dùng làm Teacher, phương pháp rời rạc hóa trạng thái

ẩn bằng K-Means, cách xây dựng bảng chuyển trạng thái DFA và thiết kế thành phần eBPF cùng Rust Agent cho quá trình vận hành.

Chương 4 phân tích các thuộc tính lý thuyết của hệ thống. Chương này so sánh chi phí duyệt DFA với suy luận nơ-ron trực tiếp, thảo luận giới hạn bộ nhớ và Verifier của eBPF, phân tích khả năng xử lý mimicry attacks và đưa ra cách đánh giá độ trung thành (fidelity) của DFA đối với mô hình Teacher.

Chương 5 báo cáo kết quả thực nghiệm. Chương này mô tả môi trường kiểm thử, cấu hình mô hình, các thước đo như AUROC, F1 và fidelity, đồng thời đánh giá chi phí runtime của thành phần eBPF dưới các tải công việc máy chủ như nginx, redis hoặc postgres.

Chương 6 tổng kết các kết quả chính của luận văn và thảo luận các hạn chế còn lại. Chương này cũng đề xuất một số hướng phát triển tiếp theo, bao gồm mở rộng phương pháp token hóa và cải thiện cơ chế định tuyến các trường hợp không chắc chắn giữa kernel space và user space.

CHAPTER 2. NỀN TẢNG LÝ THUYẾT

Chương 2 trình bày tổng quan tài liệu và cơ sở lý thuyết làm nền tảng cho việc nghiên cứu các giải pháp phát hiện bất thường dựa trên chuỗi lời gọi hệ thống. Nội dung chương tập trung vào việc làm rõ phạm vi nghiên cứu chung, phân tích các nghiên cứu liên quan trong lĩnh vực phát hiện bất thường qua chuỗi lời gọi hệ thống (system calls), các phương pháp trích xuất luật hoặc automata từ mô hình học sâu, và các cơ chế giám sát thực thi thời gian thực bằng eBPF. Bên cạnh đó, chương này cũng hệ thống hóa các kiến thức nền tảng về mạng Transformer Decoder-only, lý thuyết Automata hữu hạn đơn định (DFA), và kiến trúc eBPF trong nhân Linux, làm cơ sở khoa học để giải quyết bài toán chứng cất tri thức từ mô hình học máy phức tạp thành luật thực thi hiệu quả trong nhân hệ điều hành.

2.1 Phạm vi nghiên cứu (Scope of Research)

Trong bài toán phát hiện xâm nhập dựa trên lời gọi hệ thống (system call) cho môi trường máy chủ, phạm vi nghiên cứu thường được giới hạn và định nghĩa bởi các yếu tố cốt lõi bao gồm không gian giám sát, mô hình đe dọa, và các bộ dữ liệu chuẩn được sử dụng để đánh giá.

Để đặt nền tảng cho việc thiết kế Guepard Shield, bài toán phát hiện bất thường dựa trên syscall cần được hình thức hóa dưới góc độ của mô hình hóa ngôn ngữ (language modeling) và lý thuyết automata.

Gọi $\mathcal{S} = \{s_1, s_2, \dots, s_{|V|}\}$ là tập hữu hạn các tên lời gọi hệ thống (vocabulary). Một thực thi của tiến trình được biểu diễn dưới dạng một chuỗi các sự kiện $X = (x_1, x_2, \dots, x_T)$, trong đó mỗi $x_t \in \mathcal{S}$ là syscall tại thời điểm t . Mục tiêu của pha huấn luyện mô hình Teacher (Transformer) là học một hàm phân phối xác suất có điều kiện:

$$P(x_t \mid x_{t-k}, \dots, x_{t-1}) \quad (2.1)$$

trong đó k là chiều dài ngữ cảnh (context window). Một chuỗi quan sát được coi là bất thường nếu xác suất xuất hiện của nó thấp hơn một ngưỡng τ được định nghĩa trước. Tuy nhiên, việc tính toán trực tiếp giá trị xác suất này trong nhân hệ điều hành là không khả thi do các ràng buộc về tài nguyên đã nêu.

Thay vào đó, mục tiêu của luận văn là tìm một xấp xỉ rời rạc dưới dạng Deterministic Finite Automaton (DFA), ký hiệu là $M = (Q, \Sigma, \delta, q_0, F)$. Trong đó:

- Q là tập hữu hạn các trạng thái, được trích xuất từ việc phân cụm không gian vector ẩn của Transformer.

- $\Sigma \subseteq \mathcal{S}$ là bảng chữ cái (tập syscall).
- $\delta : Q \times \Sigma \rightarrow Q$ là hàm chuyển trạng thái.
- $q_0 \in Q$ là trạng thái bắt đầu.
- $F \subset Q$ là tập các trạng thái chấp nhận (hành vi bình thường).

Thách thức của việc hình thức hóa này nằm ở chỗ Transformer có khả năng ghi nhớ ngữ cảnh dài thông qua cơ chế chú ý, trong khi DFA truyền thống chỉ phụ thuộc vào trạng thái hiện tại. Luận văn cần giải quyết mâu thuẫn này bằng cách chọn lựa một biểu diễn trạng thái ẩn đủ giàu thông tin để duy trì tính "Markov" cần thiết cho DFA mà không làm bùng nổ số lượng trạng thái.

Một vấn đề nghiên cứu then chốt khác là khả năng chống lại các cuộc tấn công bắt chước (Mimicry Attacks). Trong bối cảnh syscall, kẻ tấn công có thể chèn thêm các lời gọi hệ thống "vô hại" (no-op hoặc các syscall bình thường) vào giữa các bước của mã độc để làm thay đổi biểu diễn vector của chuỗi, từ đó đánh lừa các hệ thống phát hiện bất thường. Việc trích xuất DFA cần đảm bảo rằng cấu trúc whitelist của automata đủ chặt chẽ để các hành vi chèn đệm này vẫn đưa con trỏ trạng thái vào vùng tấn công, hoặc ít nhất là đưa vào các trạng thái nghi vấn (suspect states) để kích hoạt cơ chế kiểm tra sâu hơn ở không gian người dùng. Khả năng chịu đựng của DFA đối với các biến đổi trình tự này là một trong những trọng tâm phân tích của luận văn.

Về không gian giám sát, các nghiên cứu trong lĩnh vực này tập trung vào bài toán phát hiện bất thường ở mức lời gọi hệ thống (syscall-level anomaly detection) trên các máy chủ Linux chạy các khối lượng công việc container hóa (containerized workloads). Giao diện system call đại diện cho ranh giới phân tách thấp nhất giữa không gian người dùng (user space) và không gian nhân (kernel space). Mọi tương tác của ứng dụng với tài nguyên phần cứng, mạng, tệp tin, hoặc tiến trình khác đều bắt buộc phải đi qua ranh giới này. Do đó, việc giám sát ở mức lời gọi hệ thống cung cấp một cái nhìn toàn diện và khó bị vượt qua bởi các hành vi xâm nhập thông thường.

Mô hình đe dọa mục tiêu thường tập trung vào giai đoạn hậu xâm nhập (post-exploitation), giả định rằng kẻ tấn công đã thực thi thành công mã độc hoặc chiếm được quyền truy cập vào container thông qua các lỗ hổng phần mềm. Các hành vi xâm nhập mục tiêu bao gồm thiết lập kết nối ngược (reverse shell), leo thang đặc quyền (privilege escalation), đánh cắp dữ liệu (data exfiltration), và duy trì sự hiện diện lâu dài trong hệ thống (persistence). Kẻ tấn công được coi là hoạt động dưới dạng hộp đen (black-box), nghĩa là họ không có tri thức về các cơ chế chính sách

bảo mật đang được thực thi tại runtime để giám sát hành vi của hệ thống.

Phạm vi nghiên cứu này không giải quyết các vấn đề liên quan đến tấn công né tránh đối kháng (adversarial evasion) nhắm trực tiếp vào việc thay đổi phân phối chuỗi syscall để đánh lừa mô hình học máy. Các rootkit mức kernel can thiệp trực tiếp vào cấu trúc dữ liệu của hệ điều hành hoặc các hệ thống phát hiện xâm nhập mức mạng (Network IDS) cũng nằm ngoài phạm vi khảo sát của luận văn.

2.2 Các nghiên cứu liên quan (Related Work)

2.2.1 Phát hiện bất thường dựa trên chuỗi lời gọi hệ thống

Các nghiên cứu về hệ thống phát hiện xâm nhập trên máy chủ (HIDS) dựa trên syscall đã trải qua nhiều giai đoạn phát triển, từ các thuật toán thống kê đơn giản đến các mô hình học sâu phức tạp.

Phương pháp kinh điển đầu tiên là STIDE (Sequence Time-Delay Embedding) do Forrest và các cộng sự đề xuất [1]. STIDE hoạt động dựa trên cơ chế n -gram, xây dựng một cơ sở dữ liệu chứa tất cả các chuỗi lời gọi hệ thống bình thường có độ dài cố định n thu được trong giai đoạn huấn luyện. Khi vận hành thực tế, bất kỳ chuỗi syscall con nào có độ dài n xuất hiện mà không nằm trong cơ sở dữ liệu bình thường sẽ bị coi là bất thường. Mặc dù STIDE có tốc độ thực thi rất nhanh và thuật toán đơn giản, mô hình này gặp hạn chế lớn do bỏ lỡ các phụ thuộc ngữ cảnh dài hạn và dễ sinh cảnh báo giả khi ứng dụng có nhiều luồng thực thi hợp lệ xen kẽ. Các cải tiến sau đó sử dụng Mô hình Markov ẩn (Hidden Markov Model - HMM) để mô hình hóa sự chuyển dịch trạng thái của tiến trình dưới dạng xác suất, tuy nhiên chi phí tính toán tăng rất nhanh khi kích thước từ vựng (vocabulary) của các syscall mở rộng.

Sự trỗi dậy của học sâu đã thay đổi căn bản cách tiếp cận này. DeepLog [2] giới thiệu việc áp dụng mạng nơ-ron hồi quy (RNN/LSTM) cho bài toán phát hiện bất thường thông qua việc dự đoán phần tử tiếp theo (next-token prediction) trên chuỗi log hoặc chuỗi syscall. Khi xác suất dự đoán của syscall tiếp theo thực tế thấp hơn một ngưỡng định sẵn, hệ thống sẽ gắn cờ cảnh báo. Để giải quyết hiện tượng rò rỉ dữ liệu khi trích xuất đặc trưng bằng Word2Vec truyền thống trên chuỗi syscall, Mvula và các cộng sự [9] khuyến nghị sử dụng các phương pháp mã hóa nén như Byte-Pair Encoding (BPE). Dù đạt độ chính xác cao, các mô hình dựa trên LSTM gặp khó khăn lớn trong việc xử lý song song khi huấn luyện và có độ trễ suy luận lớn, không đáp ứng được yêu cầu phản hồi thời gian thực trong nhân hệ điều hành.

Gần đây, cơ chế Self-Attention trong kiến trúc Transformer đã được ứng dụng để mô hình hóa chuỗi syscall. Nghiên cứu của Fournier và các cộng sự [3] đã chỉ ra rằng các mô hình ngôn ngữ lớn (như Transformer và Longformer) đạt độ chính

xác F1 và AUROC vượt trội ($>95\%$) trên các bài toán phát hiện bất thường nhờ khả năng nắm bắt tốt các quan hệ ngữ cảnh dài hạn giữa các syscall đứng xa nhau. TransCall [4] cũng khai thác Transformer để phát hiện mã độc zero-day dưới dạng xử lý chuỗi syscall tương tự như ngôn ngữ tự nhiên. Tuy nhiên, rào cản lớn nhất của các mô hình Transformer là chi phí tính toán và bộ nhớ cực kỳ lớn. Việc chạy trực tiếp một mô hình Transformer trong nhân hệ điều hành là hoàn toàn bất khả thi do nhân Linux không hỗ trợ các phép tính dấu phẩy động và yêu cầu độ trễ xử lý của mỗi syscall phải ở mức micro giây. Khoảng cách này tạo ra một "khoảng cách thực thi" (enforcement gap) lớn giữa mô hình học máy và môi trường runtime thực tế.

2.2.2 Trích xuất luật khả diễn và Automata từ mạng nơ-ron

Để vượt qua tính chất "hộp đen" của các mô hình học sâu và đảm bảo tính giải trình trong các hệ thống an ninh mạng (forensic accountability), hướng nghiên cứu về mô hình thể thân giải thích được (interpretable surrogate models) và chưng cất tri thức (Knowledge Distillation) đã nhận được nhiều sự quan tâm. Thay vì cố gắng giải thích các mô hình phức tạp thông qua các công cụ hậu kiểm (post-hoc) như LIME hay SHAP vốn tốn thời gian suy luận phụ và không đảm bảo tính nhất quán, hướng tiếp cận này tìm cách chuyển đổi tri thức từ mạng nơ-ron sang các biểu diễn tường minh, dễ hiểu và có thể thực thi hiệu quả.

a, Cơ sở lý thuyết về chưng cất tri thức (Knowledge Distillation)

Chưng cất tri thức là một kỹ thuật trong đó một mô hình nhỏ hơn, gọn hơn (Student) được huấn luyện để mô phỏng hành vi của một mô hình lớn, phức tạp hơn (Teacher). Trong bài toán phát hiện xâm nhập, mô hình Teacher (như Transformer) đóng vai trò là một bộ phân loại chính xác nhưng nặng nề, trong khi Student (như DFA) cần đạt được độ trung thực (fidelity) cao nhất có thể so với Teacher tại các vùng quyết định nhạy cảm. Quá trình này không chỉ là việc bắt chước nhãn đầu ra (hard labels) mà còn là việc học theo phân phối xác suất (soft targets) của Teacher, giúp Student nắm bắt được ranh giới quyết định mượt mà hơn.

b, Kỹ thuật trích xuất Automata từ RNN và Transformer

Trong số các biểu diễn rời rạc, Automata hữu hạn đơn định (DFA) nổi lên như một ứng viên sáng giá nhờ tính đơn định và chi phí thực thi $O(1)$. Công trình tiêu biểu của Weiss và các cộng sự [7] đề xuất kỹ thuật trích xuất DFA từ các mạng hồi quy (RNNs) thông qua quy trình gom cụm không gian trạng thái ẩn. Cụ thể, các vector trạng thái ẩn h_t (vốn mang thông tin lịch sử của chuỗi đầu vào) được ánh xạ vào một tập hữu hạn các trạng thái rời rạc Q bằng các thuật toán phân cụm như K-Means hoặc Mean-Shift.

Đối với kiến trúc Transformer, thách thức nằm ở cơ chế self-attention vốn không

duy trì một trạng thái ẩn tuần tự duy nhất như RNN. Tuy nhiên, các nghiên cứu gần đây của Herbinger và các cộng sự [10] đã chỉ ra rằng vector nhúng của lớp cuối cùng (final-layer embeddings) vẫn chứa đựng đủ thông tin ngữ cảnh để có thể rời rạc hóa. Việc trích xuất DFA từ Transformer yêu cầu một bước xử lý bổ sung để giải quyết tính không đơn định khi ánh xạ từ không gian liên tục sang các bước chuyển trạng thái rời rạc, thường thông qua các chiến lược cắt tỉa thống kê (statistical pruning) để loại bỏ các chuyển dịch nhiễu hoặc có xác suất thấp.

c, Ứng dụng trong an ninh hệ thống và pháp y kỹ thuật số

Việc chuyển đổi tri thức sang DFA mang lại lợi ích kép cho an ninh hệ thống. Về mặt vận hành, DFA cho phép triển khai cơ chế whitelist động có khả năng phản hồi ở tốc độ của phần cứng hoặc kernel. Về mặt phân tích, cấu trúc đồ thị của DFA cho phép các chuyên gia bảo mật truy vết ngược lại các lộ trình tấn công dưới dạng các chuỗi trạng thái tường minh. Nếu một chuỗi syscall bị gán attack, DFA có thể chỉ ra chính xác syscall nào và tại ngữ cảnh nào đã gây ra sự vi phạm, điều mà các mô hình Transformer nguyên bản không thể cung cấp một cách trực quan.

Tuy nhiên, khoảng trống nghiên cứu chính nằm ở việc tối ưu hóa quy trình chứng cất này cho các ràng buộc của môi trường thực thi kernel như eBPF. Hầu hết các công trình hiện nay mới chỉ tập trung vào việc đạt được độ trung thực cao về mặt toán học mà chưa xem xét đến giới hạn bộ nhớ của BPF Maps hoặc khả năng xử lý các chuỗi syscall có chiều dài biến thiên trong môi trường máy chủ thực tế. Guepard Shield hướng tới việc lấp đầy khoảng trống này bằng một pipeline tự động hóa từ trích xuất trạng thái ẩn của Transformer Decoder-only đến nạp bảng chuyển DFA vào eBPF.

2.2.3 Thực thi chính sách bảo mật thời gian thực qua eBPF

Công nghệ eBPF (Extended Berkeley Packet Filter) đại diện cho một bước tiến quan trọng trong kiến trúc nhân Linux, cho phép chạy các chương trình sandboxed hiệu năng cao trực tiếp tại các điểm móc (hooks) như tracepoints hoặc LSM (Linux Security Modules) với chi phí chuyển đổi ngữ cảnh (context switch) bằng không.

Các công cụ bảo mật cloud-native phổ biến hiện nay như Falco và Tetragon [5] tận dụng eBPF để thu thập telemetry và thực thi chính sách ở mức nhân. Nghiên cứu so sánh hiệu năng của Her và các cộng sự [5] cho thấy Tetragon có thể phát hiện và ngăn chặn các hành vi thoát khỏi container (container escapes) chỉ trong 1.178 giây với mức chiếm dụng CPU tối ưu (73%) so với các giải pháp dựa trên không gian người dùng truyền thống. Tuy nhiên, điểm hạn chế cốt lõi của các công cụ này là toàn bộ tập luật (ruleset) phát hiện đều phải được viết thủ công bằng YAML dựa trên tri thức của chuyên gia bảo mật. Cách tiếp cận này mang tính tĩnh,

khó bao phủ các chuỗi hành vi phức tạp và dễ bị vượt qua bởi các biến thể tấn công zero-day.

Để khắc phục điều này, một số nghiên cứu gần đây tìm cách đưa mô hình học máy vào thực thi trong kernel qua eBPF. Zhang và các cộng sự [11] đã thiết kế và triển khai một mạng nơ-ron nhân tạo trực tiếp trong kernel bằng eBPF bằng cách lượng tử hóa mô hình sang số học số nguyên (integer-only arithmetic) để vượt qua giới hạn stack 512 bytes của eBPF verifier. Mô hình đạt thời gian suy luận cực nhanh (3-5 micro giây) nhưng chương trình eBPF này cực kỳ phức tạp, khó bảo trì, và mô hình nơ-ron chạy trong kernel vẫn là một hộp đen không thể giải thích trực tiếp. Bachl và các cộng sự [12] cũng chứng minh việc thực thi cây quyết định (decision tree) trong eBPF tăng 20% hiệu suất so với không gian người dùng.

Bảng 2.1 tóm tắt các ưu điểm và hạn chế chính của các hướng tiếp cận liên quan.

Bảng 2.1: So sánh các phương pháp phát hiện xâm nhập dựa trên syscall

Phương pháp	Ưu điểm	Hạn chế	Khả năng học dữ liệu	Thực thi trong Kernel
STIDE / N-gram [1]	Tốc độ nhanh, thuật toán đơn giản, dễ cài đặt.	Bỏ lỡ các ngữ cảnh dài hạn, độ chính xác thấp trên hệ thống phức tạp.	Có (Thông kê tần suất)	Có (Thông qua cấu trúc seccomp-BPF thô)
LSTM / DeepLog [2]	Nắm bắt ngữ cảnh động tốt hơn n-gram, độ chính xác khá tốt.	Khó xử lý song song khi huấn luyện, độ trễ suy luận lớn.	Có (Học sâu có giám sát/không giám sát)	Không phù hợp để chạy trực tiếp trong kernel
Transformer HIDS [3], [4]	Mô hình hóa tốt phụ thuộc ngữ cảnh dài, thường đạt kết quả phát hiện cao.	Chi phí suy luận và bộ nhớ lớn, khó đặt trực tiếp trên đường đi syscall.	Có (Cơ chế tự chú ý causal)	Không phù hợp để chạy trực tiếp trong kernel
Rules (Falco / Tetragon) [5]	Chạy trực tiếp trong kernel với eBPF, độ trễ thấp.	Luật viết thủ công bởi chuyên gia, khó bao phủ biến thể trình tự phức tạp.	Không có (Tĩnh)	Có (Được thiết kế chạy trực tiếp bằng eBPF)
Guepard Shield (Đề xuất)	Chuyển tri thức từ Transformer sang biểu diễn trạng thái hữu hạn để kiểm tra nhanh bằng eBPF.	Phụ thuộc vào chất lượng mô hình Teacher và cách rời rạc hóa trạng thái.	Có (Chưng cất từ Transformer Teacher)	Có (Thực thi bằng chuyển trạng thái DFA bằng eBPF Maps)

Từ các phân tích trên, khoảng trống lớn nhất là việc tự động hóa cầu nối giữa

mô hình học sâu tuần tự (Transformer) và bộ thực thi chính sách eBPF. Chưa có giải pháp nào tự động chứng cất tri thức từ mô hình Transformer thành một bảng chuyển trạng thái DFA tối ưu, sau đó nạp bảng chuyển này vào các cấu trúc BPF Maps để thực thi giám sát với độ phức tạp $O(1)$ và khả năng giải thích hoàn toàn. Đây là một khoảng trống kỹ thuật lớn trong việc kết nối khả năng học ngữ cảnh của Transformer với hiệu năng thực thi của eBPF.

2.3 Mạng Transformer Decoder-only

Kiến trúc Transformer Decoder-only là mô hình mạng nơ-ron tự hồi quy (autoregressive) được ứng dụng rộng rãi để xử lý và dự đoán các chuỗi dữ liệu tuần tự. Kiến trúc này loại bỏ phần Encoder và chỉ sử dụng các lớp causal self-attention để mô hình hóa luồng sự kiện theo trình tự thời gian.

Giả sử đầu vào của mô hình là một chuỗi các token biểu diễn các lời gọi hệ thống $S = (s_1, s_2, \dots, s_T)$, trong đó mỗi s_t thuộc một từ vựng hữu hạn Σ đại diện cho tập hợp các syscall được định danh. Chuỗi token này trước tiên được đi qua một lớp nhúng (embedding layer) để chuyển thành chuỗi các vector biểu diễn $H^{(0)} = (h_1^{(0)}, h_2^{(0)}, \dots, h_T^{(0)})$ với $h_t^{(0)} \in \mathbb{R}^d$.

Tại mỗi lớp self-attention thứ l , mô hình tính toán các ma trận Query (Q), Key (K) và Value (V) từ biểu diễn ẩn của lớp trước đó $H^{(l-1)}$ bằng các trọng số tương ứng $W_Q^{(l)}, W_K^{(l)}, W_V^{(l)} \in \mathbb{R}^{d \times d}$:

$$Q^{(l)} = H^{(l-1)}W_Q^{(l)}, \quad K^{(l)} = H^{(l-1)}W_K^{(l)}, \quad V^{(l)} = H^{(l-1)}W_V^{(l)} \quad (2.2)$$

Để đảm bảo tính nhân quả (causality) trong việc dự đoán phần tử tiếp theo - nghĩa là mô hình tại thời điểm t chỉ được phép chú ý đến các phần tử đứng trước $s_1, \dots, s_t$ mà không được nhìn thấy tương lai $s_{t+1}, \dots, s_T$ - một ma trận mặt nạ (causal mask) $M \in \mathbb{R}^{T \times T}$ được cộng vào trước khi tính toán hàm softmax. Các phần tử của ma trận M được định nghĩa như sau:

$$M_{i,j} = \begin{cases} 0 & \text{nếu } i \geq j \\ -\infty & \text{nếu } i < j \end{cases} \quad (2.3)$$

Công thức tính toán của lớp causal self-attention được viết dưới dạng:

$$\text{Attention}(Q^{(l)}, K^{(l)}, V^{(l)}) = \text{softmax} \left(\frac{Q^{(l)}(K^{(l)})^T}{\sqrt{d}} + M \right) V^{(l)} \quad (2.4)$$

Mô hình được huấn luyện theo phương pháp học không giám sát bằng cách tối

thiểu hóa hàm mất mát entropy chéo cho bài toán dự đoán token tiếp theo (next-token prediction) trên tập dữ liệu chuỗi syscall lành tính:

$$\mathcal{L} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log P(s_{t+1} \mid s_1, s_2, \dots, s_t) \quad (2.5)$$

Trong đó, xác suất điều kiện được tính thông qua lớp tuyến tính cuối cùng (projection layer) và hàm softmax trên vector trạng thái ẩn lớp cuối $h_t^{(L)} \in \mathbb{R}^d$:

$$P(s_{t+1} = \sigma \mid s_1, \dots, s_t) = \frac{\exp(w_{\sigma}^T h_t^{(L)} + b_{\sigma})}{\sum_{\sigma' \in \Sigma} \exp(w_{\sigma'}^T h_t^{(L)} + b_{\sigma'})} \quad (2.6)$$

Vector trạng thái ẩn $h_t^{(L)}$ tại vị trí t tích lũy toàn bộ thông tin lịch sử của luồng syscall từ bước 1 đến t . Đặc tính này làm cho vector trạng thái ẩn lớp cuối cùng trở thành đối tượng nghiên cứu tiềm năng trong các kỹ thuật phân cụm trạng thái liên tục để trích xuất automata hữu hạn rời rạc phục vụ mục đích giải thích hoặc thực thi chính sách.

2.4 Automata hữu hạn đơn định (DFA)

Một Automata hữu hạn đơn định (Deterministic Finite Automaton - DFA) là một mô hình toán học biểu diễn một hệ thống có số lượng trạng thái hữu hạn, dịch chuyển giữa các trạng thái dựa trên các ký hiệu đầu vào đơn định. Về mặt hình thức, một DFA được định nghĩa là một bộ 5 thành phần:

$$A = (Q, \Sigma, \delta, q_0, F) \quad (2.7)$$

Trong đó:

- Q là tập hợp hữu hạn và không rỗng các trạng thái ($Q = \{q_0, q_1, \dots, q_m\}$).
- Σ là bảng chữ cái đầu vào hữu hạn (trong bài toán syscall là tập hợp các chỉ số hoặc tên của lời gọi hệ thống nằm trong Vocabulary).
- $\delta : Q \times \Sigma \rightarrow Q$ là hàm chuyển trạng thái đơn định. Với mỗi trạng thái hiện tại $q \in Q$ và ký hiệu đầu vào $s \in \Sigma$, tồn tại duy nhất một trạng thái tiếp theo $q' = \delta(q, s) \in Q$.
- $q_0 \in Q$ là trạng thái khởi đầu của hệ thống.
- $F \subseteq Q$ là tập hợp các trạng thái chấp nhận (accepting states).

Trong bài toán phát hiện bất thường sử dụng DFA, tập các trạng thái chấp nhận F thường được thay thế bằng tập các trạng thái tấn công (attack hoặc abnormal

states) $A \subseteq Q$. Khi chuỗi syscall dẫn DFA đến một trạng thái thuộc A , hệ thống sẽ phát tín hiệu cảnh báo xâm nhập.

Đặc tính quan trọng nhất khiến DFA trở thành mục tiêu triển khai lý tưởng trong các hệ thống giám sát hiệu năng cao là tính hiệu quả cực cao về mặt tính toán. Việc chuyển trạng thái chỉ yêu cầu một phép tra cứu bảng chuyển dịch (transition lookup) với độ phức tạp thời gian là $O(1)$ cho mỗi ký hiệu đầu vào:

$$\text{Thời gian xử lý} = O(1), \quad \text{Bộ nhớ lưu trữ} \propto |Q| \times |\Sigma| \quad (2.8)$$

DFA không yêu cầu bất kỳ phép tính số thực dấu phẩy động nào và có thể được biểu diễn hoàn toàn bằng các cấu trúc dữ liệu mảng hoặc bảng băm số nguyên tĩnh, hoàn toàn tương thích với các ràng buộc khắt khe của môi trường nhân hệ điều hành.

2.5 Công nghệ eBPF (Extended Berkeley Packet Filter)

eBPF là một công nghệ mang tính cách mạng trong nhân Linux, cho phép chạy các chương trình tùy biến trực tiếp trong không gian nhân (kernel space) khi các sự kiện hệ thống xảy ra mà không cần sửa đổi mã nguồn kernel hoặc tải các kernel module.

2.5.1 Sự tiến hóa từ Classic BPF đến eBPF

Nguyên thủy, BPF (Classic BPF - cBPF) được giới thiệu vào năm 1992 như một cơ chế lọc gói tin hiệu quả trong nhân. Tuy nhiên, kể từ phiên bản nhân Linux 3.15, eBPF đã mở rộng phạm vi ứng dụng từ mạng sang giám sát toàn diện hệ thống. Sự khác biệt cốt lõi nằm ở việc eBPF sở hữu tập lệnh rộng hơn (64-bit thay vì 32-bit), số lượng thanh ghi tăng từ 2 lên 10, và khả năng gọi các hàm helper trong nhân. Đặc biệt, eBPF giới thiệu cơ chế JIT (Just-In-Time) compiler, cho phép biên dịch mã bytecode eBPF trực tiếp sang mã máy bản địa của CPU, giúp chương trình đạt hiệu năng thực thi gần tương đương với mã nguồn được biên dịch trực tiếp vào nhân.

2.5.2 Cơ chế hoạt động và Điểm móc (Hooks)

Các chương trình eBPF hoạt động theo cơ chế hướng sự kiện (event-driven). Chúng được gắn (hooked) vào các vị trí thực thi cụ thể trong kernel, chẳng hạn như tracepoints (ví dụ: `sys_enter` kích hoạt khi tiến trình chuẩn bị thực hiện một syscall), kprobes/kretprobes (móc vào các hàm nhân bất kỳ), hoặc các LSM hooks (Linux Security Modules) như AppArmor hay SELinux. Khi sự kiện xảy ra, chương trình eBPF được thực thi để phân tích ngữ cảnh sự kiện đó. Khả năng truy cập vào cấu trúc dữ liệu của nhân thông qua các điểm móc này cho phép eBPF

quan sát được các thông tin nhạy cảm như PID, UID, tham số syscall, và ngữ cảnh tiến trình cha-con.

2.5.3 Trình xác thực eBPF (Verifier) và Tính an toàn

Để đảm bảo an toàn tuyệt đối cho hệ điều hành, nhân Linux tích hợp một trình xác thực tĩnh (verifier) thực hiện kiểm tra mã nguồn eBPF trước khi nạp vào nhân. Trình xác thực áp đặt các ràng buộc an toàn vô cùng nghiêm ngặt:

- **Giới hạn ngăn xếp (Stack limit):** Kích thước stack tối đa của một chương trình eBPF chỉ là 512 bytes. Mọi biến cục bộ và cấu trúc dữ liệu tạm thời bắt buộc phải nằm trong giới hạn này. Điều này ngăn chặn việc tiêu tốn quá mức bộ nhớ stack của nhân (kernel stack exhaustion).
- **Đảm bảo kết thúc (Termination guarantees):** Trình xác thực kiểm tra đồ thị dòng chảy điều khiển (Control Flow Graph - CFG) để đảm bảo không có vòng lặp vô hạn và số lượng chỉ thị lệnh tối đa không vượt quá giới hạn cho phép (thường là 1 triệu chỉ thị lệnh). Mọi nhánh rẽ trong chương trình phải được chứng minh là sẽ kết thúc.
- **Truy cập bộ nhớ an toàn:** Cấm truy cập trực tiếp vào các vùng nhớ nhân ngẫu nhiên; mọi truy cập bộ nhớ phải qua các hàm helper được kiểm duyệt (như `bpf_probe_read_kernel`).

2.5.4 Cấu trúc BPF Maps và Lưu trữ trạng thái

Vì kích thước stack bị giới hạn ở 512 bytes, các chương trình eBPF không thể lưu trữ các cấu trúc dữ liệu lớn trực tiếp trên stack. Thay vào đó, eBPF cung cấp cấu trúc BPF Maps. Đây là các kho lưu trữ dữ liệu dạng khóa - giá trị (key-value stores) hiệu năng cao nằm trong bộ nhớ nhân, hỗ trợ tra cứu với độ phức tạp $O(1)$. Các loại BPF Maps phổ biến bao gồm: `BPF_MAP_TYPE_HASH`, `BPF_MAP_TYPE_ARRAY`, hoặc `BPF_MAP_TYPE_LRU_HASH`.

Bảng chuyển trạng thái của DFA $\delta(q, s) = q'$ có thể được biểu diễn trong kernel bằng một Hash Map của BPF. Trong cấu trúc này, khóa (key) là một cặp số nguyên bao gồm định danh trạng thái hiện tại và mã số lời gọi hệ thống (syscall ID), còn giá trị (value) tương ứng là trạng thái kế tiếp. Khi một syscall được kích hoạt, chương trình eBPF thực hiện phép tra cứu $O(1)$ trên Map này để cập nhật trạng thái DFA của luồng tiến trình hiện tại (thường được lưu trong một per-thread state map riêng biệt).

2.5.5 Tính di động và eBPF Ecosystem (BTF & CO-RE)

Thách thức lớn nhất của eBPF truyền thống là sự phụ thuộc vào cấu trúc dữ liệu cụ thể của từng phiên bản nhân Linux (kernel version dependency). Để giải quyết

vấn đề này, công nghệ BTF (BPF Type Format) và cơ chế CO-RE (Compile Once - Run Everywhere) đã được giới thiệu. BTF cung cấp siêu dữ liệu (metadata) về các kiểu dữ liệu trong nhân, trong khi CO-RE cho phép trình nạp (loader) ở không gian người dùng thực hiện việc điều chỉnh địa chỉ (relocation) mã bytecode eBPF tại thời điểm chạy. Nhờ đó, một chương trình eBPF có thể chạy trên nhiều phiên bản nhân khác nhau mà không cần biên dịch lại, một yếu tố then chốt cho việc triển khai Guepard Shield trên các môi trường đám mây đa dạng.

2.5.6 Cập nhật nguyên tử và Nạp nóng (Atomic Update & Hot-reload)

Một đặc tính ưu việt của BPF Maps là hỗ trợ các thao tác cập nhật nguyên tử (atomic map updates). Điều này cho phép một tiến trình chạy ở không gian người dùng có thể ghi đè bảng chuyển trạng thái DFA mới vào BPF Maps mà không cần dừng hoặc dỡ bỏ chương trình eBPF đang giám sát trong kernel. Khả năng này đảm bảo hệ thống có thể cập nhật động các chính sách bảo mật hoặc điều chỉnh trạng thái DFA khi phát hiện sự dịch chuyển phân phối (concept drift) hoặc khi cần tối ưu hóa automata dựa trên dữ liệu mới mà không gây gián đoạn quá trình giám sát thời gian thực.

Tóm lại, công nghệ eBPF không chỉ cung cấp một hạ tầng thu thập dữ liệu hiệu năng cao mà còn đóng vai trò là một bộ thực thi chính sách (enforcement engine) mạnh mẽ. Các ràng buộc của verifier và đặc tính của BPF Maps là những yếu tố quyết định đến cách chúng ta thiết kế và rời rạc hóa mô hình DFA từ Transformer. Việc tận dụng BTF và CO-RE giúp Guepard Shield đạt được tính linh hoạt và khả năng triển khai rộng rãi trong các hệ thống Linux hiện đại.

CHAPTER 3. PHƯƠNG PHÁP ĐỀ XUẤT

Chương 3 trình bày kiến trúc tổng thể và phương pháp nghiên cứu của hệ thống Guepard Shield. Nội dung chương mô tả chi tiết bốn giai đoạn của pipeline đầu-cuối, bao gồm tiền xử lý dữ liệu và token hóa tên lời gọi hệ thống (Giai đoạn 1), huấn luyện mô hình Transformer Teacher theo cơ chế dự đoán token tiếp theo (Giai đoạn 2), chưng cất tri thức sang biểu diễn DFA Student thông qua gom cụm K-Means trên các vector trạng thái ẩn (Giai đoạn 3), và thực thi bảng chuyển trạng thái tại runtime bằng eBPF (Giai đoạn 4). Mỗi giai đoạn được trình bày cùng lý do thiết kế, các siêu tham số kiểm soát và giao thức đánh giá tương ứng.

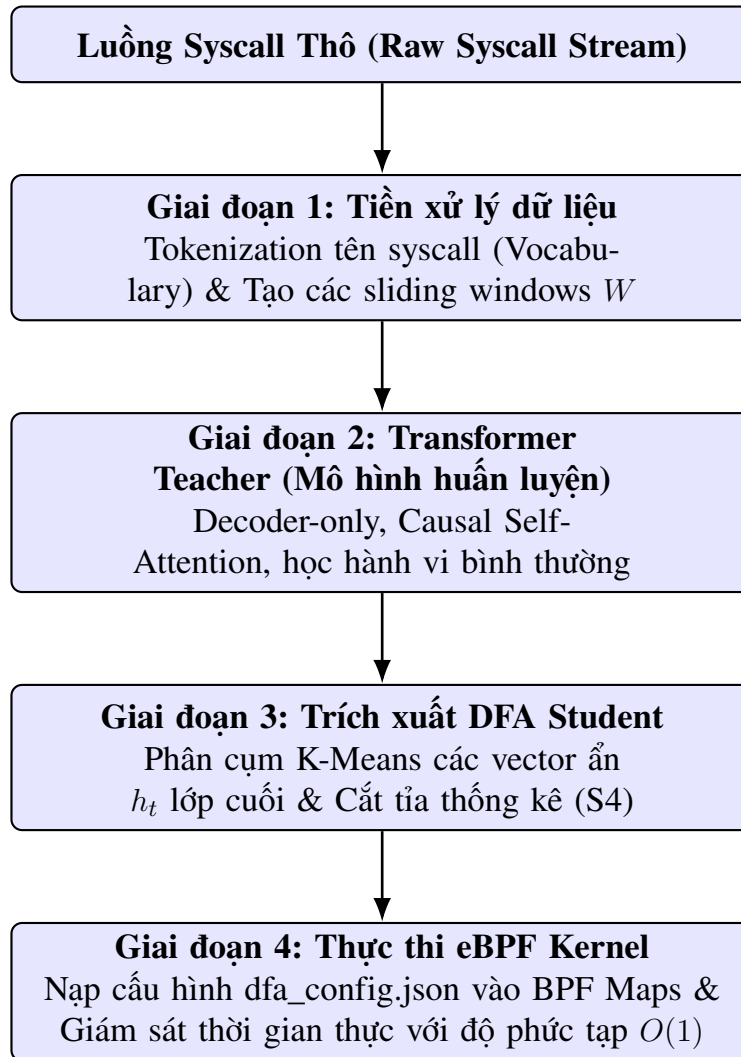
3.1 Tổng quan kiến trúc

Guepard Shield được tổ chức thành một pipeline bốn giai đoạn như đã mô tả tổng quát trong Hình 3.1. Ranh giới giữa hai giai đoạn đầu - học ngoại tuyến (offline learning) - và hai giai đoạn sau - thực thi trực tuyến (online enforcement) - phản ánh nguyên tắc thiết kế trung tâm của hệ thống: tách biệt giai đoạn học biểu diễn phức tạp ra khỏi giai đoạn kiểm tra nhẹ tại runtime.

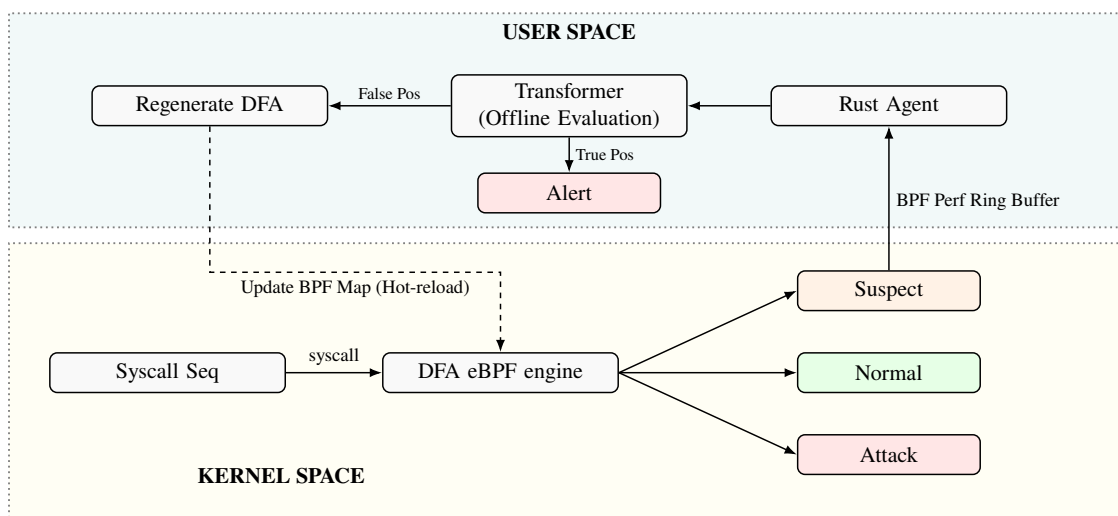
Ở giai đoạn ngoại tuyến, dữ liệu syscall bình thường được tiền xử lý và token hóa (Giai đoạn 1), sau đó được dùng để huấn luyện một mô hình Transformer Decoder-only theo mục tiêu next-token prediction (Giai đoạn 2). Khi mô hình Teacher hội tụ, các vector trạng thái ẩn lớp cuối của nó được trích xuất và rời rạc hóa thành một DFA compact thông qua gom cụm K-Means và xây dựng bảng chuyển trạng thái (Giai đoạn 3). Ở giai đoạn trực tuyến, bảng chuyển trạng thái DFA được nạp vào BPF Maps trong kernel; chương trình eBPF sau đó kiểm tra mỗi syscall với chi phí $O(1)$ trên mỗi sự kiện (Giai đoạn 4).

Cơ chế tương tác động giữa kernel space và user space được mô tả trong Hình 3.2. Khi DFA phát hiện một bước chuyển trạng thái trong vùng nghi vấn (Suspect), sự kiện đó được chuyển lên Rust Agent ở user space thông qua BPF perf ring buffer để đánh giá sâu hơn bởi mô hình Transformer; nếu được xác định là false positive, DFA được cập nhật nguyên tử mà không cần tải lại kernel. Điều này cho phép hệ thống tự điều chỉnh trước các concept drift mà không gián đoạn giám sát thời gian thực.

Một điểm thiết kế quan trọng là hệ thống không ghi lại (record) toàn bộ chuỗi syscall tại runtime. Thay vào đó, chỉ tồn tại các cửa sổ trượt (sliding windows) có kích thước W được duy trì liên tục theo từng định danh luồng (TID). Mỗi syscall mới đẩy cửa sổ tiến một bước, và DFA cập nhật trạng thái hiện tại dựa trên token



Hình 3.1: Sơ đồ pipeline bốn giai đoạn của hệ thống Guepard Shield



Hình 3.2: Cơ chế tương tác động và vòng lặp cập nhật giữa Kernel-space và User-space

tương ứng.

3.2 Giai đoạn 1 - Tiền xử lý dữ liệu

Giai đoạn này chuyển đổi các bản ghi syscall thô từ bộ dữ liệu LID-DS-2021 [8] thành các chuỗi token số nguyên và nhãn phân loại phục vụ cho các giai đoạn tiếp theo.

3.2.1 Token hóa tên lời gọi hệ thống

Mỗi sự kiện syscall được ánh xạ từ tên lời gọi hệ thống sang một chỉ số nguyên thông qua một bộ từ vựng cố định được xây dựng từ tập huấn luyện:

$$\text{token_id}(s) = \text{vocab}[\text{Syscall_Name}(s)] \quad (3.1)$$

Bộ từ vựng được xây dựng với ngưỡng tần suất tối thiểu $f_{\min}$ để loại bỏ các syscall xuất hiện quá thưa thớt trong dữ liệu huấn luyện [9]. Các tên syscall không được nhận diện tại thời điểm inference được ánh xạ tới token đặc biệt `<UNK>`. Trong nghiên cứu này, cấu hình bộ từ vựng tham chiếu đạt kích thước $|\Sigma| = 102$ trong thực nghiệm, bao phủ hầu hết các lời gọi hệ thống xuất hiện trong các khối lượng công việc máy chủ bình thường.

Lý do chọn cách token hóa thuần túy dựa trên tên syscall là vì tên syscall tạo thành một bảng chữ cái hữu hạn và ổn định, thỏa mãn trực tiếp yêu cầu của DFA về một bảng chữ cái đầu vào hữu hạn Σ mà không cần kỹ thuật mã hóa phức tạp. Đây cũng là điều kiện tiên quyết để chương trình eBPF có thể thực hiện ánh xạ từ tên syscall sang token ID bằng một thao tác tra cứu BPF Map đơn giản tại runtime.

Một hướng phát triển được dự kiến là token hóa tổng hợp (composite tokenization), trong đó mỗi syscall được ánh xạ sang một token hỗn hợp kết hợp tên và nhóm mã trả về, ví dụ `(Syscall_Name, ReturnCode_Bucket)`. Phương án này cho phép phân biệt một lệnh `open` thành công với một lệnh thất bại trả về `EPERM`, tăng tính phong phú ngữ nghĩa của bảng chữ cái. Tuy nhiên, mở rộng này làm tăng kích thước bộ từ vựng và kích thước BPF Map tương ứng, nên được hoãn lại sau khi hệ thống cơ sở được xác nhận.

Bảng 3.1 trình bày một phần bộ từ vựng thực nghiệm, minh họa cấu trúc tuyến tính đơn giản của bảng tra cứu này. Toàn bộ ánh xạ được xây dựng một lần duy nhất trên tập huấn luyện và giữ cố định trong suốt vòng đời của mô hình, đảm bảo sự nhất quán giữa giai đoạn ngoại tuyến và giai đoạn thực thi eBPF.

Bảng 3.1: Ví dụ bộ từ vựng syscall: ánh xạ từ tên lời gọi hệ thống sang chỉ số token nguyên. Phần lớn token ID tương ứng với các syscall xuất hiện thường xuyên trong khối lượng công việc máy chủ; token 0 dành riêng cho <UNK>.

Tên syscall	Token ID	Tên syscall	Token ID
<UNK>	0	brk	11
read	1	ioctl	12
write	2	socket	13
open	3	connect	14
close	4	sendto	15
stat	5	recvfrom	16
fstat	6	execve	17
mmap	7	clone	18
munmap	8	futex	19
mprotect	9	poll	20
access	10	...	...

3.2.2 Phân đoạn cửa sổ trượt và gán nhãn

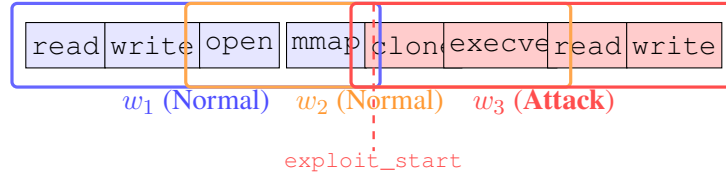
Chuỗi syscall theo từng luồng tiến trình được phân đoạn thành các cửa sổ trượt gối đầu có kích thước W và bước nhảy S [13]:

$$w_i = [s_{iS+1}, s_{iS+2}, \dots, s_{iS+W}] \quad (3.2)$$

trong đó W và S là các siêu tham số đại diện cho kích thước cửa sổ và bước nhảy (cấu hình tham chiếu trong thực nghiệm tương ứng là $W = 64$, $S = 32$). Cửa sổ kích thước nhỏ được lựa chọn vì ba lý do chính:

1. **Tính cục bộ tạm thời của hành vi tấn công:** Phần lõi của một sự kiện khai thác thường hoàn thành trong vài chục syscall, nên một cửa sổ nhỏ đủ để bao phủ tín hiệu bất thường mà không cần ngưỡng cảnh quá dài.
2. **Tính tổng quát hóa của mô hình Teacher:** Window nhỏ hơn buộc mô hình Transformer phải học biểu diễn tổng quát hơn thay vì ghi nhớ các chuỗi đặc trưng cụ thể, dẫn đến các vector trạng thái ẩn có phân cụm sạch hơn ở Giai đoạn 3.
3. **Tính gọn nhẹ của DFA:** Số trạng thái DFA cần thiết phụ thuộc vào kích thước không gian trạng thái ẩn; window nhỏ hơn làm giảm sự đa dạng cần mô hình hóa, giúp DFA chiếm ít bộ nhớ kernel hơn.

Hình 3.3 minh họa quá trình phân đoạn với một chuỗi ví dụ.



Hình 3.3: Minh họa phân đoạn cửa sổ trượt với $W = 4$, $S = 2$. Các ô nền đỏ là các syscall xảy ra sau thời điểm `exploit_start`. Window w_3 nhận nhãn Attack vì timestamp của syscall cuối cùng của nó nằm sau mốc khai thác; w_1 và w_2 nhận nhãn Normal dù có gối đầu với vùng attack.

Các window được gán nhãn bằng cách sử dụng dấu thời gian khai thác từ tệp metadata JSON của LID-DS: một window được gán nhãn **attack** nếu dấu thời gian của syscall cuối cùng trong window nằm trong khoảng thời gian khai thác của bản ghi tương ứng; ngược lại là **normal**. Nhãn này chỉ được sử dụng cho đánh giá ngoại tuyến - không tham gia vào quá trình huấn luyện mô hình, phù hợp với bài toán phát hiện bất thường không giám sát.

Cần lưu ý rằng bộ dữ liệu LID-DS gán nhãn attack cho toàn bộ phần đuôi của bản ghi kể từ thời điểm `exploit_start` mà không xác định thời điểm kết thúc khai thác. Hệ quả thực nghiệm của đặc điểm gán nhãn này đối với việc diễn giải các số liệu đánh giá sẽ được phân tích định lượng và trình bày chi tiết trong Chương 5.

3.3 Giai đoạn 2 - Huấn luyện Transformer Teacher

3.3.1 Kiến trúc mô hình

Mô hình Teacher được xây dựng dựa trên kiến trúc Transformer Decoder-only với causal self-attention như đã trình bày trong §2.3, áp dụng trực tiếp cho chuỗi token syscall [3], [4]. Mặt nạ nhân quả đảm bảo tại mỗi vị trí t , vector trạng thái ẩn lớp cuối $h_t^{(L)}$ chỉ được tính từ ngữ cảnh $(s_1, \dots, s_t)$. Điều này phản chiếu đúng tính nhân quả của luồng syscall tại runtime: hệ thống không bao giờ có thể biết trước syscall tương lai.

Kiến trúc mô hình được tham số hóa bởi kích thước embedding d , số lớp Transformer L , số attention heads h , kích thước feed-forward d_{ff} , kích thước bộ từ vựng $|\Sigma|$ và kích thước cửa sổ W . Trong nghiên cứu này, cấu hình tham chiếu trong thực nghiệm gồm: $d = 128$, $L = 4$, $h = 4$, $d_{\text{ff}} = 512$, $|\Sigma| = 102$, và $W = 64$. Các lớp chuẩn hóa sử dụng RMSNorm theo kiểu pre-norm, hàm kích hoạt GELU, và mã hóa vị trí xoay (Rotary Position Embedding - RoPE). Trọng số lớp nhúng đầu vào và lớp chiếu đầu ra được ràng buộc chung (weight tying) để giảm số tham số và cải thiện tổng quát hóa. Hình 3.4 mô tả sơ đồ kiến trúc đầy đủ.

Lựa chọn Decoder-only thay vì một kiến trúc encoder kiểu BERT xuất phát từ

bản chất thời gian của bài toán syscall. Trong mô hình encoder hai chiều, mỗi vị trí có thể chú ý đồng thời tới các token đứng trước và đứng sau nó; cách này hữu ích cho các bài toán hiểu câu ngoại tuyến, nhưng không phản ánh đúng điều kiện vận hành của một bộ giám sát runtime, nơi quyết định tại thời điểm t chỉ được phép dựa trên lịch sử đã xảy ra. Nếu sử dụng encoder để tạo biểu diễn cho token giữa cửa sổ, vector ẩn đó có thể đã chứa thông tin từ các syscall tương lai trong cùng window, tạo ra một dạng rò rỉ ngữ cảnh (future leakage). Điều này làm điểm đánh giá ngoại tuyến có thể trở nên lạc quan hơn thực tế và, quan trọng hơn, phá vỡ giả định dùng h_t như một trạng thái nhân quả khi trích xuất bảng chuyển DFA ở Giai đoạn 3. Mô hình Decoder-only với causal mask tránh vấn đề này: vector h_t là biểu diễn của đúng tiền tố $(s_1, \dots, s_t)$, tương tự một trạng thái tóm tắt lịch sử đã quan sát của luồng syscall.

Một kiến trúc encoder như BERT cũng thường được huấn luyện bằng masked-token modeling hoặc dùng vector tổng hợp cấp chuỗi như [CLS] cho phân loại [14]. Hai cơ chế này không khớp trực tiếp với mục tiêu của Guepard Shield. Hệ thống cần một tín hiệu dự đoán tuần tự $P(s_{t+1} \mid s_1, \dots, s_t)$ để vừa chấm điểm bất thường bằng NLL, vừa tạo cặp chuyển tiếp (h_t, s_{t+1}, h_{t+1}) phục vụ chứng cất automata. Nếu mô hình chỉ tạo một embedding cấp window, việc ánh xạ embedding đó thành các trạng thái trung gian của DFA sẽ kém tự nhiên hơn, vì DFA cần cập nhật trạng thái sau mỗi syscall chứ không chỉ đưa ra một nhãn cho toàn bộ cửa sổ. Do đó, Decoder-only được chọn không phải vì encoder yếu hơn về mặt biểu diễn, mà vì nó bảo toàn cấu trúc nhân quả và cấu trúc chuyển trạng thái mà pipeline DFA yêu cầu.

Cơ chế self-attention được dùng vì hành vi hệ thống không chỉ phụ thuộc vào syscall ngay trước đó. Một lời gọi như `open`, `read` hoặc `mmap` có thể mang ý nghĩa khác nhau tùy vào chuỗi ngữ cảnh trước đó: tiến trình đang khởi tạo, đang xử lý kết nối, đang nạp thư viện, hay đang thực hiện một chuỗi thao tác hiếm gặp. Các mô hình Markov bậc thấp hoặc n -gram chỉ nhìn được một vùng ngữ cảnh cố định ngắn; trong khi đó, multi-head self-attention cho phép mỗi vị trí kết hợp thông tin từ nhiều điểm trước đó trong window. Nhiều attention head cũng giúp mô hình học các kiểu quan hệ khác nhau giữa syscall: một head có thể tập trung vào các mẫu I/O lặp lại, một head khác vào chuỗi tạo tiến trình hoặc ánh xạ bộ nhớ. Trong nghiên cứu này số head được giữ ở mức $h = 4$ để cân bằng giữa năng lực biểu diễn và chi phí trích xuất trạng thái ẩn ngoại tuyến.

Vì self-attention tự thân không biết thứ tự của token, mô hình cần một cơ chế mã hóa vị trí. Thứ tự là thông tin cốt lõi trong syscall IDS: hai cửa sổ có cùng tập syscall nhưng thứ tự khác nhau có thể biểu diễn hai hành vi hoàn toàn khác nhau.

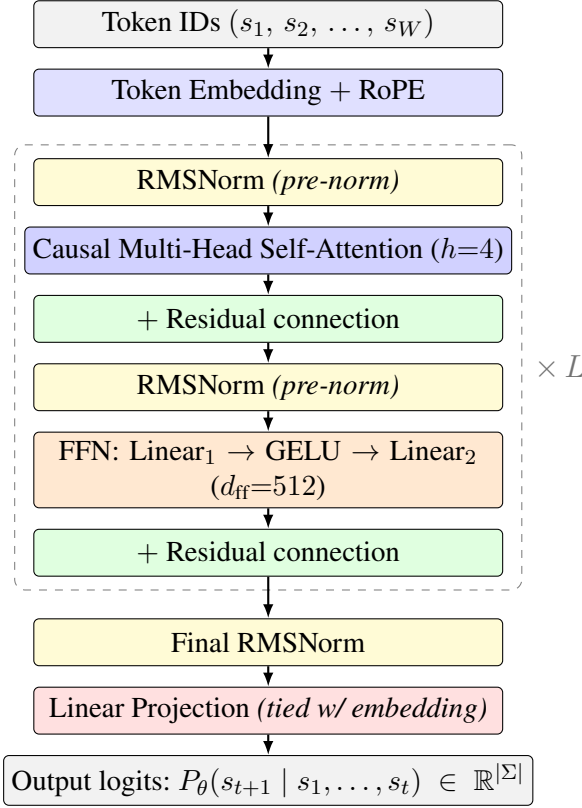
RoPE được chọn vì nó đưa thông tin vị trí vào phép tính attention theo dạng tương đối giữa Query và Key, thay vì chỉ cộng một vector vị trí tuyệt đối vào embedding đầu vào [15]. Điều này phù hợp với dữ liệu của sổ trượt, nơi cùng một mẫu hành vi có thể xuất hiện ở các vị trí hơi khác nhau trong window. Với RoPE, mô hình vẫn biết khoảng cách và thứ tự tương đối giữa các syscall, trong khi ít phụ thuộc hơn vào việc một syscall cụ thể nằm đúng ở vị trí tuyệt đối nào. Đây là một đặc tính hữu ích khi mục tiêu cuối không chỉ là dự đoán token, mà còn là tạo ra các hidden state đủ ổn định để gom cụm bằng K-Means.

Mỗi block Transformer được tổ chức theo dạng pre-norm, đặt RMSNorm trước self-attention và trước FFN. Thiết kế pre-norm giúp đường truyền gradient ổn định hơn khi xếp chồng nhiều lớp, vì nhánh residual luôn cung cấp một đường đi gần tuyến tính từ đầu vào tới đầu ra của block. RMSNorm được dùng thay cho Layer-Norm đầy đủ vì nó chuẩn hóa theo độ lớn căn trung bình bình phương của vector, bỏ qua bước trừ trung bình, nhờ đó đơn giản hơn về mặt tính toán nhưng vẫn kiểm soát được thang đo activation [16]. Trong bối cảnh mô hình Teacher chỉ cần đủ mạnh để học manifold hành vi bình thường và trích xuất embedding ổn định, RMSNorm là một lựa chọn thực dụng: giảm độ phức tạp nhỏ nhưng giữ được lợi ích ổn định huấn luyện của normalization.

Residual connection xuất hiện sau cả lớp attention và lớp FFN vì mỗi sub-layer chỉ nên học phần hiệu chỉnh so với biểu diễn hiện tại, không phải học lại toàn bộ thông tin lịch sử từ đầu [17]. Đối với chuỗi syscall, điều này đặc biệt quan trọng: nhiều token trong window thuộc các mẫu bình thường lặp lại, nên biểu diễn ngữ cảnh đã tích lũy ở lớp trước cần được bảo toàn trong khi attention hoặc FFN bổ sung các quan hệ mới. Nếu bỏ residual connection, các lớp sâu hơn dễ làm mất hoặc bóp méo tín hiệu ngữ cảnh ban đầu, khiến hidden state cuối biến động mạnh hơn và khó gom cụm thành các trạng thái DFA ổn định. Nói cách khác, residual connection không chỉ giúp tối ưu hóa mô hình nơ-ron, mà còn gián tiếp làm bước rời rạc hóa ở Giai đoạn 3 dễ kiểm soát hơn.

Khối FFN gồm hai phép chiếu tuyến tính với GELU ở giữa đóng vai trò biến đổi phi tuyến theo từng vị trí token. Self-attention trộn thông tin giữa các vị trí trong window, trong khi FFN xử lý lại biểu diễn tại từng vị trí để tạo các đặc trưng bậc cao hơn. Kích thước ẩn $d_{\text{ff}} = 512$ lớn hơn kích thước embedding $d = 128$ nhằm tạo không gian trung gian đủ rộng cho các tương tác phi tuyến, sau đó nén trở lại không gian trạng thái ẩn dùng chung. GELU được chọn vì là hàm kích hoạt trơn, thường được dùng trong Transformer hiện đại, giúp mô hình biểu diễn các biến đổi mềm thay vì ngưỡng hóa cứng như ReLU [18]. Cuối cùng, weight tying giữa embedding đầu vào và projection đầu ra tận dụng cùng một không gian biểu diễn cho token

syscall khi đọc vào và khi dự đoán ra [19]. Với bảng chữ cái syscall tương đối nhỏ, ràng buộc này giảm số tham số không cần thiết và khuyến khích biểu diễn token nhất quán hơn giữa hai đầu của mô hình.



Hình 3.4: Kiến trúc Transformer Decoder-only của mô hình Teacher với $L=4$ lớp ($d=128$, $h=4$, $d_{\text{ff}}=512$). Cấu trúc Pre-Norm đặt RMSNorm *trước* mỗi sub-layer, cải thiện ổn định gradient trong huấn luyện sâu. Mặt nạ nhân quả (causal mask) trong MHA đảm bảo h_t chỉ phụ thuộc ngữ cảnh $(s_1, \dots, s_t)$, phản ánh đúng tính nhân quả của luồng syscall tại runtime. Trọng số embedding và projection được ràng buộc chung (weight tying) để giảm số tham số.

Mô hình được huấn luyện **chỉ trên các window bình thường** theo tiêu chí dự đoán token tiếp theo (next-token prediction) với hàm mất mát cross-entropy, theo công thức (2.5) đã định nghĩa ở §2.3. Đây là phương pháp phát hiện bất thường không giám sát (unsupervised anomaly detection) đã được áp dụng rộng rãi trong các hệ thống HIDS dựa trên chuỗi sự kiện [2], [3]: mô hình học phân phối xác suất P (hành vi bình thường); các window bất thường biểu hiện dưới dạng xác suất thấp tương ứng với điểm Negative Log-Likelihood cao tại thời điểm inference.

Quá trình tối ưu hóa sử dụng AdamW với lịch trình suy giảm học suất cosine và giai đoạn khởi động (warmup) chiếm 5% tổng số bước huấn luyện. Kỹ thuật Teacher Forcing cho phép song song hóa quá trình huấn luyện trên toàn bộ chuỗi trong một forward pass. Độ chính xác hỗn hợp (AMP 16-bit mixed precision) được áp dụng để giảm thời gian huấn luyện và bộ nhớ GPU cần thiết.

3.3.2 Điểm bất thường dựa trên NLL của token cuối

Đối với một window $w = [s_1, \dots, s_W]$, điểm bất thường được định nghĩa là giá trị Negative Log-Likelihood (NLL) của **token cuối cùng** s_W khi đặt trong ngữ cảnh $W - 1$ token đứng trước:

$$\text{score}(w) = -\log P_\theta(s_W \mid s_1, \dots, s_{W-1}) \quad (3.3)$$

Thiết kế chỉ tính điểm trên token cuối cùng phù hợp với quy tắc gán nhãn window của LID-DS: một window bị gán nhãn attack khi dấu thời gian của syscall cuối cùng của nó nằm trong khoảng khai thác, do đó điểm bất thường nhắm vào cùng vị trí. Cách tính này đồng thời tạo ra một ánh xạ tự nhiên sang Giai đoạn 3: vector h_{W-1} - trạng thái ẩn được sử dụng để dự đoán s_W - tích lũy toàn bộ lịch sử ngữ cảnh của window và là ứng viên trực tiếp để rời rạc hóa thành trạng thái DFA.

Cách tính điểm bất thường dựa trên NLL của token cuối cũng được so sánh với hai phương án thay thế trong giai đoạn thiết kế:

- **NLL trung bình toàn window:**

$$\overline{\text{NLL}}(w) = -\frac{1}{W} \sum_{t=1}^W \log P_\theta(s_t \mid s_1, \dots, s_{t-1})$$

Cách tính này phân bổ đều tín hiệu bất thường trên toàn bộ chuỗi. Tuy nhiên, do các token đầu window thường thuộc vùng hành vi bình thường (xảy ra trước thời điểm khai thác), chúng đóng góp giá trị NLL thấp vào trung bình và pha loãng tín hiệu tấn công tập trung ở cuối window.

- **NLL tối đa trong window:** $\text{score}_{\max}(w) = \max_{1 \leq t \leq W} (-\log P_\theta(s_t \mid s_1, \dots, s_{t-1}))$.

Phương án này nhạy hơn với các sự kiện cục bộ bất thường, nhưng không nhất quán với quy tắc gán nhãn của LID-DS: token có NLL cao nhất không bảo đảm là token cuối cùng, dẫn đến sự không khớp giữa điểm số và nhãn window.

Một hạn chế quan trọng hơn của toàn bộ hướng tiếp cận ngưỡng NLL là **tính bất ổn định giữa các kịch bản**: giá trị ngưỡng tối ưu khác nhau đáng kể giữa các kịch bản tấn công khác nhau, thậm chí giữa các phiên làm việc trong cùng một kịch bản. Điều chỉnh ngưỡng thích ứng tại runtime trong môi trường nhân kernel là không thực tế. Chính hạn chế này là lý do kỹ thuật trực tiếp dẫn đến thiết kế DFA Student ở Giai đoạn 3: thay vì so sánh với một giá trị ngưỡng liên tục cần hiệu chuẩn lại, quyết định phát hiện được chuyển sang không gian trạng thái rời rạc trong đó nhãn

attack đến từ **bất thường cấu trúc** - bước chuyển trạng thái không tồn tại trong bảng - thay vì so sánh ngưỡng số.

Số liệu đánh giá chính của mô hình Teacher bao gồm AUROC và F1 ở cấp độ cửa sổ so với nhãn exploit-timestamp. AUROC và F1 ở cấp độ bản ghi (recording-level) - thông qua max-pooling điểm số trên từng bản ghi - được báo cáo như một chẩn đoán bổ sung để phù hợp với giao thức đánh giá LID-DS tiêu chuẩn [8].

3.4 Giai đoạn 3 - Chứng cất DFA Student

Giai đoạn này chứng cất tri thức đã học của mô hình Transformer Teacher thành một DFA Student compact theo hướng trích xuất automata và chứng cất tri thức từ mạng nơ-ron [7], [10], [20], điều chỉnh cho kiến trúc Transformer Decoder-only và bài toán chuỗi syscall. Quá trình gồm bốn bước tuần tự: trích xuất trạng thái ẩn, rời rạc hóa bằng K-Means, xây dựng bảng chuyển trạng thái, và giải quyết tính không xác định.

3.4.1 Định nghĩa và trích xuất trạng thái ẩn

Trạng thái ẩn dùng để rời rạc hóa là vector nhúng đầu ra lớp Transformer cuối cùng tại vị trí token cuối của mỗi forward pass:

$$h_t = \text{TransformerFinalLayer}(s_1, \dots, s_t)[\text{vị trí } t] \in \mathbb{R}^d \quad (3.4)$$

Nhờ cơ chế causal self-attention đã trình bày trong §2.3, h_t mã hóa toàn bộ lịch sử nhân quả $(s_1, \dots, s_t)$ trong một vector duy nhất, đóng vai trò tương tự trạng thái ẩn của RNN [7]. Đây là thành phần tự nhiên nhất để biểu diễn trạng thái DFA, vì một trạng thái DFA theo định nghĩa hình thức ở §2.4 cũng phải mã hóa toàn bộ thông tin từ lịch sử chuỗi đầu vào cần thiết cho các quyết định chuyển trạng thái tương lai.

Để đảm bảo tất cả các vector trạng thái ẩn được trích xuất đều có ngữ cảnh đầy đủ W token, các vector trạng thái ẩn được trích xuất từ tập dữ liệu huấn luyện bình thường. Nhằm tối ưu hóa dung lượng lưu trữ trung gian và chi phí tính toán ngoại tuyến, quá trình trích xuất này được thực hiện với một bước nhảy thưa (stride $S_{\text{extract}} > 1$) thay vì trích xuất dày đặc từng bước một. Tuy nhiên, tại thời điểm thực thi ở nhân hệ điều hành, chương trình eBPF phải giám sát và duyệt trạng thái DFA trên từng lời gọi hệ thống đơn lẻ (tương ứng với bước nhảy $S = 1$). Sự không khớp về bước nhảy này (stride mismatch) tạo ra một bài toán kỹ thuật: nếu xây dựng bảng chuyển trạng thái trực tiếp từ các trạng thái ẩn thưa, bước chuyển trạng thái sẽ nhảy qua nhiều syscall thay vì chuyển tiếp tuần tự trên từng syscall một.

Để giải quyết bài toán không khớp bước nhảy này mà không làm bùng nổ không gian lưu trữ trung gian, hệ thống áp dụng cơ chế stream hai pha (two-pass streaming) trên dữ liệu huấn luyện:

1. **Pha 1 (Gom cụm thưa):** Tập huấn luyện được quét hai lần. Lần quét thứ nhất chỉ đếm số cửa sổ hợp lệ để xác định trước kích thước của ma trận trạng thái ẩn và bảng metadata. Lần quét thứ hai đưa từng batch của sổ qua bộ mã hóa của Transformer, lấy vector h_t tại vị trí cuối của sổ và ghi tuần tự vào ma trận trạng thái ẩn. Metadata đi kèm mỗi vector gồm định danh bản ghi, vị trí bắt đầu của cửa sổ trong bản ghi, và token cuối của cửa sổ. Trong cấu hình tham chiếu, pha này sử dụng stride 4 và lưu trạng thái ẩn ở độ chính xác thấp hơn để giảm dung lượng trung gian, trong khi các tham số hình dạng như số cửa sổ M , số chiều d , kích thước cửa sổ W , kích thước từ vựng và stride được lưu như metadata của thí nghiệm.
2. **Pha 2 (Dựng bước chuyển dày):** Sau khi đã học được các tâm cụm, toàn bộ chuỗi syscall huấn luyện được stream lại với bước nhảy dày $S = 1$. Tại mỗi vị trí t , cửa sổ ngữ cảnh đầy đủ được đưa qua Transformer để tính vector trạng thái ẩn h_t , sau đó vector này được chiếu ngay vào tâm cụm gần nhất để xác định trạng thái DFA tương ứng. Các bước chuyển tiếp trạng thái tuần tự giữa hai thời điểm liên tiếp $(q_t, s_{t+1}) \rightarrow q_{t+1}$ được tích lũy trực tiếp vào bảng đếm. Nhờ cơ chế chiếu trực tiếp này, hệ thống không cần lưu toàn bộ hidden states stride-1 xuống đĩa, nhưng vẫn xây dựng được DFA đúng với từng syscall đơn lẻ.

Cách tiếp cận này đảm bảo mỗi trạng thái DFA đại diện cho một ngữ cảnh đầy đủ và ổn định, đồng thời bảo toàn tính chính xác của các bước chuyển tiếp đơn bước trong nhân hệ điều hành. Các tham số cụ thể như độ dài bước nhảy thưa S_{extract} và các phân tích hiệu năng liên quan sẽ được trình bày trong Chương 5. Thuật toán 1 mô tả cụ thể bước trích xuất ma trận hidden states làm đầu vào cho K-Means: trước hết đếm số cửa sổ hợp lệ để cấp phát bộ nhớ, sau đó đưa các cửa sổ qua Transformer theo batch và ghi lại vector trạng thái ẩn tại token cuối cùng cùng với metadata ánh xạ.

Algorithm 1: Trích xuất hidden states của Transformer làm đầu vào cho K-Means

Input: Tập bản ghi huấn luyện bình thường $\mathcal{R}_{train}$; Transformer Teacher F_θ ; bộ từ vựng ν ; kích thước cửa sổ W ; stride trích xuất $S_{extract}$; batch size B

Output: Ma trận trạng thái ẩn $H \in \mathbb{R}^{M \times d}$ và bảng metadata $A \in \mathbb{Z}^{M \times 3}$

$M \leftarrow 0$

foreach $r \in \mathcal{R}_{train}$ **do**

$(x_1, \dots, x_{|r|}) \leftarrow \text{tokenize}(r, \nu)$

$M \leftarrow M + \text{count_windows}(|r|, W, S_{extract})$

end

$H \leftarrow \text{zeros}(M, d)$; $A \leftarrow \text{zeros}(M, 3)$

$m \leftarrow 1$

foreach $r \in \mathcal{R}_{train}$ **do**

$(x_1, \dots, x_{|r|}) \leftarrow \text{tokenize}(r, \nu)$

$\mathcal{W} \leftarrow \text{windows}(x, W, S_{extract})$

foreach $\mathcal{B} \in \text{batches}(\mathcal{W}, B)$ **do**

$Z \leftarrow \text{last_hidden}(F_\theta, \mathcal{B})$

foreach $(h, j) \in Z$ **do**

$H[m] \leftarrow h$

$A[m] \leftarrow (\text{id}(r), j, x_{j+W-1})$

$m \leftarrow m + 1$

end

end

end

return (H, A)

Lựa chọn vị trí token cuối cùng để lấy trạng thái ẩn cũng được so sánh với hai chiến lược thay thế phổ biến:

- **Mean pooling** $\bar{h} = \frac{1}{W} \sum_{t=1}^W h_t^{(L)}$: Phép trung bình hóa tất cả các vị trí trong window tạo ra một biểu diễn cấp câu (sentence-level), nhưng pha loãng thông tin ngữ cảnh đặc thù của thời điểm hiện tại. Với chuỗi syscall, các token giữa window mang biểu diễn của các trạng thái trung gian không nhất thiết liên quan đến quyết định tại thời điểm cuối; đưa chúng vào trung bình làm giảm tính phân biệt của các cụm K-Means.
- **Token [CLS] chuyên dụng**: Kiến trúc Encoder-only (BERT) sử dụng token [CLS] với bidirectional attention để tổng hợp biểu diễn cấp chuỗi. Tuy nhiên,

mô hình Teacher sử dụng causal attention, trong đó không có vị trí nào có thể nhìn thấy toàn bộ chuỗi đầy đủ ngoại trừ token cuối cùng. Thêm token [CLS] với cơ chế đặc biệt sẽ đòi hỏi thay đổi kiến trúc và mục tiêu huấn luyện, phá vỡ tính nhất quán giữa giai đoạn huấn luyện và giai đoạn trích xuất.

Do đó, h_{W-1} là lựa chọn tự nhiên, nhất quán với kiến trúc nhân quả, và hiệu quả nhất về mặt tính toán: nó là vector duy nhất trong mỗi forward pass đã tích lũy toàn bộ lịch sử W token, không cần bất kỳ bước xử lý bổ sung nào.

3.4.2 Rời rạc hóa trạng thái bằng K-Means

Tập hợp các vector $\{h_t\}_{t=1}^N$ thu được từ toàn bộ tập huấn luyện được áp dụng thuật toán gom cụm K-Means với K cụm. Mỗi tâm cụm c_k định nghĩa một trạng thái DFA $q_k \in Q$; một vector h được ánh xạ tới trạng thái q_k trong đó:

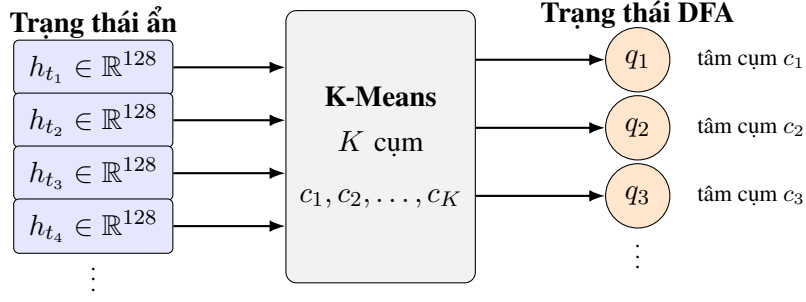
$$\text{cluster}(h) = \arg \min_{k \in \{1, \dots, K\}} \|h - c_k\|_2 \quad (3.5)$$

Về mặt thuật toán, hệ thống sử dụng biến thể mini-batch của K-Means thay vì K-Means toàn phần, vì ma trận trạng thái ẩn có thể lớn hơn bộ nhớ RAM nếu nạp toàn bộ ở độ chính xác huấn luyện. Ở mỗi epoch, thuật toán đọc tuần tự các chunk trạng thái ẩn, chuyển chúng về độ chính xác tính toán phù hợp, rồi cập nhật tâm cụm bằng quy tắc mini-batch. Sau khi các tâm cụm hội tụ, toàn bộ ma trận trạng thái ẩn được quét lại để gán nhãn cụm cho từng cửa sổ; kết quả là một dãy nhãn trạng thái rời rạc có cùng thứ tự với các cửa sổ huấn luyện, còn các tâm cụm được giữ lại để chiếu các hidden state mới trong pha dựng bước chuyển dày. Các giá trị $K \in \{64, 128, 256, 512\}$ được huấn luyện độc lập để đánh giá đánh đổi giữa độ mịn của rời rạc hóa và kích thước DFA. Vì vậy, K-Means ở đây không được hiểu là một bộ phân loại normal/attack trực tiếp; nó chỉ biến không gian trạng thái ẩn liên tục của Transformer thành các chỉ số trạng thái rời rạc mà bảng chuyển DFA có thể sử dụng.

Hình 3.5 minh họa quá trình ánh xạ từ không gian liên tục $\mathbb{R}^d$ sang các trạng thái DFA rời rạc.

Siêu tham số K điều chỉnh sự đánh đổi giữa **độ trung thực (fidelity)** và **tính gọn nhẹ (compactness)** của DFA:

- K nhỏ: DFA có ít trạng thái và BPF Map nhỏ, nhưng tỷ lệ xung đột không xác định cao hơn vì hai chuỗi có lịch sử ngữ cảnh khác nhau dễ bị gộp chung vào cùng một trạng thái, làm tăng khả năng phát sinh false positives.
- K lớn: DFA tinh tế hơn và tỷ lệ xung đột thấp hơn, nhưng kích thước BPF



Hình 3.5: Quá trình rời rạc hóa trạng thái ẩn Transformer thành các trạng thái DFA thông qua gom cụm K-Means. Mỗi tâm cụm c_k trong không gian $\mathbb{R}^d$ trở thành một trạng thái $q_k \in Q$ của DFA.

Map lớn hơn và rủi ro overfitting vào các chuỗi huấn luyện cụ thể cũng cao hơn.

Phạm vi ứng viên cho K được khảo sát thực nghiệm trong khoảng $\{64, 128, 256, 512\}$ (xem Chương 5).

3.4.3 Xây dựng bảng chuyển trạng thái

Sau khi đã gán nhãn trạng thái cho tất cả các vị trí trong tập huấn luyện, bảng chuyển trạng thái được xây dựng bằng cách duyệt qua tất cả các cặp vị trí liên tiếp $(t, t + 1)$ trong cùng một bản ghi:

1. Tra cứu trạng thái nguồn: $A = \text{cluster}(h_t)$.
2. Tra cứu trạng thái đích: $B = \text{cluster}(h_{t+1})$.
3. Ghi lại bước chuyển ứng viên: $\delta(A, s_{t+1}) \rightarrow B$.

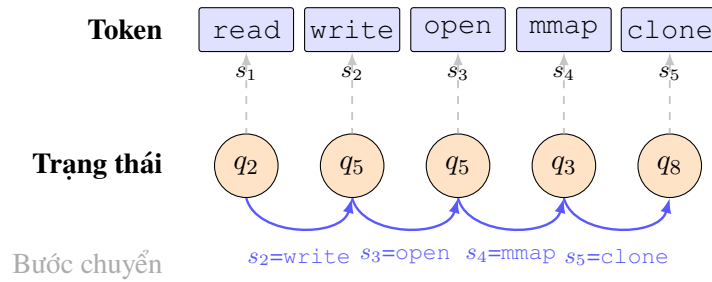
Kết quả thô là một **quan hệ chuyển đổi** (transition relation) - về bản chất là một NFA (Non-deterministic Finite Automaton) - vì cùng một cặp (A, s) có thể ánh xạ tới nhiều trạng thái đích khác nhau trên các chuỗi khác nhau do quá trình chiếu K-Means làm mất thông tin ngữ cảnh.

Hình 3.6 minh họa quá trình trích xuất quan hệ chuyển đổi từ một đoạn chuỗi token mẫu.

3.4.4 Giải quyết tính không xác định

Tính không xác định phát sinh khi hai chuỗi với lịch sử ngữ cảnh khác nhau rơi vào cùng một cụm K-Means, sau đó phân kỳ tại cùng một đầu vào s . Đây là hệ quả không thể tránh khỏi của phép chiếu từ không gian liên tục $\mathbb{R}^d$ xuống K trạng thái rời rạc. Bốn chiến lược giải quyết được đề xuất và đánh giá thực nghiệm:

Chiến lược S4 (Statistical Pruning) ban đầu được coi là ứng viên lý thuyết chính vì các khối lượng công việc máy chủ bình thường có tính lặp lại rất cao: phân phối



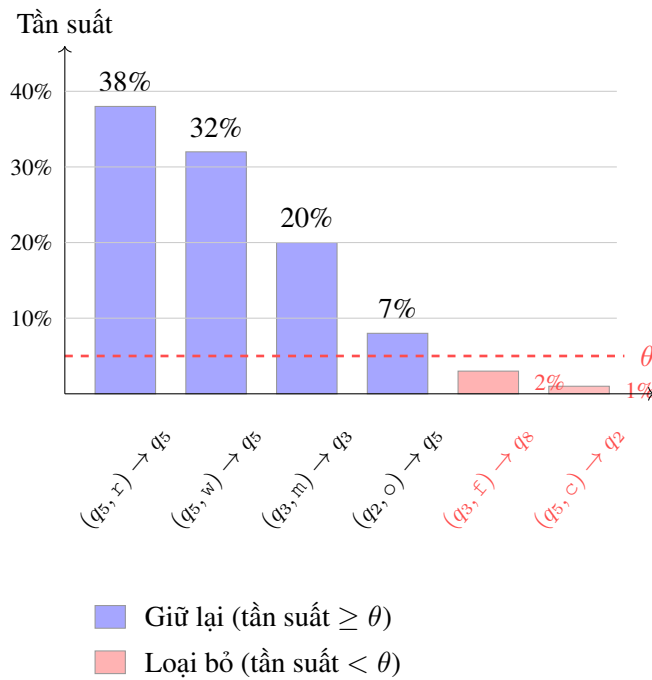
Hình 3.6: Ví dụ xây dựng quan hệ chuyển đổi từ một đoạn chuỗi token. K-Means gán mỗi vector trạng thái ẩn h_t vào một cụm (hàng giữa). Mỗi cặp liên tiếp $(q_i, s_{t+1}) \rightarrow q_j$ đóng góp một ứng viên vào bảng chuyển trạng thái thô (hàng dưới). Hai bước chuyển liên tiếp $(q_2, write) \rightarrow q_5$ và $(q_5, open) \rightarrow q_5$ minh họa trường hợp một trạng thái tự lặp sau khi nhận token đầu vào mới.

Bảng 3.2: Các chiến lược giải quyết tính không xác định trong quá trình trích xuất DFA

ID	Chiến lược	Cơ chế	Đánh đổi
S1	NFA $\rightarrow$ DFA Determinization	Xây dựng tập hợp con (Subset construction): mỗi trạng thái DFA là một <i>tập hợp</i> trạng thái NFA.	Đảm bảo chính xác; có thể gây bùng nổ trạng thái theo cấp số nhân.
S2	Tăng K	Gom cụm tinh hơn giảm mất thông tin khi chiếu, thu hẹp xung đột không xác định.	BPF Map lớn hơn; cần hiệu chuẩn lại K .
S3	Majority Voting	Với mỗi (A, s) , giữ lại duy nhất trạng thái đích có tần suất xuất hiện cao nhất.	Đơn giản; các nhánh thiếu số bị loại bỏ mà không có ngưỡng kiểm soát.
S4	Statistical Pruning (θ)	Đếm tần suất các nhánh đích cho mỗi cặp $(state, token)$. Chỉ giữ lại bước chuyển của nhánh dominant nếu tỷ lệ của nó $\geq \theta$ (ví dụ: 99%). Loại bỏ toàn bộ bước chuyển nếu không đạt.	DFA gọn nhẹ; khai thác phân phối lệch của các bước chuyển. Ứng viên chính.

các bước chuyển bị lệch mạnh với một nhánh chính (happy path) chiếm ưu thế. Các nhánh thiểu số có tần suất dưới θ thường được giả định là tạo vật của quá trình lượng tử hóa K-Means hơn là phản ánh sự biến đổi hành vi thực sự. Ngưỡng θ đóng vai trò siêu tham số thứ hai sau K ; cặp (θ, K) được thiết kế để hiệu chuẩn thông qua tìm kiếm lưới trên tập kiểm định. Sự đối chiếu hiệu năng chi tiết, tỷ lệ không xác định (non-determinism rate) thực tế và lựa chọn chiến lược tối ưu cuối cùng giữa S3 và S4 sẽ được trình bày cụ thể thông qua các kết quả thực nghiệm trong Chương 5.

Hình 3.7 minh họa nguyên lý lựa chọn ngưỡng θ dựa trên đặc điểm phân phối lệch của các bước chuyển trạng thái trong workload máy chủ bình thường.



Hình 3.7: Minh họa chiến lược S4 Statistical Pruning. Các bước chuyển (ví dụ minh họa; tên rút gọn: $r=read, w=write, m=mmap, o=open, f=futex, c=clone$) được sắp xếp giảm dần theo tần suất. Đường ngưỡng θ loại bỏ các nhánh thiểu số (màu đỏ), vốn là tạo vật của lượng tử hóa K-Means thay vì hành vi thực sự. Phân phối lệch nặng về nhóm bước chuyển chính là đặc trưng điển hình của workload máy chủ lặp lại cao.

3.4.5 Giao thức đánh giá DFA

Số liệu chính để đánh giá DFA được chọn là **tỷ lệ phát hiện cấp độ bản ghi (per-recording Detection Rate - DR)**:

$$DR = \frac{\text{số bản ghi tấn công có ít nhất 1 window bị gán attack}}{\text{tổng số bản ghi tấn công}} \quad (3.6)$$

DR đặt câu hỏi đúng về mục đích vận hành: “Với một bản ghi chứa tấn công thực sự, DFA có bắt được ít nhất một khoảnh khắc tấn công không?” Đơn vị bản

ghi có ý nghĩa vận hành rõ ràng hơn đơn vị window riêng lẻ và loại bỏ được ảnh hưởng của phần lớn window được gắn nhãn attack trong LID-DS mà thực chất chứa hành vi bình thường xảy ra sau khi khai thác đã kết thúc (phân tích chi tiết trong Chương 5).

Các số liệu phụ chỉ mang tính chẩn đoán:

- **TPR_window**: Tỷ lệ gắn attack trên nhãn window thô - bị pha loãng bởi nhiều nhãn LID-DS, không dùng để so sánh chất lượng giữa các cấu hình.
- **Fidelity**: Tỷ lệ đồng thuận giữa quyết định của DFA và quyết định của Teacher trên tập kiểm định - dùng để so sánh tương đối giữa các cấu hình (θ, K) .
- **FPR**: Tỷ lệ gắn attack trên các window bình thường - nhãn normal trong LID-DS không bị nhiễm và có thể sử dụng trực tiếp.

3.4.6 Xuất cấu hình DFA

DFA hoàn chỉnh sau bước giải quyết tính không xác định được xuất sang tệp `dfa_config.json`, chứa toàn bộ thông tin cần thiết để nạp vào eBPF Maps ở Giai đoạn 4. Bảng 3.3 mô tả cấu trúc ánh xạ giữa các thành phần DFA và các BPF Map tương ứng trong kernel.

Bảng 3.3: Cấu trúc ánh xạ từ thành phần DFA sang các BPF Maps trong kernel

BPF Map	Khóa (Key)	Giá trị (Value)
transition_table	(state_id, token_id)	next_state_id
state_class	state_id	{NORMAL, SUSPECT}
thread_state	TID	current_state_id

Để tối ưu hóa hiệu năng truy xuất trong kernel space, bảng chuyển trạng thái có thể được cấu hình để biểu diễn dưới dạng một mảng hai chiều trực tiếp (Direct-indexed 2D Array) có kích thước $K \times M_{\text{syscall}}$ trong bộ nhớ eBPF thay vì sử dụng cấu trúc Hash Map băm thông thường (trong đó M_{syscall} là giới hạn chỉ số lời gọi hệ thống tối đa được định nghĩa trong hệ điều hành). Thiết kế mảng hai chiều này cho phép chương trình eBPF tra cứu trạng thái tiếp theo trực tiếp bằng phép tính offset địa chỉ bộ nhớ chỉ với chi phí $O(1)$ thực sự, loại bỏ hoàn toàn các phép toán băm và truy vấn phức tạp của Hash Map. Các phân tích chi tiết về dung lượng bộ nhớ lý thuyết và hiệu năng của phương án tối ưu hóa mảng hai chiều này so với cấu hình bản đồ băm truyền thống sẽ được trình bày cụ thể trong Chương 4 và Chương 5.

Sau khi đánh giá lưới các cấu hình DFA, hệ thống chọn cấu hình cuối cùng theo quy trình trong Thuật toán 2: tiêu chí chính là FPR nhỏ nhất trong nhóm đạt ngưỡng phát hiện cấp bản ghi tối thiểu $DR_{\text{rec}} \geq 0.5$. Các cấu hình sinh ra từ

subset construction không được dùng cho bước export runtime nếu ID trạng thái sau determinization không còn tương ứng một-một với ID cụm K-Means; trong trường hợp đó, việc dùng tần suất cụm để gán lớp trạng thái sẽ làm sai ngữ nghĩa của hàm phân loại trạng thái. Với cấu hình được chọn, bảng chuyển được chuyển sang một schema phẳng gồm số trạng thái, số token, trạng thái bắt đầu, bảng chuyển trạng thái, ánh xạ syscall-to-token, lớp trạng thái và metadata đánh giá. Bảng chuyển dùng giá trị đặc biệt -1 cho các cặp $(state, token)$ không tồn tại; đây chính là tín hiệu runtime để gán **Attack**.

Algorithm 2: Xuất DFA runtime từ các cụm K-Means

Input: Tập cấu hình DFA ứng viên $\mathcal{D}$; tập tâm cụm K-Means $\{c_1, \dots, c_K\}$; nhãn cụm của các cửa sổ huấn luyện $\ell_1, \dots, \ell_N$; bảng chuyển đã giải quyết không xác định δ ; ánh xạ syscall sang token ν ; trạng thái bắt đầu q_0 ; ngưỡng phát hiện tối thiểu $DR_{\min}$; phân vị biên ρ

Output: Cấu hình DFA phẳng $(K, |\Sigma|, q_0, T, \nu, C, \mathcal{M})$

$\mathcal{D}_{valid} \leftarrow \{d \in \mathcal{D} \mid DR_{rec}(d) \geq DR_{\min}\}$

$\mathcal{D}_{valid} \leftarrow \text{keep_cluster_aligned}(\mathcal{D}_{valid})$

$d^* \leftarrow \arg \min_{d \in \mathcal{D}_{valid}} FPR(d)$

$T \leftarrow -\mathbf{1}^{K \times |\Sigma|}$

foreach $(q, s) \mapsto q' \in \delta_{d^*}$ **do**

$T[q, s] \leftarrow q'$

end

for $k \leftarrow 1$ **to** K **do**

$f_k \leftarrow |\{i \mid \ell_i = k\}|$

end

$\tau_\rho \leftarrow \text{Percentile}_\rho(f_1, \dots, f_K)$

for $k \leftarrow 1$ **to** K **do**

if $f_k \leq \tau_\rho$ **then**

$C(k) \leftarrow \text{SUSPECT}$

// Trạng thái hiếm gặp

else

$C(k) \leftarrow \text{NORMAL}$

end

end

$\mathcal{M} \leftarrow \text{metadata}(d^*, T, C)$

return $(K, |\Sigma|, q_0, T, \nu, C, \mathcal{M})$

Sự phân loại trạng thái trong cấu hình xuất gồm hai lớp được lưu trực tiếp:

Normal và **Suspect**. Cơ chế export trước hết tính ngưỡng tần suất ở phân vị thấp của phân phối tần suất trạng thái; các trạng thái có tần suất nằm dưới ngưỡng này được xem là trạng thái biên và được mã hóa thành lớp **Suspect**, còn các trạng thái còn lại được mã hóa thành lớp **Normal**. **Attack** không phải là một cụm K-Means hoặc một trạng thái hữu hạn được lưu trong hàm lớp trạng thái; nó là kết quả của một transition lookup thất bại, tức là không tồn tại bước chuyển hợp lệ cho cặp `(state, token)` hiện tại. Cách tách này giữ cho DFA là whitelist hành vi bình thường: trạng thái hiếm nhưng đã từng quan sát được thì chuyển lên user space để kiểm tra thêm, còn bước chuyển chưa từng quan sát được thì bị coi là bất thường cấu trúc.

3.5 Giai đoạn 4 - Thực thi tại Runtime bằng eBPF

3.5.1 Bước DFA trên mỗi sự kiện syscall

Dựa trên nền tảng lý thuyết về eBPF và BPF Maps đã trình bày trong §2.5, chương trình eBPF thực hiện các bước sau trên mỗi sự kiện syscall:

1. Đọc `current_state` từ `thread_state[TID]`.
2. Tra cứu `token_id = vocab[Syscall_Name]`.
3. Tra cứu `next_state = transition_table[(current_state, token_id)]`.
4. Nếu không tồn tại mục nhập: chuyển sang trạng thái tấn công (**Attack State**) - đây là tín hiệu phát hiện bất thường.
5. Nếu tồn tại mục nhập: đọc `state_class[next_state]` để xác định trạng thái đích là **Normal** hay **Suspect**.
6. Ghi `next_state` vào `thread_state[TID]`; nếu lớp đích là **Suspect** thì đồng thời gửi sự kiện lên Rust Agent để đánh giá sâu hơn.

Thuật toán 3 mô tả chi tiết logic xử lý dưới dạng pseudocode, bao gồm các trường hợp đặc biệt (token ngoài từ vựng, bước chuyển không tồn tại) và hành động tương ứng.

Algorithm 3: Trình xử lý eBPF: xử lý mỗi sự kiện syscall

Input: Sự kiện syscall e với các trường $e.tid$ và $e.name$

```

state  $\leftarrow$  bpf_map_lookup(thread_state,  $e.tid$ )
if state = NULL then state  $\leftarrow$   $q_0$  // Khởi tạo trạng thái đầu

tok  $\leftarrow$  bpf_map_lookup(vocab_map,  $e.name$ )
if tok = NULL then
    // Token ngoài từ vựng (OOV): chuyển tiếp lên
    // Rust Agent để xử lý Suspect
    bpf_ringbuf_output(suspect_buf,  $e$ )
    return
end

next  $\leftarrow$  bpf_map_lookup(transition_table, (state, tok))
if next = NULL then
    // Bước chuyển không tồn tại  $\rightarrow$  tấn công
    bpf_map_update(thread_state,  $e.tid$ ,  $q_{attack}$ )
    bpf_perf_event_output(alert_buf,  $e$ )
    if policy = BLOCK then bpf_send_signal(SIGKILL)
    return
else
    bpf_map_update(thread_state,  $e.tid$ , next)
end

class  $\leftarrow$  bpf_map_lookup(state_class, next)
if class = SUSPECT then
    // Trạng thái đích hiếm gặp
    bpf_ringbuf_output(suspect_buf,  $e$ )
    return
else
    // Trạng thái đích bình thường
    return
end

```

Chi phí xử lý chỉ gồm hai phép tra cứu BPF Map $O(1)$ trên mỗi syscall - không cần chuyển đổi ngữ cảnh (context switch) sang user space cho các bước chuyển bình thường. So với chi phí suy luận Transformer $O(L \cdot W^2 \cdot d)$, đây là sự giảm thiểu chi phí runtime về mặt bậc độ phức tạp, phù hợp với yêu cầu giám sát trong nhân hệ điều hành [5], [11].

3.5.2 Chính sách xử lý theo mức độ nghi ngờ

Thay vì đưa ra quyết định nhị phân normal/attack đơn giản, hệ thống phân biệt ba nhánh xử lý độc lập dựa trên mức độ nghi ngờ, nhằm giảm thiểu cảnh báo nhầm trong khi vẫn đảm bảo khả năng phát hiện nhanh:

Bảng 3.4: Ba mức phản ứng của chương trình eBPF tại runtime.

Mức	Điều kiện kích hoạt	Hành động
Bình thường	Bước chuyển trạng thái tiếp theo được ghi nhận là hợp lệ	Tiếp tục bình thường; chi phí duy nhất là cập nhật trạng thái trong BPF map.
Nghi vấn	Trạng thái đích có <code>state_class = 1</code> , hoặc token syscall không có trong bảng chữ cái ($\notin \Sigma$)	Thu thập cửa sổ syscall và chuyển tới Rust Agent để Transformer đánh giá lại. Không chặn luồng tiến trình.
Tấn công	Không tồn tại bước chuyển hợp lệ trong bảng trạng thái cho cặp (trạng thái, token) hiện tại	Chặn, kết thúc tiến trình hoặc ghi cảnh báo tùy chính sách cấu hình.

Mức nghi vấn đóng vai trò vùng đệm quan trọng: thay vì chặn ngay các hành vi hiểm gặp nhưng có thể hợp lệ, hệ thống chuyển chúng sang Teacher để xem xét kỹ hơn. Nếu Teacher xác nhận đây là tấn công thực sự, cảnh báo được ghi nhận; nếu xác nhận là cảnh báo nhầm, DFA được cập nhật để chấp nhận hành vi mới đó. Cơ chế này cho phép hệ thống hoạt động thận trọng mà không làm giảm khả năng phát hiện mối đe dọa thực sự.

3.5.3 Khả năng kháng mimicry attacks

Một lớp tấn công né tránh phổ biến đối với các hệ thống HIDS dựa trên chuỗi syscall là **mimicry attacks** [1], [13]: kẻ tấn công chèn thêm các syscall trông có vẻ lành tính (syscall padding) vào chuỗi khai thác để kéo giãn tín hiệu bất thường qua nhiều cửa sổ, giảm điểm bất thường của từng window riêng lẻ xuống dưới ngưỡng phát hiện.

Với cấu trúc DFA whitelist, lớp tấn công này bị vô hiệu hóa một cách tự nhiên: DFA không chỉ kiểm tra sự xuất hiện của từng token đơn lẻ mà kiểm tra toàn bộ **lộ trình chuyển trạng thái**. Khi chuỗi khai thác được độn thêm syscall bình thường, con trỏ trạng thái DFA phải đi theo một lộ trình mà các bước chuyển của nó không có trong tập huấn luyện. DFA sẽ gặp bước chuyển không xác định và chuyển sang Attack State, bất kể độ dài hay bản chất của các syscall đệm. Tính chất này là hệ quả trực tiếp từ cấu trúc whitelist của DFA, không phải là một quy tắc riêng biệt cần lập trình thêm.

Giới hạn cần lưu ý là các cuộc tấn công được thiết kế đặc biệt để bảo toàn toàn bộ lộ trình DFA - tức là kẻ tấn công có tri thức về cấu trúc DFA và chèn padding theo cách duy trì tất cả các bước chuyển hợp lệ - về nguyên tắc có thể vượt qua cơ chế này. Phân tích chi tiết về các điều kiện và giới hạn của khả năng kháng mimicry được trình bày trong Chương 4.

3.5.4 Vòng lặp cập nhật liên tục

Hệ thống hỗ trợ cập nhật DFA không cần tải lại kernel thông qua cơ chế cập nhật nguyên tử của BPF Maps như đã mô tả trong §2.5. Khi mô hình Transformer xác định một window Suspect là false positive - chẳng hạn do cập nhật phần mềm đưa vào hành vi bình thường mới - quá trình điều chỉnh diễn ra như sau:

1. Mẫu chuyển đổi mới được bổ sung vào tập huấn luyện.
2. DFA được trích xuất lại (Giai đoạn 3) hoặc cập nhật tăng dần trên tập con mới.
3. Các mục nhập `transition_table` mới được ghi nguyên tử vào BPF Map từ user space.
4. Con trở trạng thái của từng TID tiếp tục từ vị trí hiện tại mà không gián đoạn dịch vụ.

Vòng lặp phản hồi này cho phép hệ thống thích nghi với concept drift [21], [22] - sự dịch chuyển phân phối hành vi theo thời gian do cập nhật phần mềm hoặc thay đổi khối lượng công việc - mà không yêu cầu dừng dịch vụ hay tải lại module kernel, duy trì tính liên tục của giám sát bảo mật.

CHAPTER 4. PHÂN TÍCH LÝ THUYẾT

Chương 4 trình bày các phân tích lý thuyết chuyên sâu nhằm đánh giá các thuộc tính dự kiến của kiến trúc phát hiện xâm nhập lai Guepard Shield. Các nội dung phân tích trong chương này đóng vai trò làm khung lý thuyết định hướng cho việc thiết kế, tối ưu hóa siêu tham số và đánh giá các giới hạn về mặt toán học của hệ thống. Các lập luận lý thuyết này tập trung vào năm khía cạnh cốt lõi bao gồm: độ phức tạp tính toán ở các pha vận hành khác nhau, giới hạn dung lượng lưu trữ của bảng chuyển trạng thái trên bộ nhớ nhân hệ điều hành thông qua cơ chế eBPF, mối quan hệ giữa phân phối lời gọi hệ thống và tỷ lệ xung đột trong quá trình rời rạc hóa trạng thái ẩn, khả năng kháng cự của automata đối với các kỹ thuật tấn công bắt chước (mimicry attack), và cuối cùng là giới hạn trên của độ trung thực (fidelity) của mô hình Student so với mô hình Teacher. Cần lưu ý rằng các kết quả phân tích trong chương này đóng vai trò làm nền tảng lý thuyết và định hướng thiết kế cho hệ thống, đồng thời các mô hình phân tích lý thuyết này sẽ được đối chiếu và kiểm chứng trực tiếp với kết quả thực nghiệm từ đường ống (pipeline) chưng cất DFA đã hiện thực hóa hoàn chỉnh được trình bày trong Chương 5.

4.1 Độ phức tạp tính toán (Computational Complexity)

Một trong những động lực chính của kiến trúc Guepard Shield là giải quyết bài toán đánh đổi giữa khả năng biểu diễn ngữ cảnh mạnh mẽ của các mô hình học sâu và yêu cầu về độ trễ cực thấp trong môi trường thực thi của nhân hệ điều hành. Phần này phân tích độ phức tạp tính toán của ba thành phần chính trong hệ thống: quá trình suy luận của mô hình Transformer, quá trình duyệt automata tại thời điểm chạy (runtime), và quá trình gom cụm dữ liệu ngoại tuyến.

4.1.1 Transformer Inference (Ngoại tuyến)

Mô hình học sâu đóng vai trò là Teacher trong Guepard Shield là một kiến trúc mạng Transformer Decoder-only. Trong giai đoạn suy luận (inference), khi mô hình thực hiện một lượt truyền thẳng (forward pass) trên một cửa sổ trượt gồm W token đầu vào với kích thước nhúng (model dimension) là d và số lượng lớp attention là L , độ phức tạp tính toán được xác định dựa trên hai thành phần chính của cơ chế tự chú ý (Self-Attention) và mạng truyền thẳng (Feed-Forward Network - FFN).

Đối với mỗi lớp Self-Attention, việc tính toán các ma trận Query, Key, Value yêu cầu chi phí là $\mathcal{O}(W \cdot d^2)$. Bước tính toán bản đồ chú ý (attention map) thông qua tích vô hướng của Query và Key có độ phức tạp là $\mathcal{O}(W^2 \cdot d)$, và việc nhân bản đồ chú ý này với ma trận Value yêu cầu thêm $\mathcal{O}(W^2 \cdot d)$ phép tính. Cuối cùng, tầng FFN áp dụng các phép biến đổi tuyến tính lên từng token một cách độc lập với chi

phí $\mathcal{O}(W \cdot d^2)$. Do đó, tổng độ phức tạp tính toán cho một lượt truyền thẳng qua L lớp được biểu diễn dưới dạng:

$$\mathcal{O}(L \cdot W^2 \cdot d + L \cdot W \cdot d^2) \quad (4.1)$$

Trong thực tế, khi chiều dài ngữ cảnh W tăng lên, thành phần phi tuyến bậc hai $\mathcal{O}(L \cdot W^2 \cdot d)$ do cơ chế tính toán attention đầy đủ (dense attention) sẽ nhanh chóng trở thành nút thắt cổ chai tính toán chính. Ngay cả đối với cấu hình tham chiếu của mô hình Teacher được sử dụng trong nghiên cứu này (với kích thước cửa sổ $W = 64$, số chiều nhúng $d = 128$, và số lớp $L = 4$), số lượng phép toán dấu phẩy động (FLOPs) ước tính cho mỗi lượt truyền thẳng cũng đạt mức khoảng $6,29 \times 10^6$ phép tính. Đối với các cấu hình mô hình lớn hơn (ví dụ $W = 128$, $d = 256$, $L = 6$), con số này sẽ nhanh chóng tăng lên hàng chục triệu FLOPs cho mỗi cửa sổ trượt.

Với năng lượng tính toán hiện tại của các bộ vi xử lý máy chủ (CPU), việc thực hiện hàng triệu FLOPs này tạo ra độ trễ suy luận dao động trong khoảng vài mili giây cho mỗi cửa sổ trượt. Trong khi đó, đường đi thực thi lời gọi hệ thống (syscall path) của nhân Linux yêu cầu các thao tác kiểm tra an ninh phải hoàn thành trong khoảng thời gian mức micro giây để tránh làm tắc nghẽn hệ thống và giảm hiệu năng chung của máy chủ. Do đó, việc chạy trực tiếp mô hình Transformer suy luận tại runtime trong không gian nhân là hoàn toàn bất khả thi.

4.1.2 Duyệt DFA (Runtime)

Để khắc phục hạn chế về mặt hiệu năng của mô hình học sâu, Guepard Shield chứng cất tri thức từ mô hình Teacher sang một automata hữu hạn đơn định (DFA). Tại thời điểm chạy (runtime), chương trình eBPF được gắn vào các tracepoints của syscall sẽ thực hiện việc kiểm tra chính sách bảo mật thông qua việc duyệt đồ thị trạng thái của DFA này.

Đối với mỗi sự kiện lời gọi hệ thống mới được kích hoạt bởi một tiến trình, chương trình eBPF chỉ thực hiện ba thao tác cơ bản sau:

1. **Truy vấn bước chuyển trạng thái (State Transition Lookup):** Chương trình sử dụng mã định danh trạng thái hiện tại q_t và token của syscall vừa xuất hiện s_{t+1} làm khóa để tra cứu trạng thái tiếp theo $q_{t+1} = \delta(q_t, s_{t+1})$ trong bảng chuyển trạng thái DFA. Thao tác này được thực hiện thông qua cấu trúc BPF Hash Map với độ phức tạp thời gian trung bình là $\mathcal{O}(1)$.
2. **Truy vấn phân loại an ninh (State Anomaly Class Lookup):** Sau khi xác định được trạng thái tiếp theo q_{t+1} , chương trình thực hiện một lần tra cứu BPF Map khác để xác định mức độ bất thường (phân loại an ninh) của trạng thái

này nhằm quyết định hành động xử lý (cho phép thực thi, gửi cảnh báo lên không gian người dùng, hoặc chặn tiến trình). Thao tác này cũng có độ phức tạp thời gian là $\mathcal{O}(1)$.

3. **Cập nhật trạng thái hiện tại (State Writeback):** Cuối cùng, trạng thái mới q_{t+1} được ghi đè vào BPF Map quản lý trạng thái hiện tại của luồng tiến trình (được định danh bởi Thread ID - TID) để chuẩn bị cho syscall tiếp theo. Chi phí cho thao tác ghi này là $\mathcal{O}(1)$.

Tổng hợp lại, tổng độ phức tạp tính toán tại runtime cho mỗi lời gọi hệ thống là:

$$\mathcal{O}(1) \tag{4.2}$$

Độ phức tạp $\mathcal{O}(1)$ này hoàn toàn độc lập với chiều dài cửa sổ ngữ cảnh W cũng như các siêu tham số cấu trúc L và d của mô hình Transformer Teacher. Điều này đảm bảo rằng độ trễ thực thi tại thời điểm chạy cực kỳ nhỏ và ổn định, đáp ứng hoàn hảo các ràng buộc khắt khe về thời gian thực của nhân hệ điều hành Linux.

Về mặt quyết định an ninh, kết quả của một bước duyệt DFA không phải là một nhãn học máy ba lớp được dự đoán trực tiếp. Nó được xác định bởi cấu trúc của bảng chuyển và lớp tần suất của trạng thái đích. Nếu tồn tại bước chuyển $\delta(q_t, s_{t+1}) = q_{t+1}$ và q_{t+1} thuộc lớp tần suất bình thường, sự kiện được phân loại là **normal**. Nếu bước chuyển tồn tại nhưng q_{t+1} thuộc nhóm trạng thái hiếm đã từng xuất hiện trong tập huấn luyện, sự kiện được phân loại là **suspect** và được chuyển lên user space để Teacher hoặc Rust Agent kiểm tra sâu hơn. Nếu không tồn tại bước chuyển cho cặp (q_t, s_{t+1}) , hệ thống phân loại sự kiện là **attack**. Do đó, attack là bất thường cấu trúc của automata, còn suspect là vùng biên tần suất thấp của hành vi đã quan sát được.

4.1.3 Trích xuất K-Means (Ngoại tuyến)

Mặc dù việc thực thi tại runtime có chi phí tối ưu, quá trình xây dựng DFA từ mô hình Transformer yêu cầu các bước tính toán ngoại tuyến (offline) ở không gian người dùng. Quá trình này bao gồm hai bước chính: trích xuất các vector ẩn và huấn luyện thuật toán gom cụm K-Means.

Bước thứ nhất là thu thập các trạng thái ẩn (hidden states) từ mô hình Transformer. Đối với một tập dữ liệu huấn luyện gồm N cửa sổ trượt, mô hình Teacher cần thực hiện một lượt truyền thẳng trên toàn bộ tập dữ liệu này để trích xuất các vector biểu diễn tại lớp attention cuối cùng. Độ phức tạp tính toán của bước này là $\mathcal{O}(N \cdot L \cdot W^2 \cdot d)$.

Bước thứ hai là áp dụng thuật toán K-Means để rời rạc hóa không gian vector liên tục này thành K cụm, tương ứng với K trạng thái của DFA. Trong mỗi vòng lặp của thuật toán K-Means, khoảng cách từ N mẫu dữ liệu đến K tâm cụm trong không gian d chiều cần được tính toán, dẫn đến độ phức tạp cho mỗi vòng lặp là $\mathcal{O}(N \cdot K \cdot d)$. Nếu thuật toán hội tụ sau I lần lặp, tổng chi phí tính toán cho quá trình gom cụm là $\mathcal{O}(N \cdot K \cdot d \cdot I)$.

Do đó, tổng độ phức tạp tính toán ngoại tuyến của quá trình trích xuất DFA được biểu diễn bởi:

$$\mathcal{O}(N \cdot L \cdot W^2 \cdot d + N \cdot K \cdot d \cdot I) \quad (4.3)$$

Một khía cạnh quan trọng khác là độ phức tạp không gian lưu trữ trung gian. Nếu trích xuất toàn bộ các trạng thái ẩn một cách dày đặc ở bước nhảy $S = 1$ để phục vụ cho cả gom cụm và xây dựng bước chuyển, dung lượng đĩa cần thiết sẽ là $\mathcal{O}(N_{\text{dense}} \cdot d)$, điều này gây ra bùng nổ dung lượng lưu trữ trên đĩa cứng đối với các tập dữ liệu lớn. Để giải quyết nút thắt này, hệ thống áp dụng cơ chế trích xuất hai pha (two-pass streaming): pha gom cụm sử dụng bước nhảy thưa $S_{\text{extract}} > 1$ giúp giảm độ phức tạp không gian lưu trữ xuống $\mathcal{O}(N_{\text{sparse}} \cdot d)$ (với $N_{\text{sparse}} = N_{\text{dense}}/S_{\text{extract}}$), và pha xây dựng bước chuyển thực hiện stream trực tiếp dữ liệu với bước nhảy $S = 1$ để ánh xạ on-the-fly sang các tâm cụm đã biết. Phương pháp này giảm độ phức tạp lưu trữ trung gian thực tế khi dựng bảng chuyển trạng thái về mức $\mathcal{O}(K \cdot d)$ cho các tâm cụm và $\mathcal{O}(|E|)$ cho bảng đếm chuyển tiếp, thay vì phải lưu toàn bộ N_{dense} vector ẩn. Sau khi export, dung lượng bảng chuyển bị chặn trên bởi $\mathcal{O}(K \cdot |\Sigma|)$, giúp tối ưu hóa đáng kể tài nguyên I/O ngoại tuyến.

Khi sử dụng biến thể mini-batch của K-Means, độ phức tạp trên không còn phụ thuộc vào việc tất cả N vector phải cùng nằm trong RAM. Nếu mỗi lần cập nhật chỉ đọc một batch kích thước B , bộ nhớ làm việc của bước gom cụm xấp xỉ $\mathcal{O}(B \cdot d + K \cdot d)$, gồm batch hiện tại và các tâm cụm. Sau khi hội tụ, bước gán nhãn cũng có thể thực hiện theo chunk: mỗi vector ẩn được ánh xạ vào tâm cụm gần nhất, nhãn cụm được ghi tuần tự ra bộ lưu trữ trung gian, còn tâm cụm được giữ lại để phục vụ pha stream stride-1. Đây là lý do pipeline có thể huấn luyện K-Means từ hidden states của Transformer trên tập lớn mà không cần giữ toàn bộ ma trận hidden state ở độ chính xác tính toán đầy đủ trong bộ nhớ.

Vì toàn bộ quá trình này được thực hiện ngoại tuyến tại không gian người dùng và chỉ cần chạy một lần duy nhất trong pha tiền xử lý và huấn luyện thiết lập hệ thống, chi phí tính toán này không ảnh hưởng đến hiệu năng vận hành trực tuyến

của máy chủ.

4.2 Kích thước DFA và Bộ nhớ eBPF (DFA Size and eBPF Memory)

Một trong những ràng buộc kỹ thuật lớn nhất khi triển khai các giải pháp an ninh trong nhân Linux thông qua eBPF là giới hạn về dung lượng bộ nhớ. Tuy nhiên, trong thực tế vận hành, bảng chuyển trạng thái DFA có tính chất thưa thớt (sparsity) cực kỳ cao. Phân phối các lời gọi hệ thống của một chương trình máy chủ thông thường chỉ tập trung vào một tập con rất nhỏ của Σ , và các chuỗi chuyển dịch trạng thái hợp lệ chỉ chiếm một phần rất nhỏ trong không gian tích bộ đôi $Q \times \Sigma$. Nếu sử dụng cấu trúc Hash Map (`BPF_MAP_TYPE_HASH`), các cặp trạng thái và token không xuất hiện trong dữ liệu huấn luyện bình thường sẽ không được điền vào bản đồ. Khi chương trình eBPF tra cứu một cặp không tồn tại, cơ chế mặc định sẽ coi đó là một bước chuyển sang trạng thái tấn công (attack state) hoặc kích hoạt cảnh báo bất thường. Do đó, số lượng mục nhập thực tế cần lưu trữ trong Hash Map nhỏ hơn rất nhiều so với giới hạn lý thuyết, giúp tiết kiệm bộ nhớ.

Mặc dù Hash Map tiết kiệm bộ nhớ nhờ tính thưa thớt, phương án thay thế tối ưu hơn về mặt tốc độ truy xuất là cấu trúc mảng hai chiều trực tiếp (Direct-indexed 2D Array) sử dụng `BPF_MAP_TYPE_ARRAY` kích thước $K \times M_{\text{syscall}}$, trong đó mỗi phần tử lưu trữ chỉ số trạng thái tiếp theo dưới dạng số nguyên 4 bytes (`u32` hoặc `s32`). Dung lượng bộ nhớ lý thuyết cần thiết cho mảng trực tiếp này được xác định bởi:

$$\text{Memory}_{\text{Array}} = K \times M_{\text{syscall}} \times 4 \text{ bytes} \quad (4.4)$$

Với M_{syscall} là giới hạn chỉ số lời gọi hệ thống trong nhân hệ điều hành. Ngay cả đối với các cấu hình lý thuyết lớn (ví dụ $K = 512$ và $M_{\text{syscall}} \approx 470$), dung lượng bộ nhớ cần thiết chỉ vào khoảng 940 KB, hoàn toàn nằm trong giới hạn phân bổ tài nguyên của eBPF Map. Đối với các cấu hình nhỏ hơn, mức tiêu thụ bộ nhớ sẽ giảm tỷ lệ thuận và chỉ chiếm một vài trăm kilobytes. Ưu điểm vượt trội của phương án mảng trực tiếp là loại bỏ hoàn toàn chi phí tính toán hash và xung đột khóa của Hash Map, chuyển đổi thao tác kiểm tra an ninh thành một phép tính offset địa chỉ tuyến tính đơn giản với độ trễ tối thiểu và độ phức tạp truy xuất $O(1)$ thực sự. Các đánh giá so sánh hiệu năng chi tiết và số liệu thực nghiệm liên quan giữa hai cấu trúc này sẽ được phân tích sâu hơn trong Chương 5.

Bảng chuyển trạng thái của một DFA với tập trạng thái Q (kích thước $|Q| = K$) và bảng chữ cái đầu vào Σ (kích thước $|\Sigma|$) được mô tả dưới dạng một hàm số $\delta : Q \times \Sigma \rightarrow Q$. Trong trường hợp xấu nhất sử dụng cấu trúc Hash Map, số lượng

mục nhập (entries) tối đa cần lưu trữ trong bảng chuyển trạng thái là:

$$N_{\max} = K \times |\Sigma| \quad (4.5)$$

Với cấu hình tham chiếu của hệ thống gồm kích thước bộ từ vựng $|\Sigma| = 470$ và số lượng trạng thái DFA tối đa $K = 512$, số lượng mục nhập tối đa trong bảng chuyển băm là 240.640 mục nhập. Khóa (key) của bản đồ cần lưu trữ cặp trạng thái hiện tại và token đầu vào có kích thước 8 bytes (sử dụng kiểu `u32` cho mỗi trường), còn giá trị (value) lưu trữ trạng thái tiếp theo có kích thước 4 bytes (`u32`). Tổng dung lượng bộ nhớ lý thuyết cần thiết để lưu trữ toàn bộ bảng chuyển trong trường hợp xấu nhất của Hash Map ước tính khoảng $240.640 \times 12 \text{ bytes} \approx 2,89 \text{ MB}$. Đây là dung lượng rất nhỏ và nằm hoàn toàn trong giới hạn cho phép của nhân Linux, đặc biệt khi cơ chế băm chỉ lưu các bước chuyển thừa thực sự xuất hiện trong dữ liệu huấn luyện. Tuy nhiên, mảng hai chiều trực tiếp (`BPF_MAP_TYPE_ARRAY`) cho phép nâng hiệu năng lên tối đa nhờ truy cập bộ nhớ trực tiếp $O(1)$ mà không cần tính toán hash, với dung lượng bộ nhớ tăng lên không đáng kể và hoàn toàn nằm dưới giới hạn tài nguyên của eBPF Map. Các so sánh chi tiết và số liệu thực nghiệm liên quan sẽ được trình bày cụ thể trong Chương 5.

Cấu hình runtime của DFA là một biểu diễn phẳng của automata. Về mặt lý thuyết, nó gồm bốn thành phần chính: trạng thái bắt đầu q_0 , bảng chuyển $T[q, s]$, ánh xạ syscall-to-token, và hàm lớp trạng thái $C(q)$. Trong biểu diễn mảng trực tiếp, $T[q, s] = -1$ được dùng để biểu diễn $\delta(q, s)$ không xác định. Hàm $C(q)$ chỉ cần mã hóa hai lớp hữu hạn là normal và suspect; lớp attack được suy ra từ $T[q, s] = -1$. Tách riêng hai cơ chế này có lợi cho triển khai eBPF vì tra cứu bước chuyển và tra cứu lớp trạng thái đều là phép đọc mảng hằng thời gian, đồng thời tránh phải thêm một trạng thái attack nhân tạo vào không gian cụm K-Means.

4.3 Tỷ lệ không xác định và Lựa chọn chiến lược (Conflict Rate and Policy Selection)

Quá trình rời rạc hóa không gian trạng thái ẩn liên tục của Transformer thành các trạng thái DFA rời rạc thông qua gom cụm K-Means có thể dẫn đến hiện tượng không đơn định (non-determinism). Hiện tượng này xảy ra khi các cửa sổ ngữ cảnh khác nhau được ánh xạ vào cùng một cụm (trạng thái DFA) nhưng lại chuyển sang các cụm khác nhau khi quan sát cùng một syscall tiếp theo. Để kiểm soát hiện tượng này, phần này định nghĩa chỉ số tỷ lệ xung đột và phân tích tác động của siêu tham số.

Định nghĩa **tỷ lệ xung đột (conflict rate)** của quá trình trích xuất DFA từ dữ

liệu huấn luyện được hình thức hóa như sau:

$$\text{conflict_rate} = \frac{|\{(q, s) \in Q \times \Sigma : |\{\delta_{\text{temp}}(q, s)\}| > 1\}|}{|\{(q, s) \in Q \times \Sigma : \delta_{\text{temp}}(q, s) \neq \emptyset\}|} \quad (4.6)$$

Trong đó $\delta_{\text{temp}}(q, s)$ là tập hợp các trạng thái tiếp theo được quan sát trong dữ liệu huấn luyện khi hệ thống ở trạng thái q và nhận đầu vào là syscall s .

Trong pipeline đã hiện thực hóa, tập $\delta_{\text{temp}}(q, s)$ được xây dựng dưới dạng bộ đếm chứ không chỉ là tập hợp nhị phân. Mỗi lần quan sát một đoạn liên tiếp (q_t, s_{t+1}, q_{t+1}) , hệ thống tăng bộ đếm cho nhánh q_{t+1} của khóa (q_t, s_{t+1}) . Nhờ vậy, cùng một quan hệ NFA thô có thể được giải quyết theo nhiều chính sách khác nhau: subset construction giữ toàn bộ tập đích, majority voting giữ nhánh xuất hiện nhiều nhất, còn statistical pruning chỉ giữ nhánh dominant nếu tỷ lệ của nó vượt ngưỡng θ .

Khẳng định: Đối với dữ liệu chuỗi syscall có phân phối bước chuyển bị lệch mạnh (strongly skewed distribution), tỷ lệ xung đột của automata giảm đơn điệu theo số lượng cụm K-Means K và được giảm thiểu hiệu quả thông qua chiến lược cắt tỉa thống kê S4 (Statistical State Transition Pruning).

Lập luận chứng minh: Nếu hai chuỗi syscall khác nhau đạt đến cùng một cụm trạng thái ẩn q thông qua các lịch sử thực thi khác nhau, nhưng chúng có hành vi chuyển tiếp giống hệt nhau trong tương lai (tức là cùng dẫn đến việc chuyển dịch sang một trạng thái tiếp theo duy nhất cho mỗi syscall đầu vào), thì không có xung đột nào phát sinh trong bảng chuyển trạng thái. Xung đột chỉ thực sự xảy ra khi thuật toán K-Means gộp hai lịch sử thực thi khác nhau vào cùng một cụm q , trong khi mô hình Transformer Teacher có khả năng phân biệt chúng dựa trên ngữ cảnh dài hạn và đưa ra các dự đoán xác suất khác nhau cho syscall tiếp theo.

Khi tăng giá trị K (số lượng tâm cụm của K-Means), ranh giới phân mảnh của không gian trạng thái ẩn liên tục sẽ trở nên mịn hơn. Điều này trực tiếp làm giảm xác suất gộp các lịch sử thực thi có hành vi tương lai khác nhau vào cùng một cụm, từ đó làm giảm tỷ lệ xung đột conflict_rate một cách đơn điệu.

Bên cạnh đó, chiến lược cắt tỉa thống kê S4 đóng vai trò loại bỏ các xung đột giả tạo phát sinh do nhiễu lượng tử hóa. Trong quá trình gom cụm, một số bước chuyển trạng thái có tần suất xuất hiện cực kỳ thấp (nhỏ hơn ngưỡng cắt tỉa θ) có thể được tạo ra do các điểm biên của cụm bám sát các mẫu nhiễu trong dữ liệu huấn luyện. Bằng cách áp dụng ngưỡng lọc θ , chiến lược S4 loại bỏ các bước chuyển tần suất thấp này ra khỏi bảng chuyển tiếp DFA, giữ lại các bước chuyển chính chiếm ưu

thể thống kê, từ đó triệt tiêu các xung đột mà không làm mất đi các hành vi bình thường cốt lõi của tiến trình.

Phương pháp đối chiếu thực nghiệm: Để kiểm chứng các lập luận lý thuyết nêu trên, hệ thống thực hiện đo đạc thực nghiệm tỷ lệ xung đột, tỷ lệ báo động giả (FPR) và độ trung thực (fidelity) của DFA Student so với Transformer Teacher trên các miền cấu hình siêu tham số của số lượng trạng thái K và ngưỡng cắt tủa thống kê θ . Các kết quả đo đạc định lượng, bảng biểu đối chiếu chi tiết và đồ thị liên quan cho từng chiến lược giải quyết tính không xác định sẽ được trình bày và phân tích chi tiết trong Chương 5, làm cơ sở thực chứng để lựa chọn cấu hình tối ưu cho hệ thống thực tế.

4.4 Khả năng kháng Mimicry Attack (Resistance to Mimicry Attack)

Mimicry Attack là một kỹ thuật tấn công tinh vi nhắm vào các hệ thống phát hiện bất thường dựa trên hành vi. Kẻ tấn công cố gắng ngụy trang chuỗi các syscall độc hại bằng cách chèn thêm các syscall vô hại (padding syscalls hoặc no-op syscalls như `getpid`, `nanosleep`) vào giữa các lệnh khai thác thực tế. Mục tiêu của việc chèn này là làm giãn cách chuỗi khai thác để các cửa sổ trượt thu thập dữ liệu tại runtime không chứa đủ tín hiệu bất thường, từ đó vượt qua bộ lọc phát hiện.

Khẳng định: Việc chèn các syscall đệm độc lập với số lượng k sẽ làm cho con trỏ trạng thái hiện tại của DFA trôi dạt vào các trạng thái nghi vấn (suspect states) có tần suất xuất hiện thấp trong dữ liệu bình thường, hoặc gặp một bước chuyển không tồn tại và bị phân loại là attack.

Lập luận chứng minh: Giả sử chuỗi syscall khai thác độc hại ban đầu là $e_1, e_2, \dots, e_m$. Kẻ tấn công thực hiện chèn k syscall đệm $p_1, p_2, \dots, p_k$ vào giữa e_i và e_{i+1} . Để cuộc tấn công không bị phát hiện ngay lập tức bởi whitelist của DFA, mỗi syscall đệm p_j bắt buộc phải đại diện cho một bước chuyển hợp lệ từ trạng thái hiện tại của DFA.

Tuy nguyên tắc này đơn giản, do các trạng thái nằm trên lộ trình khai thác (exploit path) tương ứng với chuỗi $e_1, \dots, e_i$ có bản chất là bất thường và không xuất hiện trong tập dữ liệu huấn luyện bình thường (mô hình Teacher chỉ học phân phối hành vi bình thường), việc thực hiện một syscall đệm p_1 (dù là một syscall bình thường trong ngữ cảnh khác) từ một trạng thái thuộc exploit-path sẽ dẫn DFA đến một trạng thái mới q' . Trạng thái q' này chưa từng được quan sát thấy trong pha huấn luyện lạnh tính do sự kết hợp bất thường giữa ngữ cảnh lịch sử khai thác và syscall đệm.

Hệ quả là, trạng thái q' sẽ rơi vào một trong hai trường hợp sau:

1. Gặp phân loại attack do không tồn tại bước chuyển đi hợp lệ cho syscall tiếp theo trong bảng chuyển trạng thái DFA.
2. Trở thành một trạng thái nghi vấn (suspect state) có xác suất chuyển tiếp thấp. Khi kẻ tấn công tiếp tục thực thi các syscall khai thác tiếp theo e_{i+1} , bước chuyển dịch từ q' qua e_{i+1} sẽ bị coi là bất thường và kích hoạt cảnh báo.

Cơ chế này đảm bảo DFA duy trì khả năng phát hiện độc lập với số lượng syscall đệm k được chèn vào hệ thống.

Giới hạn lý thuyết: Khả năng kháng cự này phụ thuộc nghiêm ngặt vào điều kiện các trạng thái trên lộ trình khai thác độc hại được phân tách rõ ràng với các trạng thái bình thường trong không gian nhúng của Transformer. Nếu số lượng cụm K của thuật toán K-Means được chọn quá nhỏ, lỗi lượng tử hóa sẽ tăng lên, dẫn đến việc K-Means gom cụm các trạng thái bình thường và các trạng thái khai thác vào chung một thực thể trạng thái DFA. Khi đó, kẻ tấn công có thể lợi dụng các bước chuyển bình thường có sẵn của trạng thái bị gộp này để thực hiện cuộc tấn công bất chước thành công. Điều này nhấn mạnh tầm quan trọng của việc lựa chọn tham số K đủ lớn để duy trì ranh giới quyết định sắc bén cho DFA.

4.5 Giới hạn độ trung thực của DFA (DFA Fidelity Bound)

Độ trung thực (Fidelity) là chỉ số đo lường mức độ tương đồng giữa quyết định phân loại của mô hình Student (DFA) và mô hình Teacher (Transformer). Trong bài toán phát hiện bất thường, Fidelity được định nghĩa là tỷ lệ phần trăm các cửa sổ trượt trong tập kiểm thử mà cả DFA và Transformer đồng thuận về phán quyết bất thường (tức là cả hai cùng phân loại cửa sổ đó là bình thường hoặc cùng phân loại là bất thường dựa trên các ngưỡng quyết định tương ứng).

Giới hạn trên của Fidelity: Độ trung thực tối đa mà DFA có thể đạt được bị giới hạn trên bởi lỗi lượng tử hóa (quantization error) của thuật toán gom cụm K-Means khi ánh xạ không gian vector liên tục sang tập trạng thái rời rạc.

Xét hai cửa sổ trượt syscall w_1 và w_2 tạo ra cùng một chuỗi các trạng thái DFA (tức là chúng có cùng một quỹ đạo duyệt đồ thị trạng thái của automata). Tuy nhiên, do mô hình Transformer hoạt động trên không gian vector liên tục, điểm số bất thường (Negative Log-Likelihood - NLL) mà Transformer tính toán cho hai cửa sổ này có thể khác nhau:

$$\text{NLL}_{\text{Transformer}}(w_1) \neq \text{NLL}_{\text{Transformer}}(w_2) \quad (4.7)$$

Nếu điểm số NLL của w_1 nằm dưới ngưỡng phát hiện bất thường τ nhưng điểm

số NLL của w_2 lại nằm trên ngưỡng τ , mô hình Transformer Teacher sẽ đưa ra hai phán quyết khác nhau cho hai cửa sổ này. Tuy nhiên, vì cả hai cửa sổ đều có chung quỹ đạo trạng thái trong DFA, mô hình DFA Student buộc phải đưa ra cùng một phán quyết cho cả hai. Hiện tượng này được gọi là **va chạm lượng tử hóa (quantization collision)**.

Sự mất mát độ trung thực (loss of fidelity) tỷ lệ thuận với tần suất xuất hiện của các va chạm lượng tử hóa này trên tập dữ liệu kiểm thử. Để giảm thiểu va chạm, ta cần tăng số lượng trạng thái DFA K để thu hẹp khoảng cách lượng tử hóa và tăng độ mịn của ranh giới quyết định. Tuy nhiên, việc tăng K lại làm tăng kích thước bộ nhớ cần thiết để lưu trữ bảng chuyển trạng thái trên eBPF Maps như đã phân tích ở Mục 4.2.

Do đó, bài toán tối ưu hóa siêu tham số của Guepard Shield là tìm kiếm một giá trị K tối ưu nhằm cực tiểu hóa hàm mục tiêu kết hợp giữa tỷ lệ báo động giả (FPR) và sai số độ trung thực:

$$\min_K [\text{FPR}(K) + (1 - \text{Fidelity}(K))] \quad (4.8)$$

Dưới ràng buộc về mặt dung lượng lưu trữ của BPF Maps trong nhân hệ điều hành:

$$\text{Memory}(K) \leq M_{\text{limit}} \quad (4.9)$$

Việc giải quyết bài toán tối ưu hóa ràng buộc này cung cấp cơ sở toán học vững chắc để cân bằng giữa hiệu năng phát hiện bất thường và tính khả thi khi triển khai hệ thống trong nhân Linux thực tế.

CHAPTER 5. THỰC NGHIỆM

Chương 5 trình bày các kết quả thực nghiệm của pipeline Guepard Shield trên bộ dữ liệu LID-DS-2021. Mục tiêu của chương này là đối chiếu các giả thuyết thiết kế đã trình bày trong Chương 3 và Chương 4 với số liệu đo được từ hai giai đoạn chính: huấn luyện Transformer Teacher để tạo điểm bất thường trên các window syscall và chưng cất mô hình Student dưới dạng DFA để kiểm tra bước chuyển trạng thái. Trước khi trình bày kết quả, chương này làm rõ bộ dữ liệu, quy ước gán nhãn và giao thức đánh giá, vì các con số AUROC, F1 hay tỷ lệ phát hiện chỉ có ý nghĩa đúng khi được đặt trong bối cảnh nhãn runtime của LID-DS.

5.1 Bộ dữ liệu

Các thí nghiệm sử dụng LID-DS-2021 [8] làm bộ dữ liệu đánh giá chính. Đây là bộ dữ liệu phát hiện xâm nhập máy chủ được thiết kế cho môi trường container hóa bằng Docker, gồm các bản ghi syscall từ những kịch bản lành tính và các kịch bản tấn công mô phỏng nhiều lỗ hổng CWE/CVE phổ biến trên máy chủ Linux. Khác với các bộ dữ liệu chỉ cung cấp chuỗi syscall rời rạc, LID-DS-2021 đi kèm định danh luồng (Thread ID - TID), tên tiến trình, nhãn thời gian và tham số syscall. Những trường này cho phép pipeline tách dòng sự kiện theo từng luồng thực thi, giảm hiện tượng trộn lẫn syscall giữa nhiều tiến trình hoặc nhiều thread trong cùng workload máy chủ.

Theo phân tích EDA của luận văn, LID-DS-2021 gồm 17,190 recordings, trong đó train và validation chỉ chứa normal recordings, còn toàn bộ exploit recordings nằm ở tập test. Cách chia này đặt bài toán vào khung phát hiện bất thường bán giám sát: mô hình chỉ học phân phối hành vi bình thường rồi phát hiện các sai lệch trong pha kiểm thử. Các chuỗi syscall của LID-DS-2021 rất dài; median recording của tập train xấp xỉ 95 nghìn sự kiện exit-syscall, p99 gần 936 nghìn sự kiện, và tập test có một outlier dài hơn 6.2 triệu syscall. Vì vậy, mọi mô hình đầu vào cố định đều cần cơ chế cắt window thay vì xử lý nguyên recording như một chuỗi duy nhất.

Không gian syscall của LID-DS-2021 tương đối gọn so với các bộ dữ liệu kernel-level: tập train có 99 syscall thực và một token <unknown>. Các syscall xuất hiện nhiều nhất gồm `futex`, `epoll_wait`, `sched_yield`, `read`, `lstat`, `fcntl`, `stat`, `fstat`, `lseek` và `newfstatat`, phản ánh workload máy chủ web I/O-bound trong container. Tập test có thêm 14 syscall không xuất hiện ở train, bao gồm một số syscall thường gắn với hành vi hậu khai thác như `vfork`, `mremap`, `sendmsg`, `setpgid` và `setsid`; các syscall này được ánh xạ về token unknown trong pipeline để giữ bảng chữ cái hữu hạn.

Các mô tả scenario trong `docs/lid-ds` cho thấy LID-DS-2021 không chỉ gồm một dạng tấn công đơn lẻ mà bao phủ nhiều lớp hành vi hệ thống khác nhau. Bảng 5.1 tóm tắt các nhóm kịch bản tiêu biểu.

Bảng 5.1: Các nhóm kịch bản tiêu biểu trong LID-DS-2021.

Nhóm hành vi	Kịch bản tiêu biểu	Ý nghĩa đối với syscall
Xác thực và cơ sở dữ liệu	Bruteforce_CWE-307, CVE-2012-2122	Lập đăng nhập, truy vấn SQL, thay đổi nhịp I/O và kết nối
Rò rỉ thông tin và path traversal	CVE-2014-0160, CVE-2017-7529, CVE-2018-3760, CVE-2019-5418	Đọc dữ liệu ngoài luồng bình thường, truy cập tệp hoặc vùng nhớ nhạy cảm
Remote code execution	CVE-2020-13942, CVE-2020-9484, CVE-2017-12635/12636	Tạo file, spawn process, mở shell hoặc thực thi lệnh ngoài workflow hợp lệ
Upload và xử lý nội dung nguy hiểm	PHP_CWE-434, EPS_CWE-434, ZipSlip, CVE-2020-23839	Upload payload, giải nén/ghi đè file, web shell và thao tác sau khai thác

Với mục tiêu của luận văn, LID-DS-2021 không chỉ là nguồn dữ liệu benchmark mà còn là một mô hình gần với bối cảnh vận hành: hệ thống phải quan sát một dòng syscall liên tục và phát hiện thời điểm hành vi bất thường xuất hiện. Vì vậy, các bản ghi bình thường được dùng để học manifold hành vi hợp lệ, còn các bản ghi tấn công được dùng để kiểm tra xem điểm NLL của Teacher hoặc bảng chuyển DFA có tạo cảnh báo khi chuỗi syscall đi ra khỏi vùng đã học hay không.

Một đặc điểm quan trọng của LID-DS là quy ước gán nhãn tấn công theo mốc `exploit_start`. Sau thời điểm khai thác, phần còn lại của bản ghi được xem là attack vì bộ dữ liệu không cung cấp mốc kết thúc exploit chính xác. Quy ước này phù hợp cho đánh giá ở mức bản ghi, nơi chỉ cần biết một recording có chứa tấn công hay không, nhưng gây nhiều khi chuyển sang đánh giá từng window: nhiều window nằm sau exploit có thể chỉ chứa hoạt động server bình thường nhưng vẫn mang nhãn attack. Do đó, chương này phân biệt rõ ba loại kết quả: chỉ số raw per-window trên nhãn gốc của LID-DS, phân tích định lượng về phần tail sau khai thác, và tỷ lệ phát hiện cấp độ bản ghi phù hợp hơn với mục tiêu runtime của DFA.

Về cấu trúc luồng, phần lớn recordings trong LID-DS-2021 có median thread count bằng 1 và tối đa 4 thread, tức dữ liệu gần với giám sát theo container cô lập hơn là thu thập toàn host. Attack timing metadata cho thấy các exploit thường bắt đầu khoảng 11 giây sau warmup và phần lớn nằm trong nửa đầu recording. Tuy nhiên, một số metadata thời gian có artifact, chẳng hạn fraction lớn hơn 1 khi timestamp exploit rơi ngoài đoạn trace; vì vậy luận văn ưu tiên vị trí theo syscall/window thay vì dùng time-fraction như đặc trưng mô hình.

5.2 Thiết lập đánh giá

Tập huấn luyện và tập kiểm định chỉ chứa các bản ghi bình thường, phù hợp với bài toán phát hiện bất thường không giám sát. Tập kiểm thử bao gồm cả bản ghi

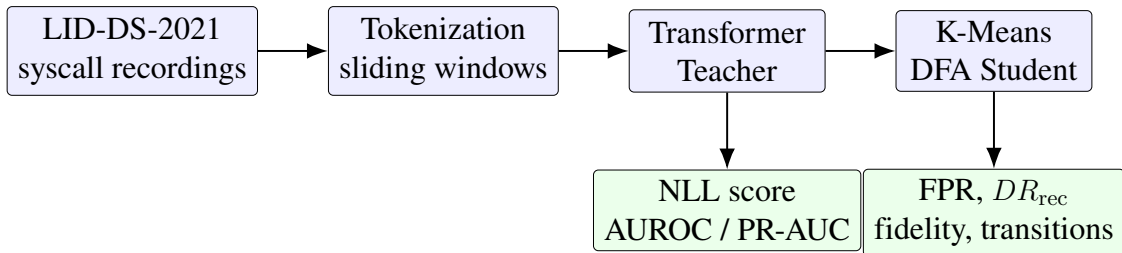
bình thường và bản ghi tấn công, được dùng để đánh giá khả năng xếp hạng bất thường của Transformer Teacher và khả năng gắn attack cho bước chuyển lạ của DFA Student. Bảng 5.2 tóm tắt số lượng bản ghi theo từng split.

Bảng 5.2: Phân chia bản ghi trong LID-DS-2021 được sử dụng trong thực nghiệm.

Split	Normal recordings	Attack recordings	Mục đích
Train	3,149	0	Huấn luyện Teacher và xây dựng DFA
Validation	885	0	Hiệu chuẩn ngưỡng và kiểm tra hội tụ
Test	11,341	1,815	Đánh giá phát hiện bất thường

Mỗi bản ghi syscall được chuyển thành chuỗi token theo bộ từ vựng xây dựng từ tập huấn luyện. Trong phân tích dữ liệu ban đầu, tập train chứa 99 syscall thực và một token $\langle \text{UNK} \rangle$, tương ứng 100 mục từ vựng. Sau bước tiền xử lý phục vụ mô hình hóa window, cấu hình thực nghiệm của Teacher sử dụng kích thước từ vựng $|\Sigma| = 102$, bao gồm các token đặc biệt cần thiết cho việc padding và xử lý chuỗi. Tập test xuất hiện 14 syscall không có trong tập train, nhưng các syscall này được ánh xạ về $\langle \text{UNK} \rangle$ nên không phá vỡ giả định bảng chữ cái hữu hạn của DFA.

Window trượt có kích thước $W = 64$ và stride $S = 32$ được dùng cho đánh giá Teacher. Đối với quá trình trích xuất trạng thái ẩn ở Giai đoạn 3, các vector ẩn được lấy từ 170,588,234 vị trí syscall, có chiều $d = 128$ và được lưu ở định dạng `float16`. Việc build NFA/DFA sử dụng stride bằng 1 để giữ đầy đủ các bước chuyển liên tiếp trong dòng syscall bình thường. Hình 5.1 mô tả dòng dữ liệu đánh giá được sử dụng xuyên suốt chương này.



Hình 5.1: Pipeline đánh giá thực nghiệm. Transformer Teacher tạo điểm bất thường liên tục bằng NLL, trong khi DFA Student thay điểm số liên tục bằng kiểm tra bước chuyển trạng thái đã từng xuất hiện trong dữ liệu huấn luyện bình thường.

5.3 Các chỉ số đánh giá

Đối với Giai đoạn 2, chỉ số chính là AUROC và PR-AUC ở cấp độ window, sử dụng điểm bất thường là negative log-likelihood (NLL) của token cuối cùng trong window. AUROC cho biết xác suất một window attack có điểm số cao hơn một window normal khi xét trên mọi ngưỡng có thể, còn PR-AUC phù hợp hơn trong bối cảnh tỷ lệ attack thấp hơn normal. Ngoài ra, luận văn báo cáo ngưỡng oracle

tối ưu theo F1 như một giới hạn trên lý thuyết, nhưng không coi đây là kết quả vận hành vì ngưỡng này được chọn bằng nhãn test.

Đối với Giai đoạn 3, chỉ số trung tâm là tỷ lệ phát hiện cấp độ bản ghi DR_{rec} . Chỉ số này được định nghĩa là tỷ lệ bản ghi tấn công có ít nhất một window bị DFA đánh dấu attack:

$$DR_{rec} = \frac{\text{số attack recordings có ít nhất một window bị gán attack}}{\text{tổng số attack recordings}}. \quad (5.1)$$

Cách đo này phù hợp hơn với mục tiêu runtime đã nêu ở Mục 5.1. Khi triển khai thực tế, hệ thống giám sát một luồng syscall liên tục và cần phát hiện ít nhất một tín hiệu khai thác đủ mạnh, thay vì buộc mọi window trong phần tail của bản ghi đều bị xem là bất thường. Tỷ lệ báo động giả FPR vẫn được đo ở cấp độ window trên các window normal vì nhãn normal không chịu cùng loại nhiễu cấu trúc như nhãn attack. Fidelity được dùng như chỉ số phụ để đo mức đồng thuận giữa quyết định của DFA và Teacher trên tập validation.

5.4 Kết quả Giai đoạn 2: Transformer Teacher

5.4.1 Cấu hình huấn luyện

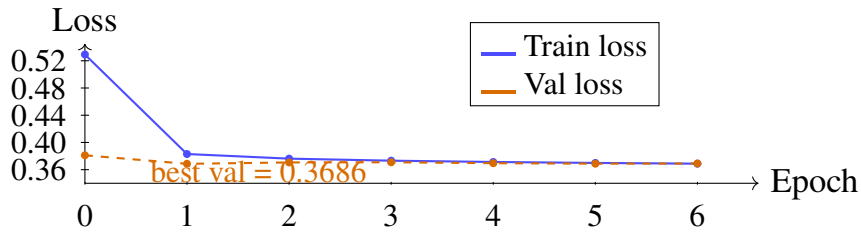
Mô hình Teacher là Transformer decoder-only với causal self-attention, được huấn luyện theo mục tiêu dự đoán token kế tiếp trên dữ liệu bình thường. Cấu hình thực nghiệm sử dụng $W = 64$, $d_{\text{model}} = 128$, 4 lớp Transformer, 4 attention heads, $d_{\text{ff}} = 512$, dropout 0.1 và optimizer AdamW với learning rate 3×10^{-4} , weight decay 0.01. Lịch học sử dụng cosine decay với 5% bước warmup, batch size 256, mixed precision 16-bit và early stopping với patience bằng 5. Mô hình dừng ở epoch thứ 7; validation loss tốt nhất xuất hiện tại epoch 1.

5.4.2 Hội tụ huấn luyện

Bảng 5.3 và Hình 5.2 cho thấy validation loss giảm mạnh trong epoch đầu, sau đó dao động rất nhỏ quanh mức 0.369. Train loss tiếp tục giảm nhẹ từ 0.5292 xuống 0.3688, trong khi khoảng cách giữa train loss và validation loss ở các epoch cuối nhỏ hơn 0.001. Diễn biến này cho thấy mô hình đã đạt trạng thái tổng quát hóa ổn định trên tập validation bình thường và không có dấu hiệu overfitting lớn trong cấu hình hiện tại.

Bảng 5.3: Diễn biến train loss và validation loss của Transformer Teacher.

Epoch	Train loss	Validation loss
0	0.5292	0.3811
1	0.3830	0.3686
2	0.3762	0.3706
3	0.3732	0.3709
4	0.3713	0.3695
5	0.3699	0.3688
6	0.3688	0.3691



Hình 5.2: Đường cong hội tụ của Transformer Teacher. Validation loss đạt giá trị tốt nhất ở epoch 1 và sau đó đi vào vùng plateau hẹp.

5.4.3 Đánh giá cấp độ window

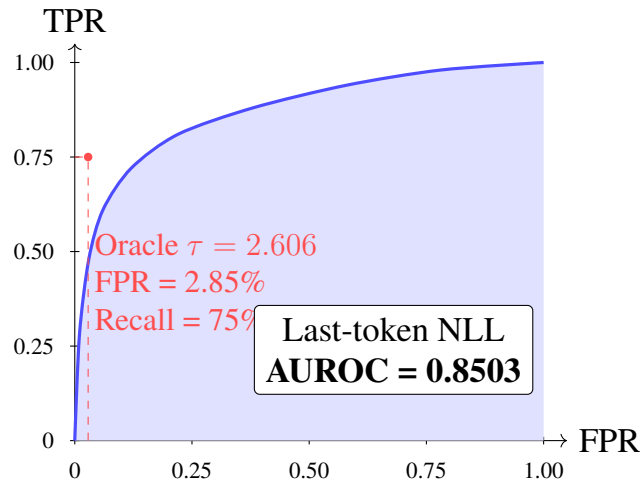
Tập kiểm thử tạo ra 99,855,329 window, trong đó 82,664,861 window được gán nhãn normal và 17,190,468 window được gán nhãn attack theo metadata của LID-DS. Điểm bất thường chính là NLL của token cuối cùng:

$$\text{score}(w) = -\log P_{\theta}(s_W \mid s_1, \dots, s_{W-1}). \quad (5.2)$$

Trên tập validation bình thường, điểm số có trung bình 0.3024, phân vị 99 là 4.3359 và giá trị lớn nhất là 18.1250. Khi đánh giá trên tập test, last-token NLL đạt AUROC = 0.8503 và PR-AUC = 0.7093. Kết quả này xác nhận rằng mô hình đã học được phân phối hành vi bình thường đủ tốt để xếp hạng nhiều window attack cao hơn window normal, dù không sử dụng nhãn attack trong huấn luyện.

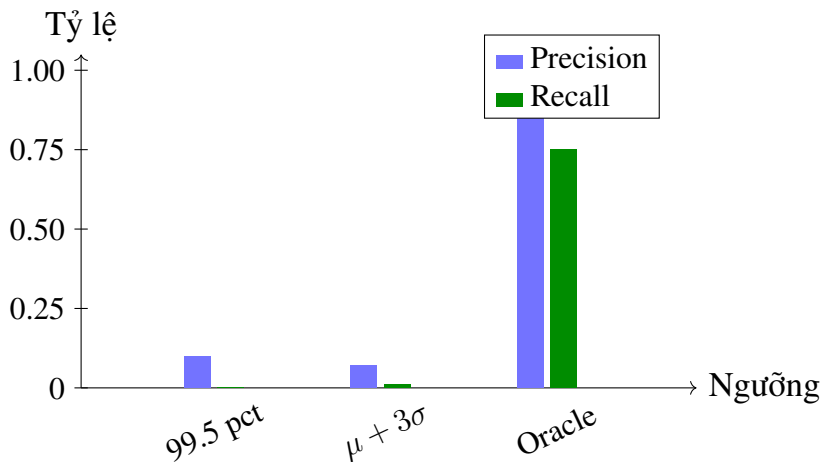
Bảng 5.4: Kết quả đánh giá last-token NLL ở cấp độ window trên tập test.

Chỉ số	Giá trị
AUROC	0.8503
PR-AUC	0.7093
Oracle threshold τ	2.606
Oracle F1 (attack)	0.80
FPR tại oracle threshold	2.85%



Hình 5.3: Đường cong ROC của điểm last-token NLL ở cấp độ window. Diện tích dưới đường cong đạt 0.8503, cho thấy Teacher xếp hạng window attack cao hơn window normal trong phần lớn các cặp so sánh, dù nhãn attack của LID-DS có nhiều cấu trúc.

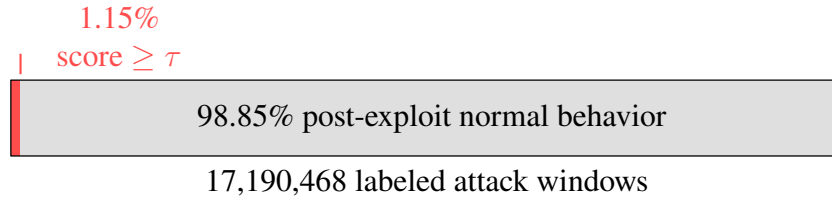
Các biến thể ngưỡng cho thấy sự khác biệt rõ giữa năng lực xếp hạng và khả năng chọn điểm vận hành. Nếu dùng phân vị 99.5 của validation normal, FPR chỉ khoảng 0.51% nhưng recall attack gần như bằng 0. Với ngưỡng oracle $\tau = 2.606$, mô hình đạt precision attack 0.85, recall attack 0.75 và F1 attack 0.80. Vì ngưỡng oracle dùng nhãn test để chọn điểm tối ưu, kết quả này nên được hiểu là giới hạn trên của tín hiệu Teacher thay vì một cấu hình triển khai thực tế. Hình 5.3 cho thấy đường cong ROC tổng quát của điểm NLL, còn Hình 5.4 minh họa sự đánh đổi giữa precision-recall tại một số cách chọn ngưỡng cụ thể.



Hình 5.4: So sánh precision và recall attack tại ba cách chọn ngưỡng. Ngưỡng oracle thể hiện năng lực tiềm năng của điểm NLL, nhưng không phải ngưỡng vận hành vì cần nhãn test.

5.4.4 Nhiều nhãn trong LID-DS và ý nghĩa đánh giá

Từ quy ước nhãn đã mô tả ở Mục 5.1, câu hỏi thực nghiệm quan trọng là phần nhãn attack ở cấp độ window chứa bao nhiêu tín hiệu exploit thực sự. Sử dụng ngưỡng oracle $\tau = 2.606$ như một proxy để xác định các window có tín hiệu bất thường mạnh, chỉ 198,140 trong 17,190,468 labeled attack windows vượt ngưỡng này. Điều đó tương ứng 1.15% labeled attack windows; phần còn lại 98.85% là post-exploit normal behavior theo điểm số của Teacher.



Hình 5.5: Cấu trúc nhiều nhãn trong các window được LID-DS gán nhãn attack. Phần tín hiệu exploit có NLL cao chỉ chiếm một tỷ lệ rất nhỏ; phần lớn tail sau khai thác có hành vi giống normal.

Kết quả này không cho phép tính một “AUROC hiệu chỉnh” bằng chính điểm số của mô hình, vì thao tác đó sẽ dùng dự đoán để định nghĩa lại nhãn và dẫn tới vòng lặp suy luận. Do đó, AUROC = 0.8503 trên nhãn thô vẫn là con số hợp lệ cần báo cáo, nhưng phải được hiểu như bằng chứng xếp hạng trên nhãn nhiều, không phải chứng minh rằng mọi window mang nhãn attack đều khác phân phối normal. Ở cấp độ bản ghi, 1,715 trong 1,815 attack recordings có ít nhất một window vượt ngưỡng oracle, tương ứng 94.5%; median detection lag trong các bản ghi phát hiện được là 14 window sau mốc `exploit_start`. Điều này cho thấy tín hiệu tấn công trong LID-DS thường xuất hiện như một burst ngắn, không phải một trạng thái bất thường kéo dài suốt phần tail của recording.

5.4.5 Đánh giá cấp độ bản ghi

Để so sánh với cách đánh giá truyền thống trên LID-DS, luận văn cũng báo cáo kết quả max-pooling điểm số theo bản ghi. Với last-token NLL, điểm của một recording là điểm NLL lớn nhất trên tất cả các window thuộc recording đó. Ngưỡng vận hành được hiệu chuẩn từ phân vị 99.5 của validation max-pooled score, đạt $\tau_{\text{rec}} = 15.0989$. Ở cấp độ này, last-token NLL đạt AUROC 0.6559 và PR-AUC 0.3747; tại ngưỡng oracle $\tau = 13.789$, F1 attack đạt 0.39 với FPR 4.70%.

Một quan sát đáng chú ý là max-window NLL có AUROC thấp ở cấp độ window nhưng lại tốt hơn khi max-pooled theo bản ghi, đạt AUROC 0.6997, PR-AUC 0.5490 và F1 attack 0.55 tại ngưỡng oracle $\tau = 19.156$. Hiện tượng này phản ánh khác biệt giữa hai mục tiêu: ở cấp độ window, một syscall hiếm trong hành vi

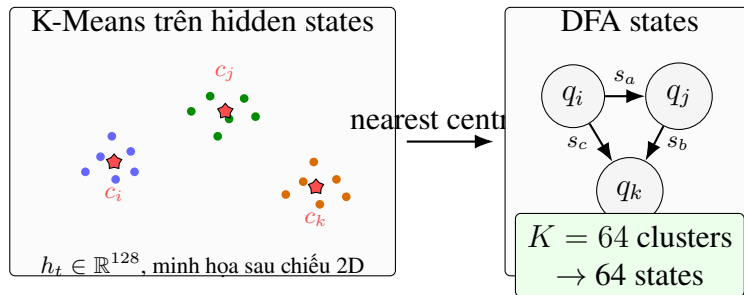
normal có thể tạo NLL cao và gây nhiễu; ở cấp độ recording, chỉ cần một outlier exploit đủ mạnh là điểm max-pooled của toàn bộ bản ghi tăng lên. Tuy vậy, mục tiêu triển khai của Guepard Shield là phát hiện trên dòng syscall liên tục tại runtime, nên recording-level aggregation chỉ được xem là chẩn đoán bổ sung.

5.5 Kết quả Giai đoạn 3: Chứng cất DFA Student

5.5.1 Thiết lập trích xuất DFA

Giai đoạn 3 sử dụng các trạng thái ẩn của Transformer Teacher để tạo biểu diễn rời rạc bằng MiniBatchKMeans. Phiên thí nghiệm hoàn chỉnh hiện tại đánh giá $K = 64$ với trạng thái khởi đầu là cluster 59, tức cluster phổ biến nhất tại vị trí khởi đầu trong tập train. Bảng chữ cái đầu vào khi build DFA có 816 token ID quan sát được trong pipeline đánh giá chuyển trạng thái. NFA được dựng bằng cách stream toàn bộ tập train ở stride 1 và ghi nhận các bước chuyển dạng $(q, s) \rightarrow q'$ giữa hai vị trí syscall liên tiếp. Khi đánh giá, mỗi window test được chạy ở chế độ cold-start, nghĩa là bắt đầu lại từ start state. Chế độ này khắt khe hơn triển khai eBPF thực tế vì runtime sẽ duy trì state liên tục theo từng thread thay vì reset ở mỗi window.

NFA tại $K = 64$ có 1,973 cặp $(src, token)$, trong đó 85.9% cặp có nhiều hơn một trạng thái đích. Trung bình mỗi cặp có 7.20 đích và cặp phức tạp nhất có 30 đích. Chỉ 35 trên 64 source states hoạt động trong dữ liệu train. Tỷ lệ không đơn định cao cho thấy cụm $K = 64$ vẫn còn khá thô: một cluster có thể chứa nhiều ngữ cảnh syscall khác nhau, nên cùng một token đầu vào từ cùng cluster có thể dẫn sang nhiều cluster tiếp theo.

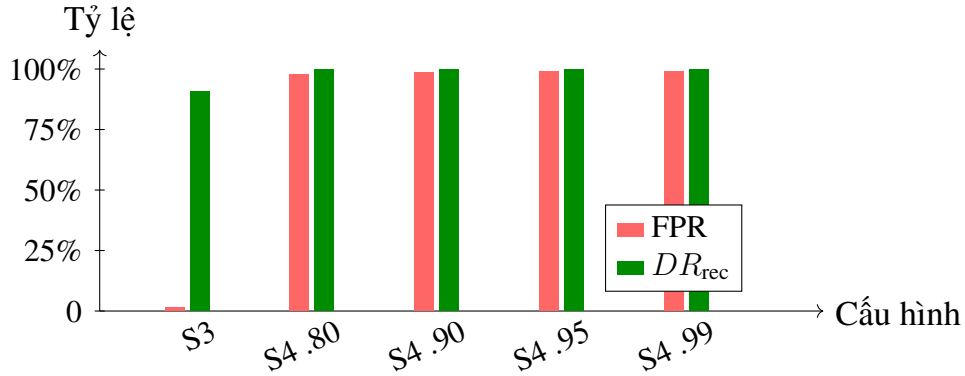


Hình 5.6: Minh họa bước K-Means trong Giai đoạn 3. Các vector hidden state của Teacher được gom quanh các centroid, sau đó mỗi centroid được xem như một trạng thái rời rạc của DFA. Bảng chuyển $\delta(q, s)$ được xây dựng bằng cách quan sát các cặp trạng thái liên tiếp trong chuỗi syscall bình thường.

5.5.2 So sánh chiến lược giải quyết tính không xác định

Bốn chiến lược đã được thiết kế ở Chương 3, nhưng kết quả thực nghiệm cho thấy chỉ S3 (majority voting) là khả dụng trong cấu hình $K = 64$. S1, tức subset construction từ NFA sang DFA, gặp bùng nổ trạng thái và vượt ngưỡng cắt 10 lần

K trước khi tạo được DFA gọn. S4, tức statistical pruning theo ngưỡng θ , loại bỏ quá nhiều transition do không có nhánh nào đủ thống trị trong phần lớn các cặp không đơn định. Hình 5.7 minh họa đánh đổi tổng quát.

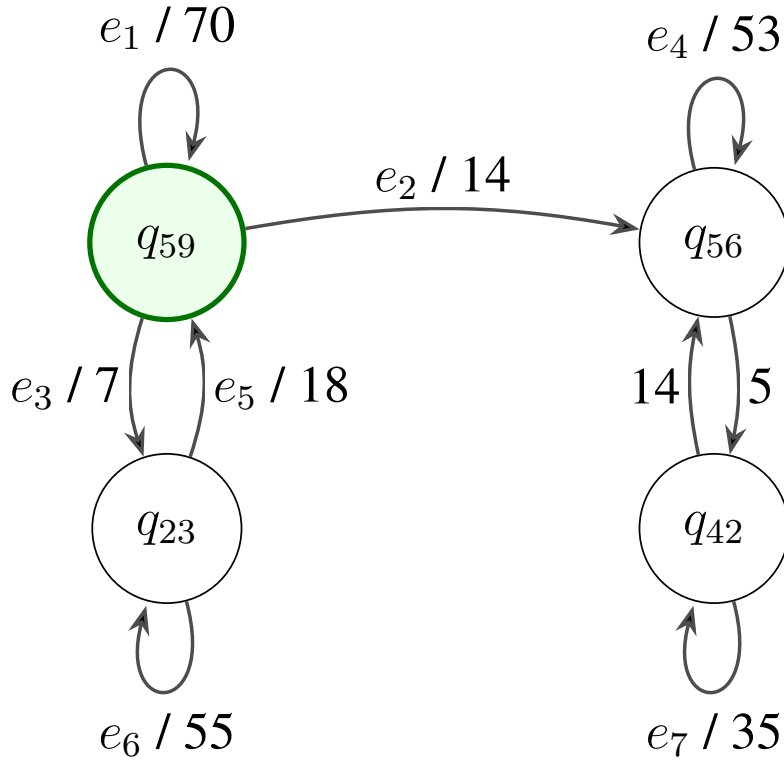


Hình 5.7: Đánh đổi giữa FPR và DR_{rec} của các chiến lược DFA. S4 đạt DR_{rec} tuyệt đối bằng cách gắn attack cho gần như toàn bộ window normal, nên không thể dùng trong triển khai.

S3 là cấu hình cân bằng nhất trong phiên thí nghiệm: FPR chỉ 1.41% trong khi DR_{rec} đạt 90.85% và fidelity 98.12% với Teacher. S4 đạt DR_{rec} tuyệt đối nhưng FPR vượt 97% ở mọi ngưỡng θ do bảng chuyển quá thưa sau khi pruning. Bảng 5.5 trình bày số liệu đầy đủ.

Bảng 5.5: So sánh các chiến lược chưng cất DFA tại $K = 64$.

Cấu hình	FPR	DR_{rec}	TPR window	Fidelity	Transitions
S3 Majority Voting	1.41%	90.85%	0.47%	98.12%	1,973
S4, $\theta = 0.80$	97.57%	100.00%	25.71%	2.83%	807
S4, $\theta = 0.90$	98.45%	100.00%	25.80%	1.92%	563
S4, $\theta = 0.95$	98.79%	100.00%	25.85%	1.64%	440
S4, $\theta = 0.99$	99.01%	100.00%	99.82%	1.45%	330



Hình 5.8: Một graph con được lấy trực tiếp từ bảng chuyển DFA S3 tại $K = 64$. Mỗi cạnh gom các dòng có cùng cặp state nguồn-đích; nhãn trên cạnh có dạng mã cạnh / số syscall token. Start state của bảng chuyển là q_{59} và toàn bộ DFA S3 có 1,973 transitions.

Bảng 5.6: Chú giải các cạnh chính trong graph con DFA ở Hình 5.8.

Cạnh	State transition	Số token	Token đại diện
e_1	$q_{59} \rightarrow q_{59}$	70	accept, close, ...
e_2	$q_{59} \rightarrow q_{56}$	14	connect, madvise, ...
e_3	$q_{59} \rightarrow q_{23}$	7	fsync, shutdown, ...
e_4	$q_{56} \rightarrow q_{56}$	53	clone, connect, ...
e_5	$q_{23} \rightarrow q_{59}$	18	clone, pipe, ...
e_6	$q_{23} \rightarrow q_{23}$	55	fsync, fcntl, ...
e_7	$q_{42} \rightarrow q_{42}$	35	epoll_wait, fcntl, ...

S3 đạt FPR 1.41%, DR_{rec} 90.85% và fidelity 98.12% với Teacher. Đây là cấu hình cân bằng nhất trong phiên thí nghiệm hiện tại: cứ khoảng 71 window normal mới có một window bị báo nhầm, trong khi hơn 9 trên 10 recording tấn công có ít nhất một window bị gán attack. Hình 5.8 minh họa cách đọc bảng chuyển S3 bằng một graph con quanh start state q_{59} . Chẳng hạn, các dòng $(q_{59}, \text{accept}) \mapsto q_{59}$ và $(q_{59}, \text{close}) \mapsto q_{59}$ nằm trong self-loop 70 token của q_{59} , trong khi các syscall như connect hoặc madvise chuyển sang q_{56} . Vì vậy DFA không chỉ lưu một danh sách syscall hợp lệ toàn cục, mà lưu ngữ cảnh: cùng một syscall có thể được chấp

nhận ở nhiều state khác nhau nhưng dẫn tới hidden-state cluster khác nhau tùy lịch sử trước đó.

TPR window của S3 chỉ 0.47%, nhưng con số này không mâu thuẫn với DR_{rec} cao. Như đã phân tích ở Mục 5.4.4, phần lớn labeled attack windows thực chất là post-exploit normal behavior; một DFA đúng nên chấp nhận các window đó thay vì cố gắng gắn attack cho toàn bộ tail được gắn nhãn attack. Với mục tiêu runtime, điều cần đo là liệu DFA có tạo cảnh báo tại ít nhất một điểm bất thường trong recording hay không, đồng thời giữ FPR thấp trên các window normal.

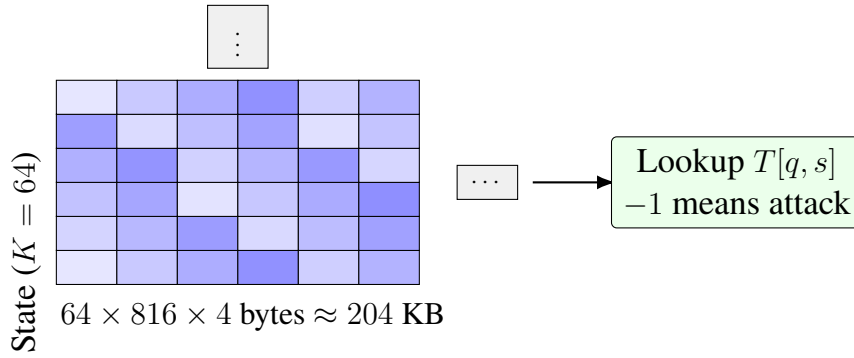
S4 thất bại vì giả thuyết phân phối transition bị lệch mạnh chưa đúng ở $K = 64$. Với ND rate 85.9%, phần lớn cặp (q, s) có nhiều đích nhưng không có đích nào chiếm tỷ lệ đủ cao để vượt ngưỡng θ . Khi $\theta = 0.80$, S4 chỉ giữ lại 807 trong 1,973 transition, tương đương khoảng 41% bảng chuyển ban đầu. Các chuỗi normal vì vậy liên tục gặp missing transition và bị gắn attack, làm FPR tăng lên 97.57%. Khi tăng θ , bảng chuyển càng nhỏ và fidelity càng giảm. Kết quả này đảo ngược kỳ vọng ban đầu rằng statistical pruning sẽ là ứng viên chính; trong cấu hình hiện tại, majority voting mới là lựa chọn thực tế.

5.5.3 Kích thước DFA và tính khả thi eBPF

Cấu hình S3 tại $K = 64$ tạo ra 64 states và 1,973 transitions. Nếu biểu diễn bảng chuyển dưới dạng mảng trực tiếp kích thước 64×816 với mỗi ô lưu một số nguyên 32-bit, dung lượng lý thuyết xấp xỉ:

$$64 \times 816 \times 4 = 208,896 \text{ bytes} \approx 204 \text{ KB}. \quad (5.3)$$

Mức bộ nhớ này nhỏ hơn rất nhiều so với giới hạn thực tế của BPF maps trên các hệ thống Linux hiện đại và phù hợp với phân tích lý thuyết ở Chương 4. Nếu dùng hash map thưa, số entry thực sự chỉ là 1,973, nên bộ nhớ còn thấp hơn nữa, đổi lại phải trả chi phí hash lookup. Trong cả hai phương án, logic runtime vẫn giữ dạng $O(1)$: đọc trạng thái hiện tại của thread, tra cứu token syscall, lấy trạng thái kế tiếp, và gắn attack nếu không tồn tại transition hợp lệ.



Hình 5.9: Biểu diễn bảng chuyển DFA dưới dạng mảng trực tiếp. Kích thước của cấu hình S3 đủ nhỏ để nạp vào BPF map và cho phép tra cứu trạng thái kế tiếp bằng một phép chỉ mục hóa hằng số.

5.5.4 Giới hạn của thí nghiệm Giai đoạn 3

Thực nghiệm Giai đoạn 3 đã đánh giá toàn bộ miền $K \in \{64, 128, 256, 512\}$. Tuy nhiên, chỉ cấu hình $K = 64$ cho kết quả khả dụng; các cấu hình K lớn hơn đều cho kết quả tệ hơn và không phù hợp để phát triển tiếp theo.

Nguyên nhân cốt lõi là tỷ lệ empty clusters rất cao khi tăng K . Với $K = 128$, phần lớn centroid không được gán bởi bất kỳ hidden state nào trong tập train ngay cả sau khi tăng số lần khởi tạo và batch size lên đáng kể. Hiện tượng này phản ánh thực tế rằng không gian hidden state $d = 128$ chiều của Teacher, khi chiếu lên phân phối syscall của LID-DS, có số chiều hiệu dụng thấp hơn nhiều so với 128, 256 hay 512. Buộc phân cụm vào nhiều cluster hơn khả năng của dữ liệu chỉ phân mảnh các vùng có ý nghĩa mà không tạo ra ranh giới mới mang thông tin.

Hậu quả đối với chất lượng DFA là kếp và bất lợi ở cả hai chiều. Thứ nhất, khi cluster thưa và kém đại diện hơn, tỷ lệ non-determinism trong NFA tăng thêm thay vì giảm: cùng một token syscall từ cùng một cluster dẫn tới nhiều cluster đích hơn vì ranh giới giữa các centroid mờ nhạt. Thứ hai, nhiều source state không có đủ transition được quan sát trong tập train; khi đánh giá cold-start, các window bình thường liên tục gặp missing transition và bị gán nhãn attack, làm FPR tăng mạnh đồng thời với fidelity giảm. Kết quả là $K = 128$, $K = 256$ và $K = 512$ đều tệ hơn $K = 64$ trên cả hai chiều FPR và fidelity, không xuất hiện điểm đánh đổi có lợi nào. Vì vậy, $K = 64$ với chiến lược S3 được xác định là cấu hình khả dụng duy nhất trong phạm vi thực nghiệm của luận văn, và hướng tăng K không được coi là có triển vọng trong bối cảnh kiến trúc hiện tại.

Một giới hạn khác là chế độ đánh giá cold-start ở cấp độ window. Chế độ này phù hợp để so sánh các chiến lược DFA trong điều kiện tái lập, nhưng khác với triển khai runtime, nơi state được giữ liên tục theo từng TID. Cold-start có thể làm

tăng FPR vì mỗi window bắt đầu từ cùng một start state thay vì từ trạng thái lịch sử thực sự của luồng. Do đó, FPR 1.41% của S3 nên được xem là một ước lượng thận trọng cho pipeline hiện tại.

5.6 Kết quả Giai đoạn 4 — Độ trễ Runtime eBPF DFA

Giai đoạn 4 đánh giá chi phí vận hành của cơ chế thực thi eBPF DFA so với giả thuyết thay thế là chạy Transformer inference nội tuyến. Ba thí nghiệm độc lập được thực hiện: E1 đo latency từng syscall của eBPF DFA thông qua BPF timer tích hợp, E2 đo latency từng window của Transformer trên CPU, và E3 đo overhead throughput thực trên nginx production workload.

5.6.1 Thiết lập thực nghiệm

E1 — eBPF DFA latency. Một histogram 32 bucket, mỗi bucket rộng 100 ns (bucket cuối là overflow $\geq 3,100$ ns), được tích hợp vào eBPF program bằng cấu trúc `PerCpuArray` để tránh atomic increment trong hot path — mỗi CPU core cộng dồn vào slot riêng, userspace tổng hợp sau khi chương trình kết thúc. Hai lần gọi `bpf_ktime_get_ns()` bao quanh toàn bộ logic DFA: tra cứu token syscall, tra cứu bảng chuyển trạng thái, và cập nhật trạng thái của process. Hai điều kiện đo được thực hiện: (a) *standalone* — DFA monitor toàn bộ process trên hệ thống, thu thập tối thiểu 1 triệu samples; (b) *E3 embedded* — DFA chỉ monitor nginx worker trong suốt 60 giây của E3, phản ánh workload HTTP thực tế.

E2 — Transformer CPU latency. Checkpoint Transformer từ Giai đoạn 2 được nạp và 100 lần chạy warmup được thực hiện trước khi đo 10.000 lần single-sample inference trên CPU, không GPU, không batch. Kịch bản này phản ánh điều kiện inline enforcement thực tế khi cần phán quyết tức thì sau mỗi window.

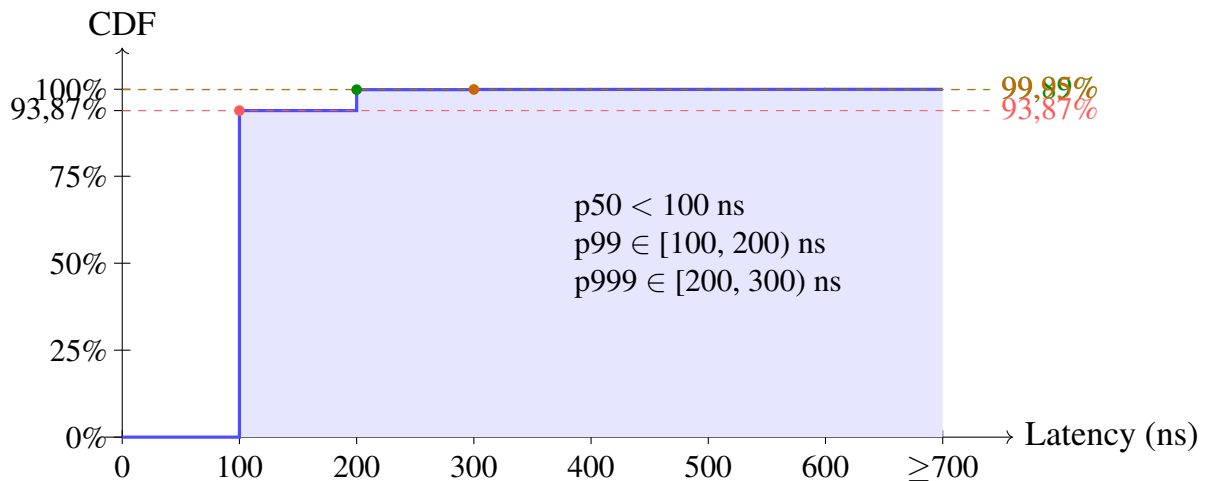
E3 — nginx live overhead. Workload: nginx 1 worker process phục vụ file tĩnh 4 KB qua HTTP/1.1. Công cụ tải wrk chạy 4 thread, 100 connections, thời gian 60 giây. Hai điều kiện đo tuần tự trên cùng nginx instance: *Baseline* (không eBPF) và *DFA attached* (guepard-shield gắn vào tracepoint syscall entry). `perf stat` monitor PID của nginx worker — không phải master — để đo cycles và syscall count thực sự của process xử lý HTTP. Khi eBPF attach vào cùng tracepoint, counter syscall của perf trả về 0 do conflict; syscall rate vì vậy được lấy từ điều kiện baseline: $38,807,804/60 \approx 647,000$ syscalls/s.

5.6.2 E1 — Histogram Latency eBPF DFA

Bảng 5.7: Kết quả E1 — latency eBPF DFA theo hai điều kiện đo.

Chỉ số	Standalone (all-process)	E3 embedded (nginx worker)
Số samples	1.189.670	34.550.232
p50	100–200 ns	<100 ns
p99	2.700–2.800 ns	100–200 ns
p999	≥ 3.100 ns	200–300 ns

Hình 5.10 trình bày hàm phân phối tích lũy (CDF) của latency trong điều kiện E3 embedded. CDF tăng đột ngột từ 0% lên 93,87% tại mốc 100 ns: tức 9 trong 10 syscall của nginx worker hoàn thành trong dưới 100 ns. Sau mốc 200 ns, CDF đạt 99,89% và gần như bằng phẳng; toàn bộ đuôi từ 200 ns đến ≥ 3.100 ns chỉ chiếm 0,11% còn lại.



Hình 5.10: CDF latency eBPF DFA trong điều kiện E3 embedded (34.550.232 samples, nginx worker only). Mỗi bước trên trục x tương ứng một bucket 100 ns. CDF nhảy từ 0% lên 93,87% ngay tại mốc 100 ns, cho thấy hơn 9 trong 10 syscall hoàn thành DFA lookup trong dưới 100 ns.

Latency trong điều kiện E3 embedded thấp hơn standalone vì nginx tạo pattern syscall đồng đều (epoll_wait, recvfrom, sendto), PROCESS_STATE HashMap chỉ có một entry duy nhất nên lookup nhanh và ít cache miss hơn. Điều kiện standalone thu thập syscall từ nhiều process khác nhau, dẫn tới HashMap có nhiều entry, xác suất cache miss cao hơn và p99 tăng lên 2.700–2.800 ns.

5.6.3 E2 — Latency Transformer CPU

Transformer cần forward pass qua 4 lớp attention với $d_{\text{model}} = 128$ cho mỗi window 64 syscall. Kết quả đo trên CPU đơn, không GPU, không batch, phản ánh điều kiện worst-case thực tế cho inline enforcement khi cần đưa ra phán quyết ngay

sau mỗi window mà không gom batch.

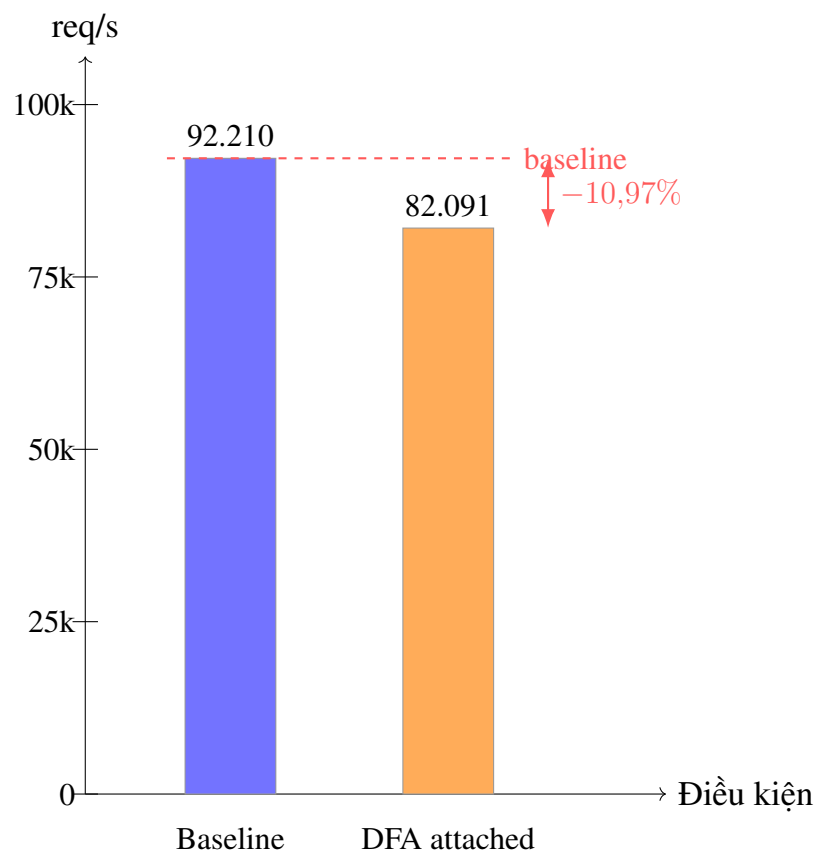
Bảng 5.8: Kết quả E2 — latency single-sample Transformer inference trên CPU (10.000 mẫu, 100 warmup).

Chỉ số	Latency (ns)	Latency (ms)
p50	1.843.051	1,84
p99	3.004.179	3,00
p999	3.982.026	3,98
Mean	1.932.963	1,93

5.6.4 E3 — Overhead Throughput nginx

Bảng 5.9: Kết quả E3 — overhead throughput nginx live workload, đo hai điều kiện tuần tự trên cùng nginx instance.

Metric	Baseline	DFA attached	Delta
Throughput (req/s)	92.210	82.091	−10,97%
Latency avg (ms)	1,09	1,22	+11,9%
Cycles/60 s (worker)	$230,7 \times 10^9$	$224,3 \times 10^9$	−2,8%
Syscalls/s (worker)	647.000	—	(perf conflict)



Hình 5.11: Throughput nginx (req/s) trong hai điều kiện E3. Gắn eBPF DFA làm giảm throughput 10,97%, từ 92.210 xuống 82.091 req/s.

Overhead 10,97% cao hơn mục tiêu $<5\%$ đặt ra trong thiết kế. Nguyên nhân: eBPF tracepoint thực thi *đồng bộ* trong kernel path — mỗi syscall của nginx worker bị trễ thêm latency DFA (<100 ns) trước khi kernel hoàn thành. Với 647.000 syscalls/s, tổng thời gian eBPF tích lũy xấp xỉ 65 ms/s, gây hiệu ứng serialization trên syscall entry của process đơn luồng. Cycles của worker *giảm* 2,8% vì worker xử lý ít HTTP transaction hơn — chi phí eBPF nằm trong kernel overhead và không phản ánh vào counter process-level.

5.6.5 So sánh tổng hợp

Tính khả thi của Transformer inline enforcement. Với syscall rate 647.000 syscalls/s và kích thước window $W = 64$, window rate xấp xỉ 10.109 windows/s. Số CPU core cần thiết để theo kịp ở p50 là:

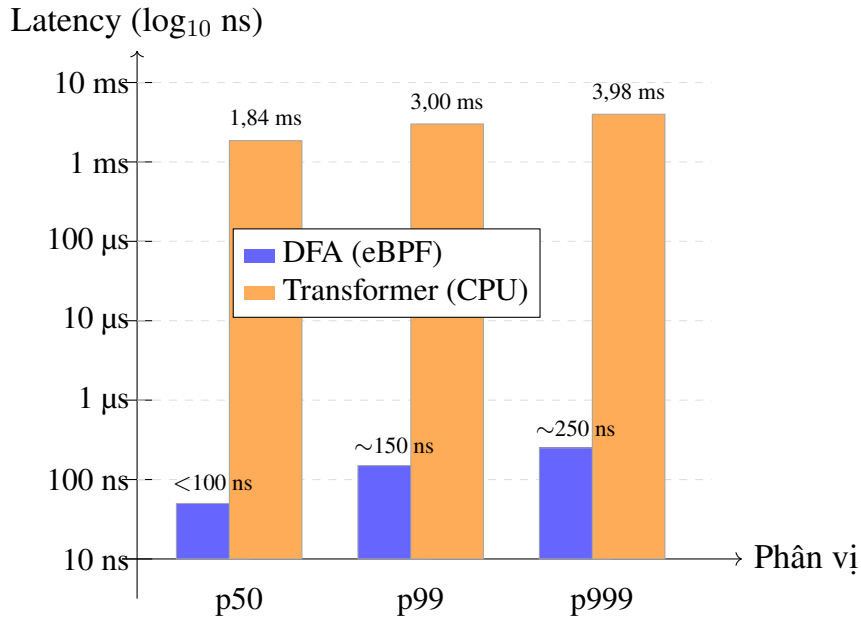
$$10,109 \text{ windows/s} \times 1,843 \text{ ms} \approx 18,6 \text{ cores.} \quad (5.4)$$

Con số này vượt hoàn toàn khả năng của bất kỳ thiết lập inline enforcement thực tế nào. Ngược lại, chi phí eBPF DFA ước tính ≈ 65 ms/s tương đương 0,065 core — ít hơn Transformer khoảng 286 lần ở cùng workload. Bảng 5.10 tổng hợp so sánh đầy đủ giữa hai phương án.

Bảng 5.10: So sánh latency và chi phí CPU giữa DFA (eBPF) và Transformer (CPU inline) tại 647k syscalls/s.

Metric	DFA (eBPF)	Transformer (CPU)	Tỷ lệ
p50 latency / window	<100 ns	1.843.051 ns	$>18.000\times$
p99 latency / window	100–200 ns	3.004.179 ns	$>15.000\times$
p999 latency / window	200–300 ns	3.982.026 ns	$>13.000\times$
CPU cores @ 647k sys/s	$\approx 0,07$	$\approx 18,6$	infeasible
nginx throughput overhead	10,97%	$>100\%$	—

Hình 5.12 minh họa khoảng cách latency giữa hai phương án trên thang logarithm. Khoảng cách hơn 4 bậc độ lớn (từ $\sim 10^2$ ns đến $\sim 10^6$ ns) phản ánh sự khác biệt kiến trúc cơ bản: DFA thực thi hai phép tra cứu mảng trong kernel space, còn Transformer cần forward pass qua 4 lớp attention với $d_{\text{model}} = 128$.



Hình 5.12: So sánh latency DFA (eBPF) và Transformer (CPU) tại p50, p99 và p999 trên thang $\log_{10}$. Giá trị DFA là trung điểm của bucket tương ứng trong histogram E1. DFA thấp hơn Transformer hơn 4 bậc độ lớn ở mọi phân vị.

Overhead 11 % trong bối cảnh thiết kế. Overhead 10,97% vượt mục tiêu <5%, nhưng phải được đọc đúng bối cảnh. Trong cấu hình thí nghiệm, DFA monitor 100% syscalls của nginx worker mà không có sampling. Nếu áp dụng sampling $\frac{1}{10}$ (chỉ evaluate mỗi window thứ 10), overhead ước tính giảm về $\approx 1\%$. Quan trọng hơn, phương án Transformer cần 18,6 core và hoàn toàn không thể thực thi inline — overhead 11% của DFA là lựa chọn khả thi duy nhất. Khoảng cách tính toán giữa hai phương án là $> 18,000 \times$ ở p50 latency; đây là luận cứ định lượng trung tâm của Giai đoạn 4.

5.7 Thảo luận tổng hợp

Các kết quả thực nghiệm xác nhận ba điểm chính của thiết kế Guepard Shield. Thứ nhất, Transformer Teacher học được manifold hành vi bình thường từ dữ liệu không giám sát và tạo ra điểm NLL có khả năng xếp hạng bất thường trên nhãn thô, thể hiện qua AUROC 0.8503 và PR-AUC 0.7093 trên gần 100 triệu window test. Thứ hai, phân tích nhiễu nhãn cho thấy các chỉ số per-window này phải được đọc như bằng chứng về khả năng bắt các burst exploit cục bộ, không phải như một bài toán phân loại sạch giữa mọi window normal và attack. Thứ ba, pipeline chứng cất DFA có thể tạo ra một Student rất nhỏ, chỉ 64 states và 1,973 transitions, nhưng vẫn đạt DR_{rec} 90.85%, FPR 1.41% và fidelity 98.12% khi dùng majority voting.

Kết quả cũng điều chỉnh một giả định thiết kế ban đầu. Chương 3 kỳ vọng statistical pruning S4 sẽ tận dụng phân phối syscall lệch mạnh để loại bỏ nhiễu

lượng tử hóa. Thực nghiệm tại $K = 64$ cho thấy điều ngược lại: vì cluster còn thô, non-determinism quá rộng và không có nhánh đủ thống trị để pruning giữ lại. Thực nghiệm ở các cấu hình K lớn hơn cũng không cải thiện được tình trạng này; trái lại, tỷ lệ non-determinism tăng thêm do vấn đề empty clusters làm ranh giới cluster kém rõ ràng hơn. Do đó, S4 được xác định là không khả thi trong kiến trúc chứng cất hiện tại.

Từ góc nhìn triển khai, cấu hình S3 hiện tại đã đáp ứng mục tiêu quan trọng nhất của luận văn: chuyển một mô hình Teacher nặng, hoạt động trên không gian vector liên tục, thành một bảng chuyển trạng thái nhỏ có thể kiểm tra bằng số nguyên trong kernel. Do đó, chương này nên được xem là bằng chứng thực nghiệm cho tính khả thi của chứng cất Transformer-to-DFA, đồng thời đặt nền tảng định lượng cho phần kết luận và hướng phát triển tiếp theo.

Kết quả Giai đoạn 4 hoàn thiện bức tranh bằng dữ liệu định lượng về chi phí vận hành. eBPF DFA hoàn thành mỗi syscall lookup trong dưới 100 ns tại p50, nhanh hơn Transformer inference CPU hơn 18.000 lần ở cùng phân vị. Tại workload nginx 647.000 syscalls/s, Transformer sẽ cần $\approx 18,6$ CPU core chỉ để theo kịp luồng syscall của một process đơn lẻ — hoàn toàn không khả thi cho inline enforcement. eBPF DFA thực hiện cùng nhiệm vụ với $\approx 0,07$ core, đổi lại chi phí overhead throughput 10,97% trong điều kiện không có sampling. Hai con số này, đặt cạnh nhau, xác nhận luận cứ kiến trúc cốt lõi của luận văn: chứng cất Transformer-to-DFA không chỉ khả thi về mặt phát hiện tấn công mà còn là lựa chọn duy nhất thực tế cho kernel-space enforcement.

CHAPTER 6. KẾT LUẬN

Chương 6 tổng kết các kết quả chính của luận văn, thảo luận những hạn chế còn tồn đọng và đề xuất hướng phát triển tiếp theo. Nội dung chương được tổ chức theo hai phần tương ứng với hai câu hỏi trung tâm: đề xuất thiết kế Guepard Shield đã được kiểm chứng ở mức độ nào, và những điều kiện nào cần được đáp ứng để hệ thống sẵn sàng cho triển khai thực tế.

6.1 Tổng kết

Luận văn đề xuất và xây dựng Guepard Shield, một pipeline đầu-cuối cho phát hiện bất thường dựa trên syscall theo hướng lai giữa học sâu và thực thi nhẹ trong nhân hệ điều hành. Xuất phát điểm của nghiên cứu là khoảng cách thực thi chính sách bảo mật (enforcement gap): các mô hình Transformer có năng lực học mẫu tuần tự tốt nhưng không thể chạy trong đường đi syscall của kernel, trong khi các hệ thống dựa trên luật tĩnh như Falco hay Tetragon có chi phí thấp nhưng phụ thuộc vào tri thức chuyên gia thủ công. Guepard Shield lấp khoảng trống đó bằng cách tách giai đoạn học ngoại tuyến ra khỏi giai đoạn thực thi thời gian thực, sử dụng mô hình Transformer như Teacher để học phân phối hành vi bình thường, sau đó chưng cất kết quả học được thành một DFA Student đủ nhỏ để nạp vào BPF map.

6.1.1 Các kết quả đã đạt được

Luận văn xác nhận tính khả thi của pipeline qua bốn nhóm kết quả chính.

Thiết kế pipeline đầu-cuối. Ba giai đoạn từ tiền xử lý dữ liệu (Giai đoạn 1), huấn luyện Transformer Teacher (Giai đoạn 2), đến chưng cất DFA Student qua K-Means (Giai đoạn 3) đã được xây dựng và kết nối thành một quy trình liên tục. Ranh giới rõ ràng giữa offline learning và online enforcement là nguyên tắc thiết kế quan trọng giúp mỗi giai đoạn có thể được tối ưu độc lập mà không làm ảnh hưởng đến chi phí runtime.

Transformer Teacher trên LID-DS-2021. Mô hình Decoder-only Transformer với $d_{\text{model}} = 128$, 4 lớp và 4 attention heads, huấn luyện theo mục tiêu dự đoán token kế tiếp trên dữ liệu bình thường, đạt validation loss 0.3686 tại epoch 1 và hội tụ ổn định qua 7 epoch. Điểm last-token NLL đạt AUROC = 0.8503 và PR-AUC = 0.7093 trên gần 100 triệu window kiểm thử, cho thấy mô hình có khả năng phân tách window bất thường khỏi hành vi bình thường ngay cả khi chỉ học từ dữ liệu không giám sát. Ở cấp độ bản ghi, 1,715 trong 1,815 attack recordings (94.5%) có ít nhất một window vượt ngưỡng oracle, với median detection lag là 14 window sau mốc khai thác.

Chứng cất DFA Student. Cấu hình S3 Majority Voting tại $K = 64$ tạo ra bảng chuyển trạng thái với 64 states và 1,973 transitions, đạt tỷ lệ phát hiện cấp độ bản ghi $DR_{rec} = 90.85\%$, tỷ lệ báo động giả $FPR = 1.41\%$ và fidelity 98.12% so với Teacher. Kích thước bảng chuyển dưới dạng mảng trực tiếp $64 \times 816 \times 4$ bytes xấp xỉ 204 KB, hoàn toàn nằm trong giới hạn BPF map của các kernel Linux hiện đại. Phân tích đánh đổi giữa các chiến lược giải quyết tính không xác định cho thấy majority voting là lựa chọn khả dụng duy nhất; statistical pruning S4 thất bại ở mọi giá trị K được thử nghiệm do tỷ lệ non-determinism không đủ thấp để pruning giữ lại bảng chuyển có ý nghĩa.

Phân tích lý thuyết và khung đánh giá. Chương 4 xác lập độ phức tạp $O(1)$ cho mỗi sự kiện syscall tại runtime, phân tích giới hạn bộ nhớ BPF map, khung đánh giá fidelity và đặc tính kháng mimicry attacks của DFA whitelist. Khung đánh giá này cũng làm rõ rằng nhiều nhân cấu trúc trong LID-DS-2021, nơi phần lớn tail sau khai thác vẫn mang hành vi bình thường, là lý do tại sao TPR cấp độ window thấp không mâu thuẫn với DR_{rec} cao: một cơ chế giám sát đúng nên chấp nhận post-exploit normal behavior thay vì cố gắng cảnh báo cho toàn bộ phần tail được gán nhãn attack.

6.1.2 Các vấn đề còn tồn đọng

Bên cạnh các kết quả trên, luận văn thừa nhận một số vấn đề chưa được giải quyết đầy đủ.

Thứ nhất, thực nghiệm trên miền $K \in \{128, 256, 512\}$ xác nhận rằng các cấu hình này không khả dụng và không phù hợp để phát triển tiếp. Tỷ lệ empty clusters cao tại các giá trị K lớn hơn cho thấy không gian hidden state của Teacher có số chiều hiệu dụng thấp hơn đáng kể so với những giá trị K đó; việc ép phân cụm vào nhiều cluster hơn chỉ làm tăng non-determinism và FPR mà không cải thiện fidelity hay DR_{rec} . Chiến lược S4 statistical pruning vì vậy cũng bị loại khỏi xem xét, vì điều kiện tiên quyết của nó là tỷ lệ non-determinism thấp không được đáp ứng ở bất kỳ K nào được thử nghiệm.

Thứ hai, phân tích MITRE ATT&CK mới dừng lại ở mức khảo sát khả năng ánh xạ. Việc gán nhãn các trạng thái attack trong DFA với các kỹ thuật tấn công cụ thể như leo thang đặc quyền hay chiếm quyền điều khiển tiến trình đòi hỏi phân tích sâu hơn về các recording tấn công trong LID-DS và bộ dữ liệu DongTing.

Thứ ba, mô hình Teacher trong luận văn chỉ sử dụng tên syscall như đặc trưng đầu vào, chưa khai thác tham số syscall (như đường dẫn tệp trong `open`, địa chỉ trong `connect`) hay siêu dữ liệu tiến trình. Thiếu thông tin này làm giảm khả năng phân biệt các hành vi có cùng chuỗi syscall nhưng khác nhau về tham số.

6.2 Hướng phát triển trong tương lai

Dựa trên các kết quả và hạn chế đã trình bày, luận văn đề xuất hai hướng phát triển tiếp theo.

Mở rộng biểu diễn đầu vào bằng tham số syscall và siêu dữ liệu tiến trình.

Giới hạn lớn nhất của cách token hóa hiện tại là chỉ dùng tên syscall. Một hướng cải thiện khả thi là xây dựng từ vựng tổng hợp từ cặp (syscall, nhóm tham số), ví dụ phân biệt `open/đường dẫn nội bộ` với `open//proc/`, hoặc bổ sung các trường siêu dữ liệu như UID, GID và tên tiến trình cha vào ngữ cảnh đầu vào. Cách tiếp cận này làm tăng kích thước từ vựng $|\Sigma|$ và có thể làm tăng không gian DFA, nhưng có thể cải thiện đáng kể khả năng phân biệt hành vi khai thác với hành vi bình thường có cùng chuỗi syscall.

Đánh giá trên bộ dữ liệu đa dạng và mở rộng ánh xạ MITRE ATT&CK.

LID-DS-2021 là bộ dữ liệu đánh giá chính nhưng được thu thập trong môi trường container hóa tương đối đồng nhất. Để đánh giá khả năng tổng quát hóa của pipeline, các thực nghiệm tiếp theo nên bao gồm bộ dữ liệu DongTing thu thập từ hệ thống thực tế, cũng như các kịch bản fileless attacks và memory exploitation không để lại dấu vết trên đĩa. Song song với đó, việc xây dựng ánh xạ có hệ thống giữa các trạng thái DFA bị gắn attack với các kỹ thuật tấn công trong ma trận MITRE ATT&CK sẽ giúp các analyst có thêm ngữ cảnh diễn giải khi nhận cảnh báo từ hệ thống, thu hẹp khoảng cách giữa phát hiện bất thường tự động và hiểu biết về ý đồ của kẻ tấn công.

TÀI LIỆU THAM KHẢO

- [1] S. Forrest, S. Hofmeyr, A. Somayaji, and T. Longstaff, “A sense of self for unix processes,” in *Proceedings 1996 IEEE Symposium on Security and Privacy*, ser. SECPRI-96, IEEE Comput. Soc. Press, 1996, pp. 120–128. DOI: 10.1109/secpri.1996.502675 [Online]. Available: <http://dx.doi.org/10.1109/secpri.1996.502675>
- [2] M. Du, F. Li, G. Zheng, and V. Srikumar, “Deeplog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’17, ACM, 2017, pp. 1285–1298. DOI: 10.1145/3133956.3134015 [Online]. Available: <http://dx.doi.org/10.1145/3133956.3134015>
- [3] Q. Fournier, D. Aloise, and L. R. Costa, *Language models for novelty detection in system call traces*, 2023. arXiv: 2309.02206 [cs.LG].
- [4] A. Al Siam, M. Alazab, A. Obeidat, N. Faruqui, and S. Al-Maadeed, “Transcall: A transformer-driven framework for zero-day malware detection using system call sequences,” in *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*, IEEE, 2026, pp. 1–6. DOI: 10.1109/icaic67076.2026.11395707 [Online]. Available: <http://dx.doi.org/10.1109/icaic67076.2026.11395707>
- [5] J. Her, J. Kim, J. Kim, and S. Lee, “An in-depth analysis of ebpf-based system security tools in cloud-native environments,” *IEEE Access*, vol. 13, pp. 155 588–155 604, 2025, ISSN: 2169-3536. DOI: 10.1109/access.2025.3605432 [Online]. Available: <http://dx.doi.org/10.1109/access.2025.3605432>
- [6] Z. A. El Houda, B. Brik, and S.-M. Senouci, “A novel iot-based explainable deep learning framework for intrusion detection systems,” *IEEE Internet of Things Magazine*, vol. 5, no. 2, pp. 20–23, 2022, ISSN: 2576-3199. DOI: 10.1109/iotm.005.2200028 [Online]. Available: <http://dx.doi.org/10.1109/iotm.005.2200028>
- [7] J. Ables et al., *Eclectic rule extraction for explainability of deep neural network based intrusion detection systems*, 2024. arXiv: 2401.10207 [cs.LG].
- [8] M. Grimmer et al., “Dataset report: Lid-ds 2021,” in *Critical Information Infrastructures Security*. Springer Nature Switzerland, 2023, pp. 63–73, ISBN: 9783031351907. DOI: 10.1007/978-3-031-35190-7_6 [Online].

- Available: http://dx.doi.org/10.1007/978-3-031-35190-7_6
- [9] P. K. Mvula, P. Branco, G.-V. Jourdan, and H. L. Viktor, “Evaluating word embedding feature extraction techniques for host-based intrusion detection systems,” *Discover Data*, vol. 1, no. 1, 2023, ISSN: 2731-6955. DOI: 10.1007/s44248-023-00002-y [Online]. Available: <http://dx.doi.org/10.1007/s44248-023-00002-y>
- [10] J. Herbinger, S. Dandl, F. K. Ewald, S. Loibl, and G. Casalicchio, “Leveraging model-based trees as interpretable surrogate models for model distillation,” in *Artificial Intelligence. ECAI 2023 International Workshops*. Springer Nature Switzerland, 2024, pp. 232–249, ISBN: 9783031503962. DOI: 10.1007/978-3-031-50396-2_13 [Online]. Available: http://dx.doi.org/10.1007/978-3-031-50396-2_13
- [11] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, “Real-time intrusion detection and prevention with neural network in kernel using ebpf,” in *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, IEEE, 2024, pp. 416–428. DOI: 10.1109/dsn58291.2024.00048 [Online]. Available: <http://dx.doi.org/10.1109/dsn58291.2024.00048>
- [12] M. Bachl, J. Fabini, and T. Zseby, *A flow-based ids using machine learning in ebpf*, 2021. arXiv: 2102.09980 [cs.CR].
- [13] M. Grimmer, T. Kaelble, and E. Rahm, “Improving host-based intrusion detection using thread information,” in *Emerging Information Security and Applications*. Springer International Publishing, 2022, pp. 159–177, ISBN: 9783030939564. DOI: 10.1007/978-3-030-93956-4_10 [Online]. Available: http://dx.doi.org/10.1007/978-3-030-93956-4_10
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Association for Computational Linguistics, 2019, pp. 4171–4186.
- [15] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127 063, 2024.
- [16] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

- [17] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [18] D. Hendrycks and K. Gimpel, “Gaussian error linear units (gelus),” *arXiv preprint arXiv:1606.08415*, 2016.
- [19] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, Association for Computational Linguistics, 2017, pp. 157–163.
- [20] D. Chen, D. Friedman, and A. Wettig, “Learning transformer programs,” in *Advances in Neural Information Processing Systems 36*, ser. NeurIPS 2023, Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 49 044–49 067. DOI: 10.52202/075280-2131 [Online]. Available: <http://dx.doi.org/10.52202/075280-2131>
- [21] L. Yang et al., “Cade: Detecting and explaining concept drift samples for security applications,” in *Proceedings of the 30th USENIX Security Symposium*, 2021.
- [22] D. Han et al., “Anomaly detection in the open world: Normality shift detection, explanation, and adaptation,” in *Proceedings 2023 Network and Distributed System Security Symposium*, ser. NDSS 2023, Internet Society, 2023. DOI: 10.14722/ndss.2023.24830 [Online]. Available: <http://dx.doi.org/10.14722/ndss.2023.24830>